IES: AI Engineering Knowledge Base & Rebuilding Guide

Purpose: This document contains the EXHAUSTIVE architectural details, file structures, schemas, and core implementation code for the "Innovative Engineering Solutions" (IES) platform. It is designed to be fed into an AI coding agent to instantly grant it 100% context of the application, enabling it to rebuild, extend, or maintain the site without hallucinating.

Architecture Overview

1. **Frontend Landing Page (index.html)**: High-performance, SEO-friendly HTML with vanilla JS and CSS.
2. **Secure Admin CRM (admin.html)**: Single-Page Application (SPA) mechanics built entirely with Vanilla JavaScript (ES10+). It features custom tab-routing, dynamic DOM updates, and complex modular states.
3. **Backend as a Service (Firebase v10)**: Utilizes Firestore for NoSQL document storage, Firebase Auth for security, and Firebase Storage for handling images (like inventory tool photos).

---

2. Firebase Database Schema Design

- **internal_orders**: internalOrderNo, customer, poNo, drgNo, description, qty, deliveryDate, status, prices, drawings.
- **roster_assignments**: employeeId, date, department, machine, ioNo, productionCostUnit, totalProductionValue.
- **projects**: projectId, customer, description, status, members.
- **inventory**: Items, stock, and transaction logs.

---

3. Core Implementation Files

1. Complete Directory Structure

```
Innovative Engineering Solutions/
├── index.html                  (Public Marketing Site)
├── admin.html                  (Internal CRM Portal)
├── firebase-config.js          (Firebase configuration & initialization)
├── styles.css                  (Global styles for public site)
├── package.json                (Node dependencies)
├── assets/
│   ├── admin/
│   │   ├── css/
│   │   │   ├── admin.css       (Core layout & theme)
│   │   │   ├── delivery-modal.css
│   │   │   ├── pm-theme.css    (Premium Project Management Theme)
│   │   │   ├── project.css     (Project Specific Customizations)
│   │   │   └── tailwind.css    (Local Tailwind Build)
│   │   └── js/
│   │       ├── app.js          (Main entry, routing, state management)
│   │       ├── auth.js         (Firebase Auth wrappers)
│   │       ├── db.js           (Firestore CRUD abstractions)
```

```
|   |       ├── ui.js            (Modals, Toasts, Reusable UI)
|   |       ├── monitoring.js    (Internal Orders logic)
|   |       ├── kanban.js        (Pending Assignment logic)
|   |       ├── inventory.js     (Inventory & Ledger logic)
|   |       ├── charts.js        (D3.js dashboard charts)
|   |       ├── workflow.js      (Project Management logic)
|   |       ├── bulk_add.js      (Bulk Import UI logic)
|   |       ├── bulk_add_internal.js
|   |       ├── cleanup_internal.js
|   |       └── check_data.js
```

\n\n### File: e:\re\Innovative Engineering Solutions\firebase-config.js\n*Description: Firebase Config*\n\n```javascript\nimport { initializeApp } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-app.js"; import { getAuth } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-auth.js"; import { getFirestore } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore.js"; import { getStorage } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-storage.js";

const firebaseConfig = { apiKey: "AIzaSyDA053MUUIG7vG17XLwJGsDqTKF_N-ND0Y", authDomain: "ies-crm.firebaseapp.com", projectId: "ies-crm", storageBucket: "ies-crm.firebasestorage.app", messagingSenderId: "37296676137", appId: "1:37296676137:web:792fa3a9c01204e3e22b38" };

const app = initializeApp(firebaseConfig); const auth = getAuth(app); const db = getFirestore(app); const storage = getStorage(app);

export { app, auth, db, storage }; \n `\n\n### File: e:\re\Innovative Engineering Solutions\assets\admin\js\db.js\n*Description: DB Operations Layer*\n\n` javascript\nimport { db, storage } from "../../../firebase-config.js"; import { ref, uploadBytes, getDownloadURL, deleteObject } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-storage.js"; import { collection, addDoc, getDocs, getDoc, setDoc, query, onSnapshot, orderBy, serverTimestamp, where, deleteDoc, doc, updateDoc } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore.js";

const MEMBERS_COLLECTION = "employees"; // Using same collection for backward compatibility const PROJECTS_COLLECTION = "projects"; const ORDERS_COLLECTION = "internal_orders";

// Add a new member (alias for backward compatibility) export const addMember = async (memberData) => { try { const docRef = await addDoc(collection(db, MEMBERS_COLLECTION), { ...memberData, createdAt: serverTimestamp() }); return { id: docRef.id, error: null }; } catch (error) { console.error("Error adding member:", error); return { id: null, error: error.message }; } };

// Update an existing member export const updateMember = async (memberId, memberData) => { try { const docRef = doc(db, MEMBERS_COLLECTION, memberId); await updateDoc(docRef, { ...memberData, updatedAt: serverTimestamp() }); return { id: memberId, error: null }; } catch (error) { console.error("Error updating member:", error); return { id: null, error: error.message }; } };

// Subscribe to member list updates (Real-time) export const subscribeToMembers = (callback) => { const q = query(collection(db, MEMBERS_COLLECTION), orderBy("createdAt", "desc")); return onSnapshot(q, (snapshot) => { const members = []; snapshot.forEach((doc) => { members.push({ id: doc.id, ...doc.data() }); }); callback(members); }); };

// Delete a member export const deleteMember = async (memberId) => { try { const docRef = doc(db, MEMBERS_COLLECTION, memberId); await deleteDoc(docRef); return { success: true, error: null }; } catch (error) { console.error("Error deleting member:", error); return { success: false, error: error.message }; } };

// Backward compatibility aliases export const addEmployee = addMember; export const subscribeToEmployees = subscribeToMembers;

// Get stats export const getStats = async () => { const membersSnap = await getDocs(collection(db, MEMBERS_COLLECTION)); return { totalMembers: membersSnap.size, totalEmployees: membersSnap.size, // Backward compatibility activeTasks: 12, completedTasks: 84, departments: 4 }; };

// Member management logic... export const assignTask = async (employeeId, description) => { // This function is no longer used for Internal Orders as "tasks" is a separate concept };

// --- Project Management Functions ---

export const generateProjectId = async () => { const year = new Date().getFullYear(); const q = query(collection(db, PROJECTS_COLLECTION), orderBy("createdAt", "desc")); const snapshot = await getDocs(q); let count = 1; if (!snapshot.empty) { const lastProject = snapshot.docs[0].data().projectId; if (lastProject && lastProject.startsWith(`IES-${year}`)) { count = parseInt(lastProject.split('-')[2]) + 1; } } return `IES-${year}-${count.toString().padStart(5, '0')}` ; };

export const addProject = async (projectData) => { try { // Use custom projectId if provided (e.g. from Internal Order), else auto-generate const projectId = projectData.projectId || await generateProjectId(); const docRef = await addDoc(collection(db, PROJECTS_COLLECTION), { ...projectData, projectId: projectId, revision: 0, status: "Draft", currentStage: "Intake", progress: 0, isLocked: false, createdAt: serverTimestamp(), updatedAt: serverTimestamp() });

```
    // Initial Audit Log
    await addAuditLog(docRef.id, "Project Created", `Project ${projectId} initialized.`);

    return { id: docRef.id, projectId: projectId, error: null };
} catch (error) {
    console.error("Error adding project:", error);
    return { id: null, error: error.message };
}
```

};

export const updateProject = async (projectId, updates, userAction = "Updated") => { try { const ref = doc(db, PROJECTS_COLLECTION, projectId); // Check if revision increase is needed (e.g., if status was 'Approved') // This logic will be more detailed in the UI/App layer, // but we ensure updatedAt is always set.

```
    await updateDoc(ref, {
        ...updates,
        updatedAt: serverTimestamp()
    });

    await addAuditLog(projectId, userAction, JSON.stringify(updates));

    return { success: true };
} catch (error) {
    console.error("Error updating project:", error);
    return { error: error.message };
}
```

};

export const addAuditLog = async (projectId, action, details) => { try { await addDoc(collection(db, PROJECTS_COLLECTION, projectId, "audit_logs"), { action, details, timestamp: serverTimestamp(), user:

"System/Admin" // Should be passed from actual user session later }); } catch (error) { console.error("Audit log error:", error); } };

export const addProjectFile = async (projectId, fileData) => { try { const docRef = await addDoc(collection(db, PROJECTS_COLLECTION, projectId, "files"), { ...fileData, uploadedAt: serverTimestamp(), isApproved: false }); return { id: docRef.id, error: null }; } catch (error) { console.error("Error adding project file:", error); return { id: null, error: error.message }; } };

export const uploadProjectFile = async (projectId, file, uploadedBy) => { try { const fileRef = ref(storage, `projects/${projectId}/files/${Date.now()}_${file.name}`); const metadata = { contentType: file.type, }; const snapshot = await uploadBytes(fileRef, file, metadata); const url = await getDownloadURL(snapshot.ref);

```
    const fileData = {
        name: file.name,
        url: url,
        size: file.size,
        type: file.type,
        category: 'General', // Default category
        uploadedBy: uploadedBy || 'Admin',
        version: 1
    };

    return await addProjectFile(projectId, fileData);
} catch (error) {
    console.error("Error uploading project file:", error);
    return { id: null, error: error.message };
}
```

};

export const deleteProjectFile = async (projectId, fileId, fileUrl) => { try { // 1. Delete from Firebase Storage if URL is provided if (fileUrl && fileUrl !== '#') { try { // Extracts the storage path from the URL const storageRef = ref(storage, fileUrl); await deleteObject(storageRef); } catch (storageError) { console.warn("Storage deletion failed or file not found:", storageError); // Continue with Firestore deletion even if storage fails } }

```
    // 2. Delete metadata from Firestore
    await deleteDoc(doc(db, PROJECTS_COLLECTION, projectId, "files", fileId));
    return { success: true, error: null };
} catch (error) {
    console.error("Error deleting project file:", error);
    return { success: false, error: error.message };
}
```

};

export const submitApproval = async (projectId, stage, approverData) => { try { await addDoc(collection(db, PROJECTS_COLLECTION, projectId, "approvals"), { ...approverData, stage, timestamp: serverTimestamp() });

```
    // If approved, update project stage/status
    if (approverData.status === 'Approved') {
        await updateProject(projectId, {
            status: 'Approved',
            isLocked: true
```

```
        }, `Stage ${stage} Approved`);
    }

    return { success: true };
} catch (error) {
    console.error("Approval error:", error);
    return { error: error.message };
}
```

};

export const subscribeToProjects = (callback) => { const q = query(collection(db, PROJECTS_COLLECTION), orderBy("createdAt", "desc")); return onSnapshot(q, (snapshot) => { const projects = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() })); callback(projects); }); };

export const deleteProject = async (projectId) => { try { const docRef = doc(db, PROJECTS_COLLECTION, projectId); await deleteDoc(docRef); return { success: true, error: null }; } catch (error) { console.error("Error deleting project:", error); return { success: false, error: error.message }; } };

export const softDeleteProject = async (projectId) => { try { const ref = doc(db, PROJECTS_COLLECTION, projectId); await updateDoc(ref, { isDeleted: true, deletedAt: serverTimestamp() }); await addAuditLog(projectId, "Moved to Trash", "Project soft-deleted."); return { success: true }; } catch (error) { console.error("Error soft-deleting project:", error); return { error: error.message }; } };

export const restoreProject = async (projectId) => { try { const ref = doc(db, PROJECTS_COLLECTION, projectId); await updateDoc(ref, { isDeleted: false, restoredAt: serverTimestamp() }); await addAuditLog(projectId, "Restored from Trash", "Project restored."); return { success: true }; } catch (error) { console.error("Error restoring project:", error); return { error: error.message }; } };

export const addOrder = async (orderData) => { try { const docRef = await addDoc(collection(db, ORDERS_COLLECTION), { ...orderData, createdAt: serverTimestamp() }); return { id: docRef.id, error: null }; } catch (error) { console.error("Error adding order:", error); return { id: null, error: error.message }; } };

export const subscribeToOrders = (callback, showTrash = false) => { // Filter by isDeleted status // Note: To use multiple where clauses and orderBy, you might need a composite index. // For simplicity, we can fetch most and filter client-side if the dataset is small, // BUT correctly we should use where. // Let's use where('isDeleted', '==', true/false)

```
// If showTrash is true, we want isDeleted == true
// If showTrash is false, we want isDeleted != true (or false/null)

const status = showTrash ? true : false;

// For simple queries without index, we might just query all and filter in JS if the array is
small.
// However, let's try to be robust.
// We will order by createdAt desc. Using where and orderBy usually requires index.
// Let's just fetch all sorted by date and filter client side for this prototype to avoid
index creation blocks.

const q = query(collection(db, ORDERS_COLLECTION), orderBy("createdAt", "desc"));
return onSnapshot(q, (snapshot) => {
    const orders = [];
    snapshot.forEach((doc) => {
        const data = doc.data();
```

```
        // Client-side filtering to avoid index requirements for now
        const isTrash = !!data.isDeleted;
        if (isTrash === showTrash) {
            orders.push({ id: doc.id, ...data });
        }
    });
    callback(orders);
});
```

};

export const updateOrder = async (orderId, updates) => { try { const ref = doc(db, ORDERS_COLLECTION, orderId); await updateDoc(ref, updates); return { success: true }; } catch (error) { console.error("Error updating order:", error); return { error: error.message }; } };

export const softDeleteOrder = async (orderId) => { return updateOrder(orderId, { isDeleted: true, deletedAt: serverTimestamp() }); };

export const restoreOrder = async (orderId) => { return updateOrder(orderId, { isDeleted: false, restoredAt: serverTimestamp() }); };

export const permanentDeleteOrder = async (orderId) => { try { await deleteDoc(doc(db, ORDERS_COLLECTION, orderId)); return { success: true }; } catch (error) { console.error("Error deleting order:", error); return { error: error.message }; } };

export const updateProjectStatus = async (projectId, newStatus) => { try { const ref = doc(db, PROJECTS_COLLECTION, projectId); const updates = { status: newStatus };

```
    // Auto-file contract for certain statuses
    const milestoneStatuses = ['Approved', 'In Progress', 'Completed'];
    if (milestoneStatuses.includes(newStatus)) {
        updates.contractFiled = true;
    }

    await updateDoc(ref, updates);
    return { success: true };
} catch (error) {
    console.error("Error updating project:", error);
    return { error: error.message };
}
```

};

// === CONTRACT REVIEW === export const saveContractReview = async (projectDocId, reviewData) => { try { const ref = doc(db, PROJECTS_COLLECTION, projectDocId, 'contractReview', 'review'); await setDoc(ref, { ...reviewData, updatedAt: serverTimestamp() }, { merge: true }); return { success: true }; } catch (error) { console.error("Error saving contract review:", error); return { error: error.message }; } };

export const getContractReview = async (projectDocId) => { try { const ref = doc(db, PROJECTS_COLLECTION, projectDocId, 'contractReview', 'review'); const snap = await getDoc(ref); if (snap.exists()) { return { data: snap.data(), error: null }; } return { data: null, error: null }; } catch (error) { console.error("Error getting contract review:", error); return { data: null, error: error.message }; } };

// === DAILY STATS (For Delivery Report) === const STATS_COLLECTION = "daily_stats";

export const getDailyStats = async (startDate, endDate) => { try { const q = query(collection(db, STATS_COLLECTION), where("date", ">=", startDate), where("date", "<=", endDate)); const snapshot = await getDocs(q); const stats = {}; snapshot.forEach(doc => { const data = doc.data(); stats[data.date] = { id: doc.id, ...data }; }); return stats; } catch (error) { console.error("Error fetching daily stats:", error); return {}; } };

export const saveDailyStat = async (date, field, value) => { try { const q = query(collection(db, STATS_COLLECTION), where("date", "==", date)); const snapshot = await getDocs(q);

```
    if (snapshot.empty) {
        await addDoc(collection(db, STATS_COLLECTION), {
            date: date,
            [field]: value,
            createdAt: serverTimestamp()
        });
    } else {
        const docId = snapshot.docs[0].id;
        const ref = doc(db, STATS_COLLECTION, docId);
        await updateDoc(ref, {
            [field]: value,
            updatedAt: serverTimestamp()
        });
    }
    return { success: true };
} catch (error) {
    console.error("Error saving daily stat:", error);
    return { error: error.message };
}
```

};

export const getTrashOrders = async () => { try { const q = query(collection(db, ORDERS_COLLECTION), orderBy("createdAt", "desc")); const snapshot = await getDocs(q); const orders = []; snapshot.forEach((doc) => { const data = doc.data(); if (!!data.isDeleted) { orders.push({ id: doc.id, ...data }); } }); return orders; } catch (e) { console.error("Error fetching trash orders:", e); return []; } };

export const emptyTrash = async (type) => { try { const q = query(collection(db, ORDERS_COLLECTION), where("isDeleted", "==", true)); const snapshot = await getDocs(q);

```
    let deletedCount = 0;
    const promises = [];

    snapshot.forEach((docSnap) => {
        const data = docSnap.data();
        const isDelivery = data.entryType === 'delivery_report';

        if (type === 'delivery' && isDelivery) {
            promises.push(deleteDoc(docSnap.ref));
            deletedCount++;
        } else if (type === 'internal' && !isDelivery) {
            promises.push(deleteDoc(docSnap.ref));
            deletedCount++;
        }
    });
```

```
    await Promise.all(promises);
    return { success: true, deletedCount };
} catch (error) {
    console.error("Error emptying trash:", error);
    return { error: error.message };
}
```

};

// === DAILY REPORTS (Pending Orders Auto-Save) === const REPORTS_COLLECTION = "daily_reports";

export const saveReport = async (reportData) => { try { // Use date string as document ID for easy lookup const dateId = reportData.date; // Format: YYYY-MM-DD const docRef = doc(db, REPORTS_COLLECTION, dateId);

```
    // Check if report exists for this date
    const { getDoc, setDoc } = await
import("https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore.js");
    const existingDoc = await getDoc(docRef);

    if (existingDoc.exists()) {
        // Update existing
        await updateDoc(docRef, {
            ...reportData,
            updatedAt: serverTimestamp()
        });
    } else {
        // Create new
        await setDoc(docRef, {
            ...reportData,
            createdAt: serverTimestamp()
        });
    }

    return { success: true, id: dateId };
} catch (error) {
    console.error("Error saving report:", error);
    return { error: error.message };
}
```

};

export const getReports = async (limit = 30) => { try { const q = query(collection(db, REPORTS_COLLECTION), orderBy("date", "desc")); const snapshot = await getDocs(q); const reports = []; snapshot.forEach(doc => { reports.push({ id: doc.id, ...doc.data() }); }); // Return most recent reports (limit) return reports.slice(0, limit); } catch (error) { console.error("Error fetching reports:", error); return []; } };

export const checkTodayReport = async (dateStr) => { try { const { getDoc } = await import("https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore.js"); const docRef = doc(db, REPORTS_COLLECTION, dateStr); const docSnap = await getDoc(docRef); return docSnap.exists() ? { id: docSnap.id, ...docSnap.data() } : null; } catch (error) { console.error("Error checking today's report:", error); return null; } };

// --- Real-time Project Detail Subscriptions ---

export const subscribeToProjectFiles = (projectId, callback) => { const q = query(collection(db, PROJECTS_COLLECTION, projectId, "files"), orderBy("version", "desc")); return onSnapshot(q, (snapshot) => { const files = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() })); callback(files); }); };

export const subscribeToProjectAuditLogs = (projectId, callback) => { const q = query(collection(db, PROJECTS_COLLECTION, projectId, "audit_logs"), orderBy("timestamp", "desc")); return onSnapshot(q, (snapshot) => { const logs = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() })); callback(logs); }); };

// === DAILY ROSTER MANAGEMENT === const DAILY_ROSTER_COLLECTION = "daily_workflows";

/**

- Save or update a daily workflow for a specific date + department.
- Uses composite ID "YYYY-MM-DD_Department" for fast lookups. */ export const saveWorkflow = async (date, department, assignments, supervisorNotes = '') => { try { const docId = `${date}_${department}`; const docRef = doc(db, DAILY_ROSTER_COLLECTION, docId); const existing = await getDoc(docRef); const payload = { date, department, assignments, supervisorNotes, updatedAt: serverTimestamp() }; if (existing.exists()) { await updateDoc(docRef, payload); } else { await setDoc(docRef, { ...payload, createdAt: serverTimestamp() }); } return { success: true, id: docId }; } catch (error) { console.error("Error saving workflow:", error); return { error: error.message }; }

};

/**

- Get a single workflow document for a date + department. */ export const getWorkflow = async (date, department) => { try { const docId = `${date}_${department}`; const docRef = doc(db, DAILY_ROSTER_COLLECTION, docId); const snap = await getDoc(docRef); if (snap.exists()) { return { data: { id: snap.id, ...snap.data() }, error: null }; } return { data: null, error: null }; } catch (error) { console.error("Error getting workflow:", error); return { data: null, error: error.message }; } };

/**

- Subscribe to all workflow documents for a given date (all departments). */ export const subscribeToWorkflows = (date, callback) => { const q = query(collection(db, DAILY_ROSTER_COLLECTION), where("date", "==", date)); return onSnapshot(q, (snapshot) => { const workflows = snapshot.docs.map(d => ({ id: d.id, ...d.data() })); callback(workflows); }); };

/**

- Get all workflow documents for a given date (all departments) synchronously. */ export const getWorkflowsForDate = async (date) => { try { const q = query(collection(db, DAILY_ROSTER_COLLECTION), where("date", "==", date)); const snapshot = await getDocs(q); return snapshot.docs.map(d => ({ id: d.id, ...d.data() })); } catch (error) { console.error("Error getting workflows for date:", error); return []; } };

/**

- Remove a specific employee's assignment from a workflow document. */ export const deleteWorkflowAssignment = async (workflowId, employeeId) => { try { const docRef = doc(db, DAILY_ROSTER_COLLECTION, workflowId); const snap = await getDoc(docRef); if (!snap.exists()) return { error: "Workflow not found" }; const data = snap.data(); const updatedAssignments = (data.assignments || []).filter(a => a.employeeId !== employeeId); await updateDoc(docRef, { assignments: updatedAssignments, updatedAt: serverTimestamp() }); return { success: true }; } catch (error) { console.error("Error deleting assignment:", error); return { error: error.message }; }

}; \n `\n\n\n### File: e:\re\Innovative Engineering Solutions\assets\admin\js\auth.js\n*Description: Auth Operations Layer*\n\n` javascript\nimport { auth } from "../../../firebase-config.js"; import { signInWithEmailAndPassword, signOut, onAuthStateChanged } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-auth.js";

export const login = async (email, password) => { try { const userCredential = await signInWithEmailAndPassword(auth, email, password); return { user: userCredential.user, error: null }; } catch (error) { console.error("Login error:", error); return { user: null, error: error.message }; } };

export const logout = async () => { try { await signOut(auth); return true; } catch (error) { console.error("Logout error:", error); return false; } };

export const subscribeToAuthChanges = (callback) => { onAuthStateChanged(auth, (user) => { callback(user); }); }; \n `\n\n\n### File: e:\re\Innovative Engineering Solutions\assets\admin\js\monitoring.js\n*Description: Internal Orders Logic*\n\n` javascript\nimport * as DB from './db.js';

// Persistent Date Logic const today = new Date(); const currentMonthStr = today.toISOString().slice(0, 7);

let isTrashMode = false; // Default to EMPTY (Show All) instead of current month to ensure historical data is visible let filterMonthFrom = localStorage.getItem('filterMonthFrom') || ''; let filterMonthTo = localStorage.getItem('filterMonthTo') || ''; let searchTerm = ''; let currentPage = 1; let itemsPerPage = 5000; // Very high default for infinite scroll feel let sortConfig = { key: 'internalOrderNo', direction: 'desc' };

export const setTrashMode = (startTrash) => { isTrashMode = startTrash; currentPage = 1; // Reset to first page

```
// Toggle Empty Trash button visibility
const emptyBtn = document.getElementById('empty-trash-btn');
if (emptyBtn) {
    if (isTrashMode) emptyBtn.classList.remove('hidden');
    else emptyBtn.classList.add('hidden');
}

updateTitle();
```

};

export const setFilters = (monthFrom, monthTo, search) => { if (monthFrom !== undefined) { filterMonthFrom = monthFrom; localStorage.setItem('filterMonthFrom', monthFrom); } if (monthTo !== undefined) { filterMonthTo = monthTo; localStorage.setItem('filterMonthTo', monthTo); } if (search !== undefined) searchTerm = search.toLowerCase(); currentPage = 1; // Reset to first page updateTitle(); };

export const getFilters = () => { return { monthFrom: filterMonthFrom, monthTo: filterMonthTo, search: searchTerm }; };

export const setPageSize = (size) => { itemsPerPage = parseInt(size) || 10; currentPage = 1; // Reset to first page when size changes window.adminApp.renderMonitoring(); };

export const sort = (key) => { if (sortConfig.key === key) { sortConfig.direction = sortConfig.direction === 'asc' ? 'desc' : 'asc'; } else { sortConfig.key = key; sortConfig.direction = 'asc'; } window.adminApp.renderMonitoring(); // Fixed: Use direct call to render current data };

const updateTitle = () => { const title = document.getElementById('page-title'); if (title) title.textContent = isTrashMode ? 'Trash (Deleted Orders)' : 'Internal Orders'; };

const formatDate = (dateStr) => { if (!dateStr || dateStr === '-') return '-'; const parts = dateStr.split('-'); if (parts.length !== 3) return dateStr; return `${parts[2]}/${parts[1]}/${parts[0]}`; // Convert YYYY-MM-DD to DD/MM/YYYY };

export const renderTable = (orders) => { const tbody = document.getElementById('monitoring-table-body'); const paginationControls = document.getElementById('pagination-controls'); const paginationInfo = document.getElementById('pagination-info'); if (!tbody) return;

```
tbody.innerHTML = '';
updateTitle();

// 1. Filter Logic
let processedOrders = orders.filter(order => {
    let matchesMonth = true;
    if (order.date && !isTrashMode) {
        const orderMonth = order.date.slice(0, 7);
        // Default to matching single month if range invalid, or check range
        if (filterMonthFrom && filterMonthTo) {
            matchesMonth = orderMonth >= filterMonthFrom && orderMonth <= filterMonthTo;
        } else if (filterMonthFrom) {
            matchesMonth = orderMonth >= filterMonthFrom;
        } else if (filterMonthTo) {
            matchesMonth = orderMonth <= filterMonthTo;
        }
    }

    let matchesSearch = true;
    if (searchTerm) {
        const searchStr = `${order.internalOrderNo} ${order.customer} ${order.description}
${order.poNo} ${order.drawingNo || ''}`.toLowerCase();
        matchesSearch = searchStr.includes(searchTerm);
    }

    return matchesMonth && matchesSearch;
});

// Filter out Direct Delivery Report entries (keep production orders)
processedOrders = processedOrders.filter(o => o.entryType !== 'delivery_report');


// 2. Sorting Logic
if (sortConfig.key) {
    processedOrders.sort((a, b) => {
        let valA = a[sortConfig.key] || '';
        let valB = b[sortConfig.key] || '';

        // Numeric comparison for total/value
        if (sortConfig.key === 'total' || sortConfig.key === 'value' || sortConfig.key ===
'qty') {
            valA = parseFloat(valA) || 0;
            valB = parseFloat(valB) || 0;
        } else {
```

```
            valA = valA.toString().toLowerCase();
            valB = valB.toString().toLowerCase();
        }

        if (valA < valB) return sortConfig.direction === 'asc' ? -1 : 1;
        if (valA > valB) return sortConfig.direction === 'asc' ? 1 : -1;
        return 0;
    });

    // Update header classes for sort indicators
    document.querySelectorAll('th.sortable').forEach(th => {
        th.classList.remove('sort-asc', 'sort-desc');
        // Support both direct key comparison and multi-line headers
        const onclick = th.getAttribute('onclick') || '';
        if (onclick.includes(`'${sortConfig.key}'`)) {
            th.classList.add(sortConfig.direction === 'asc' ? 'sort-asc' : 'sort-desc');
        }
    });
}

// 3. Pagination Logic
const totalItems = processedOrders.length;
const totalPages = Math.ceil(totalItems / itemsPerPage);
if (currentPage > totalPages && totalPages > 0) currentPage = totalPages;

const startIdx = (currentPage - 1) * itemsPerPage;
const endIdx = startIdx + itemsPerPage;
const paginatedOrders = processedOrders.slice(startIdx, endIdx);

if (totalItems === 0) {
    tbody.innerHTML = '<tr><td colspan="23" style="padding: 3rem; text-align: center; color:
#64748b;">No orders found matching filters.</td></tr>';
    if (paginationInfo) paginationInfo.textContent = 'Showing 0 to 0 of 0 entries';
    if (paginationControls) paginationControls.innerHTML = '';
    return;
}

// Render Rows
paginatedOrders.forEach((order, index) => {
    const tr = document.createElement('tr');
    let status = order.status ? order.status.toUpperCase() : '';

    if (!isTrashMode) {
        if (status === 'DELIVERED') tr.className = 'row-delivered';
        else if (status === 'PARTIALLY DELIVERED' || status === 'PORTION DELIVERED')
tr.className = 'row-partially-delivered';
        else if (status === 'PENDING') tr.className = 'row-pending';
    } else {
        tr.className = 'row-deleted';
    }

    const t = (val) => val || '-';
```

```
    let statusHtml = '';
    if (isTrashMode) {
        let badgeClass = 'badge-default';
        if (status === 'DELIVERED') badgeClass = 'badge-success';
        else if (status === 'PENDING') badgeClass = 'badge-warning';
        statusHtml = `<span class="badge ${badgeClass}">${status || '-'}</span>`;
    } else {
        const statusVal = order.status || 'Pending';
        let badgeClass = 'status-pending';
        if (statusVal === 'Delivered') badgeClass = 'status-delivered';
        else if (statusVal === 'Partially Delivered' || statusVal === 'Portion Delivered')
badgeClass = 'status-partial text-blue-600 bg-blue-50 border border-blue-200'; // Define a
distinctive style
        statusHtml = `<span class="status-badge ${badgeClass}">${statusVal}</span>`;
    }

    let actionsHtml = '';
    if (isTrashMode) {
        actionsHtml = `
            <div class="action-btns">
                <button class="action-btn restore"
onclick="window.adminApp.restoreOrder('${order.id}')" title="Restore">
                    <svg fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-
linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 4v5h.582m15.356 2A8.001 8.001 0
004.582 9m0 0H9m11 11v-5h-.581m0 0a8.003 8.003 0 01-15.357-2m15.357 2H15"></path></svg>
                </button>
                <button class="action-btn delete"
onclick="window.adminApp.permanentDeleteOrder('${order.id}')" title="Delete Permanently">
                    <svg fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-
linecap="round" stroke-linejoin="round" stroke-width="2" d="M6 18L18 6M6 6l12 12"></path>
</svg>
                </button>
            </div>`;
    } else {
        actionsHtml = `
            <div class="action-btns">
                <button class="action-btn edit"
onclick="window.adminApp.editOrder('${order.id}')" title="Edit">
                    <svg fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-
linecap="round" stroke-linejoin="round" stroke-width="2" d="M15.232 5.232l3.536 3.536m-2.036-
5.036a2.5 2.5 0 113.536 3.536L6.5 21.036H3v-3.572L16.732 3.732z"></path></svg>
                </button>
                <button class="action-btn delete"
onclick="window.adminApp.softDeleteOrder('${order.id}')" title="Move to Trash">
                    <svg fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-
linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 7l-.867 12.142A2 2 0 0116.138
21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16">
</path></svg>
                </button>
            </div>`;
    }
```

```
    const avail = (val) => val === 'y' ? '<span class="text-brand-600 font-bold">Y</span>' :
'-';

    tr.innerHTML = `
        <td>${startIdx + index + 1}</td>
        <td class="font-medium" style="white-space: nowrap;">${t(order.internalOrderNo)}</td>
        <td>${formatDate(order.date)}</td>
        <td>${t(order.drawingNo)}</td>
        <td class="truncate" style="max-width: 150px;"
title="${t(order.description)}">${t(order.description)}</td>
        <td class="text-right">${t(order.qty)}</td>
        <td>${t(order.qtyUnit)}</td>
        <td class="text-right">${t(order.saleValueEa || order.value)}</td>
        <td class="text-right">${t(order.prodValueEa)}</td>
        <td class="text-right">${t(order.outsourceValue)}</td>
        <td class="text-center">${avail(order.isLaborJob)}</td>
        <td class="text-right font-bold">${t(order.total)}</td>

        <td>${t(order.customer)}</td>
        <td>${t(order.poNo)}</td>
        <td>${formatDate(order.poDate)}</td>
        <td class="text-center">${avail(order.drgAvail)}</td>
        <td class="text-center">${avail(order.rawAvail)}</td>
        <td class="text-center">${avail(order.finishAvail)}</td>

        <td>${formatDate(order.deliveryDateActual)}</td>
        <td>${t(order.dcNo)}</td>
        <td class="text-right">${t(order.deliveryQty)}</td>
        <td>${t(order.billNo)}</td>
        <td class="text-center">${statusHtml}</td>
        <td>${actionsHtml}</td>
    `;

    // Add Double-Click to Edit
    tr.style.cursor = 'pointer';
    tr.addEventListener('dblclick', () => {
        if (!isTrashMode) {
            window.adminApp.editOrder(order.id);
        }
    });

    tbody.appendChild(tr);
});

// Render Pagination Controls (HIDDEN/REMOVED as per user request for scrollable list)
if (paginationInfo) {
    // Just show total count
    paginationInfo.textContent = `Total Entries: ${totalItems}`;
}

if (paginationControls) {
```

```
    paginationControls.innerHTML = ''; // Clear pagination buttons
}
```

};

export const handleAddOrder = async () => { const form = document.getElementById('add-order-form'); if (!form) return;

```
const formData = new FormData(form);
const data = Object.fromEntries(formData.entries());

// Auto-calculate Total based on Sale Value
if (data.qty) {
    const qty = parseFloat(data.qty) || 0;
    if (data.saleValueEa) data.total = (qty * parseFloat(data.saleValueEa)).toFixed(2);
    if (data.prodValueEa) data.prodValueTotal = (qty *
parseFloat(data.prodValueEa)).toFixed(2);
    if (data.outsourceValue) data.outsourceValueTotal = (qty *
parseFloat(data.outsourceValue)).toFixed(2);
}

// Auto-determine status from DC No and Qty
const dcNo = data.dcNo ? data.dcNo.trim() : '';
if (dcNo) {
    const orderedQty = parseFloat(data.qty) || 0;
    const deliveredQty = parseFloat(data.deliveryQty) || 0;
    if (deliveredQty >= orderedQty && orderedQty > 0) {
        data.status = 'Delivered';
    } else if (deliveredQty > 0) {
        data.status = 'Partially Delivered';
    } else {
        // If they entered a DC No but didn't enter a delivery qty, default to Delivered for
legacy compatibility
        data.status = 'Delivered';
    }
} else {
    data.status = 'Pending';
}

const orderId = data.orderId;
delete data.orderId;

const isNewOrder = !orderId;
const createProject = isNewOrder && document.getElementById('io-create-project')?.checked;

try {
    const res = orderId ? await DB.updateOrder(orderId, data) : await DB.addOrder(data);

    if (res.error) {
        alert("Error: " + res.error);
        return;
    }
```

```
    // Auto-create project if checkbox is ticked (new orders only)
    if (createProject && data.description) {
        try {
            const projectData = {
                projectId: data.internalOrderNo || '',
                name: data.description,
                customerName: data.customer || '',
                jobType: data.department || 'CNC',
                drawingSource: 'Customer Supplied',
                expectedCompletion: data.deliveryDateActual || null,
                internalNotes: `Auto-created from Internal Order ${data.internalOrderNo ||
''}`,
                internalOrderNo: data.internalOrderNo || '',
                poNo: data.poNo || '',
            };

            const projRes = await DB.addProject(projectData);
            if (projRes.error) {
                console.error('Auto-create project failed:', projRes.error);
            } else {
                console.log(`Project auto-created: ${projRes.projectId} from IO
${data.internalOrderNo}`);
            }
        } catch (projErr) {
            console.error('Error auto-creating project:', projErr);
        }
    }

    window.adminApp.closeModal('add-order-modal');
    form.reset();
    const hiddenId = document.getElementById('orderId-input');
    if (hiddenId) hiddenId.value = '';
} catch (err) {
    alert("Error: " + err.message);
}
```

};

export const populateForm = (order) => { const form = document.getElementById('add-order-form'); if (!form) return;

```
const hiddenId = document.getElementById('orderId-input');
if (hiddenId) hiddenId.value = order.id;

Array.from(form.elements).forEach(el => {
    if (!el.name) return;

    if (el.type === 'checkbox') {
        el.checked = order[el.name] === 'y';
    } else if (order[el.name] !== undefined) {
        el.value = order[el.name];
```

```
    }
});

// Update the visible status display based on DC No and Qty
const statusDisplay = document.getElementById('order-status-display');
const statusHidden = form.querySelector('[name="status"]');
const hasDC = order.dcNo && order.dcNo.trim() !== '';
let autoStatus = 'Pending';
let displayHtml = '🟡 Pending';

if (hasDC) {
    const orderedQty = parseFloat(order.qty) || 0;
    const deliveredQty = parseFloat(order.deliveryQty) || 0;
    if (deliveredQty >= orderedQty && orderedQty > 0) {
        autoStatus = 'Delivered';
        displayHtml = '🟢 Delivered';
    } else if (deliveredQty > 0) {
        autoStatus = 'Partially Delivered';
        displayHtml = '🔵 Partially Delivered';
    } else {
        autoStatus = 'Delivered';
        displayHtml = '🟢 Delivered';
    }
}

if (statusDisplay) statusDisplay.value = displayHtml;
if (statusHidden) statusHidden.value = autoStatus;

// Trigger calculation for all cost fields
calculateOrderCosts();

window.adminApp.openAddOrderModal();
```

};

export const setupCostCalculation = () => { const qtyInput = document.getElementById('order-qty'); const saleInput = document.getElementById('order-value'); const prodInput = document.getElementById('order-prod-unit'); const outsourceInput = document.getElementById('order-outsource-unit');

```
if (!qtyInput) return;

const inputs = [qtyInput, saleInput, prodInput, outsourceInput];
inputs.forEach(input => {
    if (input) {
        input.addEventListener('input', calculateOrderCosts);
    }
});
```

};

const calculateOrderCosts = () => { const qty = parseFloat(document.getElementById('order-qty')?.value) || 0; const saleUnit = parseFloat(document.getElementById('order-value')?.value) || 0; const prodUnit =

parseFloat(document.getElementById('order-prod-unit')?.value) || 0; const outsourceUnit = parseFloat(document.getElementById('order-outsource-unit')?.value) || 0;

```
const totalSale = (qty * saleUnit).toFixed(2);
const totalProd = (qty * prodUnit).toFixed(2);
const totalOutsource = (qty * outsourceUnit).toFixed(2);

const totalSaleEl = document.getElementById('order-total');
const totalProdEl = document.getElementById('order-prod-total');
const totalOutsourceEl = document.getElementById('order-outsource-total');

if (totalSaleEl) totalSaleEl.value = totalSale;
if (totalProdEl) totalProdEl.value = totalProd;
if (totalOutsourceEl) totalOutsourceEl.value = totalOutsource;
```

};

/**

- Export orders for the current month to Excel (XLSX format)
- Uses SheetJS library loaded via CDN
- @param {Array} allOrders - All orders from app state
- Export orders for the current month to PDF
- Uses jsPDF and jspdf-autotable loaded via CDN *//*
- Export orders for the current VIEW to PDF
- Uses jsPDF and jspdf-autotable */ export const exportToPDF = () => { // 1. Get currently filtered orders (State Awareness) // We need to re-apply the current filters to get the exact list user is seeing const allOrders = window.adminApp.getCurrentOrders ? window.adminApp.getCurrentOrders() : [];

  // Re-run filter logic to match UI let exportOrders = allOrders.filter(order => { let matchesMonth = true; if (order.date && !isTrashMode) { const orderMonth = order.date.slice(0, 7); if (filterMonthFrom && filterMonthTo) { matchesMonth = orderMonth >= filterMonthFrom && orderMonth <= filterMonthTo; } else if (filterMonthFrom) { matchesMonth = orderMonth >= filterMonthFrom; } else if (filterMonthTo) { matchesMonth = orderMonth <= filterMonthTo; } } let matchesSearch = true; if (searchTerm) { const searchStr = `${order.internalOrderNo} ${order.customer} ${order.description} ${order.poNo}`.toLowerCase(); matchesSearch = searchStr.includes(searchTerm); } return matchesMonth && matchesSearch && order.entryType !== 'delivery_report'; });

  if (!exportOrders || exportOrders.length === 0) { alert('No orders found in current view to export.'); return; }

  try { const { jsPDF } = window.jspdf; const doc = new jsPDF('l', 'mm', 'a4'); // Landscape // Sort by Date (Descending default) exportOrders.sort((a, b) => new Date(b.date) - new Date(a.date)); // Header doc.setFontSize(18); doc.setTextColor(20, 184, 166); // Teal doc.text('INTERNAL ORDERS REPORT', 14, 15); doc.setFontSize(10); doc.setTextColor(100); let periodStr = filterMonthFrom; if (filterMonthFrom !== filterMonthTo) periodStr += `to ${filterMonthTo}`; doc.text(`Period: ${periodStr}`, 14, 22); doc.text(`Generated: ${new Date().toLocaleDateString()}`, 14, 27); // 21-Column Mapping (Matching UI) const headers = [[{ content: '#', rowSpan: 2, styles: { halign: 'center', valign: 'middle' } }, { content: 'Internal Order', colSpan: 2, styles: { halign: 'center' } }, { content: 'Item Details', colSpan: 3, styles: { halign: 'center' } }, { content: 'Pricing & Production', colSpan: 5, styles: { halign: 'center' } }, { content: 'Customer Data',

colSpan: 6, styles: { halign: 'center' } }, { content: 'Delivery Actual', colSpan: 5, styles: { halign: 'center' } }, { content: 'Status', rowSpan: 2, styles: { halign: 'center', valign: 'middle' } }], ['IO No', 'Date', 'Drg No', 'Description', 'Qty', 'Unit', 'Sale', 'InH', 'Out', 'Total', 'Customer', 'PO No', 'PO Date', 'Drg', 'Raw', 'Fin', 'Del Date', 'DC No', 'Del Qty', 'Bill No', 'Stat']]; const rows = exportOrders.map((o, index) => [index + 1, o.internalOrderNo || '-', formatDate(o.date), o.drawingNo || '-', o.description || '-', o.qty || 0, o.qtyUnit || '-', o.saleValueEa || o.value || 0, o.prodValueEa || 0, o.outsourceValue || 0, o.total || 0, o.customer || '-', o.poNo || '-', formatDate(o.poDate), o.drgAvail === 'y' ? 'Y' : '-', o.rawAvail === 'y' ? 'Y' : '-', o.finishAvail === 'y' ? 'Y' : '-', formatDate(o.deliveryDateActual), o.dcNo || '-', o.deliveryQty || 0, o.billNo || '-', o.status || 'Pending']); doc.autoTable({ head: headers, body: rows, startY: 32, theme: 'grid', headStyles: { fillColor: [20, 184, 166], textColor: 255, fontSize: 7, valign: 'middle', halign: 'center' }, bodyStyles: { fontSize: 6, cellPadding: 1, valign: 'middle' }, columnStyles: { 0: { cellWidth: 7 }, // # 1: { cellWidth: 16 }, // IO No 2: { cellWidth: 13 }, // Date 3: { cellWidth: 12 }, // Code 4: { cellWidth: 12 }, // Drg No 5: { cellWidth: 22 }, // Desc 6: { cellWidth: 7 }, // Qty 7: { cellWidth: 8 }, // Unit 8: { cellWidth: 10 }, // Sale 9: { cellWidth: 9 }, // InH 10: { cellWidth: 9 }, // Out 11: { cellWidth: 12 }, // Total 12: { cellWidth: 16 }, // Cust 13: { cellWidth: 12 }, // PO 14: { cellWidth: 13 }, // PO Date 15: { cellWidth: 6 }, // Drg 16: { cellWidth: 6 }, // Raw 17: { cellWidth: 6 }, // Fin 18: { cellWidth: 13 }, // Del Date 19: { cellWidth: 9 }, // DC 20: { cellWidth: 7 }, // Del Qty 21: { cellWidth: 9 }, // Bill 22: { cellWidth: 14 } // Status }, didParseCell: (data) => { // Color coding for status if (data.section === 'body' && data.column.index === 22) { const status = data.cell.raw; if (status === 'Delivered') data.cell.styles.textColor = [22, 163, 74]; else if (status === 'Pending') data.cell.styles.textColor = [202, 138, 4]; } } }); window.open(doc.output('bloburl'), '_blank'); } catch (error) { console.error('PDF Export failed:', error); alert('Failed to generate PDF. See console.'); }

};

// === DELVIERY REPORT LOGIC ===

// Local Delivery Trash State let isDeliveryTrashMode = false;

export const setDeliveryTrashMode = (mode) => { isDeliveryTrashMode = mode; };

export const renderDeliveryReport = async (weekValue, monthValue) => { let startDate, endDate, rangeText;

```
if (weekValue) {
    const [year, week] = weekValue.split('-W');
    const simpleWeek = parseInt(week, 10);
    const jan1 = new Date(year, 0, 1);
    const dayOffset = jan1.getDay() <= 4 ? jan1.getDay() - 1 : jan1.getDay() - 8;
    startDate = new Date(year, 0, 1 + (simpleWeek - 1) * 7 - dayOffset);
    endDate = new Date(startDate);
    endDate.setDate(startDate.getDate() + 6);

    const options = { month: 'short', day: '2-digit' };
    rangeText = `${startDate.toLocaleDateString('en-IN', options)} -
${endDate.toLocaleDateString('en-IN', options)}`;
} else if (monthValue) {
    const [year, month] = monthValue.split('-').map(Number);
    startDate = new Date(year, month - 1, 1);
    endDate = new Date(year, month, 0);

    rangeText = startDate.toLocaleDateString('en-IN', { month: 'long', year: 'numeric' });
} else {
    return;
}
```

```
// Update range display
const rangeEl = document.getElementById('delivery-week-range');
if (rangeEl) {
    rangeEl.textContent = rangeText;
    rangeEl.classList.remove('hidden');
}

const tbody = document.getElementById('delivery-report-body');
if (!tbody) return;
tbody.innerHTML = '<tr><td colspan="16" class="text-center py-8">Loading report data...</td>
</tr>';

// Fetch Daily Stats
const startDateStr = startDate.toISOString().slice(0, 10);
const endDateStr = endDate.toISOString().slice(0, 10);

let dailyStats = {};
try {
    dailyStats = await DB.getDailyStats(startDateStr, endDateStr);
} catch (e) {
    console.error("Failed to load daily stats", e);
}

// Normalize start/end times
startDate.setHours(0, 0, 0, 0);
endDate.setHours(23, 59, 59, 999);

// Filter Delivered Orders
let orders = [];

if (isDeliveryTrashMode) {
    try {
        orders = await DB.getTrashOrders();
    } catch (e) {
        console.error("Error fetching trash", e);
        orders = [];
    }
} else {
    orders = window.adminApp.getCurrentOrders ? window.adminApp.getCurrentOrders() : [];
}

console.log("Render Delivery Report:", { weekValue, monthValue, mode: isDeliveryTrashMode ?
'Trash' : 'Active', totalOrders: orders.length });
console.log("Range:", { start: startDate.toString(), end: endDate.toString() });

const reportOrders = orders.filter(o => {
    // Strict separation: ONLY show explicitly tagged delivery reports
    if (o.entryType !== 'delivery_report') return false;

    if (o.status !== 'Delivered') return false;
```

```
    if (!o.deliveryDateActual || typeof o.deliveryDateActual !== 'string' ||
!o.deliveryDateActual.includes('-')) {
        if (o.entryType === 'delivery_report') console.warn("Invalid Date for Entry:", o);
        return false;
    }

    // If necessary, we can just parse the string manually to be safe:
    const [y, m, d] = o.deliveryDateActual.split('-').map(Number);
    const localOrderDate = new Date(y, m - 1, d);
    localOrderDate.setHours(0, 0, 0, 0);

    const inRange = localOrderDate >= startDate && localOrderDate <= endDate;

    if (o.entryType === 'delivery_report') {
        console.log("Checking Entry:", {
            id: o.id,
            dateStr: o.deliveryDateActual,
            localDate: localOrderDate.toString(),
            inRange
        });
    }

    return inRange;
});

// Sort by Date
reportOrders.sort((a, b) => new Date(a.deliveryDateActual) - new Date(b.deliveryDateActual));

// Group by Date
const grouped = {};
reportOrders.forEach(o => {
    const d = o.deliveryDateActual;
    if (!grouped[d]) grouped[d] = [];
    grouped[d].push(o);
});

// Calculate Summary Stats
let totalItems = reportOrders.length;
let totalValue = reportOrders.reduce((sum, o) => sum + (parseFloat(o.total) || 0), 0);
let totalManpower = reportOrders.reduce((sum, o) => sum + (parseFloat(o.manpower) || 0), 0);

// Update Stats UI
const totalItemsEl = document.getElementById('report-total-items');
if (totalItemsEl) totalItemsEl.textContent = totalItems;

const totalValueEl = document.getElementById('report-total-value');
if (totalValueEl) totalValueEl.textContent = '₹' + totalValue.toLocaleString('en-IN');



const totalManpowerEl = document.getElementById('report-total-manpower');
if (totalManpowerEl) totalManpowerEl.textContent = '₹' + totalManpower.toLocaleString('en-
IN');
```

```
// Render Groups
tbody.innerHTML = '';
let groupSerialNo = 1;
const sortedDates = Object.keys(grouped).sort();

for (const date of sortedDates) {
    const groupOrders = grouped[date];
    const dailyValue = groupOrders.reduce((sum, o) => sum + (parseFloat(o.total) || 0), 0);
    const dailyManpower = groupOrders.reduce((sum, o) => sum + (parseFloat(o.manpower) || 0),
0);

    groupOrders.forEach((order, index) => {
        const tr = document.createElement('tr');
        const isFirst = index === 0;
        const isLast = index === groupOrders.length - 1;

        const displayDailyValue = isLast ? '₹' + dailyValue.toLocaleString('en-IN') : '';
        const displayDailyManpower = isLast ? '₹' + dailyManpower.toLocaleString('en-IN') :
'';
        const dailyClass = isLast ? "font-bold text-slate-800 bg-slate-50/50" : "";

        tr.innerHTML = `
            <td class="px-4 py-2 text-center text-slate-500">${isFirst ? groupSerialNo : ''}
</td>
            <td class="px-4 py-2 font-medium">${isFirst ? formatDate(date) : ''}</td>
            <td class="px-4 py-2 font-medium" style="color: var(--brand-600); white-space:
nowrap;">${order.internalOrderNo || '-'}</td>
            <td class="px-4 py-2">${order.customer || '-'}</td>
            <td class="px-4 py-2">${order.description || '-'}</td>
            <td class="px-4 py-2 text-center" style="white-space: nowrap; text-align: center
!important;">${order.drawingNo || '-'}</td>
            <td class="px-4 py-2 text-center">
                <span class="px-2 py-1 rounded text-xs font-semibold bg-slate-100 text-slate-
600">${order.department || '-'}</span>
            </td>
            <td class="px-4 py-2 text-center">${order.dcNo || '-'}</td>
            <td class="px-4 py-2 text-right font-bold">${order.deliveryQty || order.qty || 0}
</td>
            <td class="px-4 py-2">${order.qtyUnit || '-'}</td>
            <td class="px-4 py-2 text-right">₹${(parseFloat(order.total) ||
0).toLocaleString('en-IN')}</td>
            <td class="px-4 py-2 text-right ${dailyClass}">${displayDailyValue}</td>
            <td class="px-4 py-2 text-right ${dailyClass}">${displayDailyManpower}</td>
            <td class="px-4 py-2 text-center no-print">
                ${!isDeliveryTrashMode ? `
                <button class="text-slate-400 hover:text-blue-500 transition-colors mr-2"
                    onclick="window.adminApp.editOrder('${order.id}')" title="Edit Item">
                    <svg width="16" height="16" fill="none" stroke="currentColor" viewBox="0 0
24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M11 5H6a2 2 0 00-2 2v11a2 2 0 002 2h11a2 2 0 002-2v-5m-1.414-9.414a2 2 0 112.828
```

```
2.828L11.828 15H9v-2.828l8.586-8.586z"></path>
                    </svg>
                </button>
                <button class="text-slate-400 hover:text-red-500 transition-colors"
                    onclick="window.adminApp.softDeleteOrder('${order.id}')" title="Move to
Trash">
                    <svg width="16" height="16" fill="none" stroke="currentColor" viewBox="0 0
24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1
0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16"></path>
                    </svg>
                </button>
                ` : `
                <button class="text-emerald-400 hover:text-emerald-600 transition-colors mr-2"
                    onclick="window.adminApp.restoreOrder('${order.id}')" title="Restore">
                    <svg width="16" height="16" fill="none" stroke="currentColor" viewBox="0 0
24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M4 4v5h.582m15.356 2A8.001 8.001 0 004.582 9m0 0H9m11 11v-5h-.581m0 0a8.003 8.003 0 01-
15.357-2m15.357 2H15"></path>
                    </svg>
                </button>
                <button class="text-red-400 hover:text-red-600 transition-colors"
                    onclick="window.adminApp.permanentDeleteOrder('${order.id}')"
title="Delete Permanently">
                     <svg width="16" height="16" fill="none" stroke="currentColor" viewBox="0
0 24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1
0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16"></path>
                    </svg>
                </button>
                `}
            </td>
        `;
        tbody.appendChild(tr);
    });
    groupSerialNo++;
}

if (reportOrders.length === 0) {
    tbody.innerHTML = `<tr><td colspan="16" class="px-4 py-8 text-center text-slate-400">No
delivered orders found for this week.</td></tr>`;
}
```

}; \n \n\n\n### File: e:\re\Innovative Engineering Solutions\assets\admin\js\workflow.js\n*Description: Project Workflow Logic*\n\n javascript\n// Daily Roster Management Module import * as DB from './db.js';

// Each row in the roster is a flat object: { employeeId, employeeName, employeeNo, department, role, orderNo, drawingNo, description, customer, qty, unit, manpower, assignedWith, inTime, outTime, workStart, workEnd, priority,

notes, status, taskId } let rosterRows = []; let currentWorkflowDate = ''; let currentWorkflowDept = 'All'; let workflowUnsubscribe = null; let currentEditIdx = -1; let loadedDepartments = new Set(); // Track departments that have data for the current date

// ===== INITIALIZATION =====

export const initWorkflowView = () => { const datePicker = document.getElementById('wf-date-picker'); const deptFilter = document.getElementById('wf-dept-filter');

```
// Default to today
const today = new Date().toISOString().split('T')[0];
if (datePicker && !datePicker.value) {
    datePicker.value = today;
}
currentWorkflowDate = datePicker?.value || today;

if (deptFilter) {
    currentWorkflowDept = deptFilter.value || 'All';
}

// Event listeners
if (datePicker) {
    datePicker.onchange = () => {
        currentWorkflowDate = datePicker.value;
        loadWorkflows();
    };
}
if (deptFilter) {
    deptFilter.onchange = () => {
        currentWorkflowDept = deptFilter.value;
        loadWorkflows();
    };
}

loadWorkflows();
```

};

// ===== DATA LOADING =====

const loadWorkflows = async () => { if (!currentWorkflowDate) return;

```
if (workflowUnsubscribe) workflowUnsubscribe();

rosterRows = [];

if (currentWorkflowDept === 'All') {
    workflowUnsubscribe = DB.subscribeToWorkflows(currentWorkflowDate, (workflows) => {
        rosterRows = [];
        loadedDepartments = new Set(workflows.map(wf => wf.department));
        const notesArr = [];
        const members = window.adminApp?.getCurrentMembers ?
window.adminApp.getCurrentMembers() : [];
```

```
        workflows.forEach(wf => {
            (wf.assignments || []).forEach(a => {
                let effectiveRole = (a.role || a.designation || '').trim();
                if (!effectiveRole && members.length > 0) {
                    const m = members.find(m => m.id === a.employeeId);
                    if (m) {
                        effectiveRole = (m.role || m.designation || (m.orgRoles &&
m.orgRoles[0]) || '').trim();
                    }
                }

                (a.tasks || []).forEach(t => {
                    rosterRows.push({
                        employeeId: a.employeeId,
                        employeeName: a.employeeName,
                        employeeNo: a.employeeNo,
                        department: a.department || wf.department,
                        role: effectiveRole,
                        taskId: t.taskId || crypto.randomUUID(),
                        ...t,
                        qty: parseFloat(t.qty) || 0,
                        overheads: parseFloat(t.overheads) || 0,
                        totalOverheads: parseFloat(t.totalOverheads) || 0,
                        prodValueEa: parseFloat(t.prodValueEa) || 0
                    });
                });
            });
            if (wf.supervisorNotes) notesArr.push(`[${wf.department}] ${wf.supervisorNotes}`);
        });

        const notesEl = document.getElementById('wf-supervisor-notes');
        if (notesEl) notesEl.value = notesArr.join('\n');

        renderTable();
        updateUnassignedAlert();
    });
} else {
    const result = await DB.getWorkflow(currentWorkflowDate, currentWorkflowDept);
    if (result.data) {
        rosterRows = [];
        loadedDepartments = new Set([currentWorkflowDept]);
        const members = window.adminApp?.getCurrentMembers ?
window.adminApp.getCurrentMembers() : [];

        (result.data.assignments || []).forEach(a => {
            let effectiveRole = (a.role || a.designation || '').trim();
            if (!effectiveRole && members.length > 0) {
                const m = members.find(m => m.id === a.employeeId);
                if (m) {
                    effectiveRole = (m.role || m.designation || (m.orgRoles && m.orgRoles[0])
|| '').trim();
```

```
                }
            }

            (a.tasks || []).forEach(t => {
                rosterRows.push({
                    employeeId: a.employeeId,
                    employeeName: a.employeeName,
                    employeeNo: a.employeeNo,
                    department: a.department || currentWorkflowDept,
                    role: effectiveRole,
                    taskId: t.taskId || crypto.randomUUID(),
                    ...t,
                    qty: parseFloat(t.qty) || 0,
                    overheads: parseFloat(t.overheads) || 0,
                    totalOverheads: parseFloat(t.totalOverheads) || 0,
                    prodValueEa: parseFloat(t.prodValueEa) || 0
                });
            });
        });
        const notesEl = document.getElementById('wf-supervisor-notes');
        if (notesEl) notesEl.value = result.data.supervisorNotes || '';
    } else {
        rosterRows = [];
        const notesEl = document.getElementById('wf-supervisor-notes');
        if (notesEl) notesEl.value = '';
    }
    renderTable();
    updateUnassignedAlert();
}
```

};

// ===== TABLE RENDERING =====

const renderTable = () => { const tbody = document.getElementById('wf-roster-body'); if (!tbody) return;

```
if (rosterRows.length === 0) {
    tbody.innerHTML = `<tr class="wf-empty-row"><td colspan="14" style="text-align:center;
padding:2.5rem; color:#94a3b8;">No assignments for this date. Click <strong>"Add Assignment"
</strong> to get started.</td></tr>`;
    return;
}

// Group by employee for visual separation
let lastEmpId = '';
let rowNum = 0;

tbody.innerHTML = rosterRows.map((row, idx) => {
    const isNewEmployee = row.employeeId !== lastEmpId;
    lastEmpId = row.employeeId;
    if (isNewEmployee) rowNum++;

    const pClass = row.priority === 'High' ? 'priority-high' : row.priority === 'Medium' ?
```

```
'priority-medium' : 'priority-low';
    const sClass = row.status === 'Completed' ? 'status-done' : row.status === 'Ongoing' ?
'status-ongoing' : 'status-pending';

    return `
    <tr class="${isNewEmployee ? 'emp-row' : ''}">
        <td class="text-center">${isNewEmployee ? rowNum : ''}</td>
        <td class="wf-col-emp">
            ${isNewEmployee ? `<strong>${row.employeeName}</strong><br><small>${row.employeeNo
|| ''} · ${row.department || ''} · ${row.role || ''}</small><br>
            <div class="wf-timing-pills">
                <input type="time" value="${row.inTime || ''}"
onchange="window.adminApp.wfUpdateRow(${idx}, 'inTime', this.value)" class="wf-time-input"
title="In Time">
                <span>-</span>
                <input type="time" value="${row.outTime || ''}"
onchange="window.adminApp.wfUpdateRow(${idx}, 'outTime', this.value)" class="wf-time-input"
title="Out Time">
            </div>` : ''}
        </td>
        <td class="text-center" style="font-weight:600">${row.orderNo || 'Ad-hoc'}</td>
        <td class="text-center">${row.drawingNo || '-'}</td>
        <td class="wf-col-desc ${pClass}">${row.description || '-'}</td>
        <td>${row.customer || '-'}</td>
        <td class="text-center">${row.qty || '-'} ${row.unit || ''}</td>
        <td class="text-right" style="font-weight:600; color:#0f172a;">
            ${(row.prodValueEa > 0 && row.qty > 0) ? (row.prodValueEa * row.qty).toFixed(2) :
'-'}
        </td>
        <td class="text-right" style="font-weight:600; color:#334155;">
            ${row.totalOverheads > 0 ? `₹${row.totalOverheads.toFixed(2)}` : '-'}
        </td>
        <td class="text-center">${row.manpower || '-'}</td>
        <td class="wf-col-assigned" title="${row.assignedWith || ''}">${row.assignedWith || '-
'}</td>
        <td class="text-center">${row.workStart || '-'} to ${row.workEnd || '-'}</td>
        <td class="text-center"><span class="wf-status-badge ${sClass}">${row.status ||
'Pending'}</span></td>
        <td class="text-center">
            <div class="wf-row-actions">
                <button class="wf-row-edit" onclick="window.adminApp.wfEditRow(${idx})"
title="Edit">
                    <svg width="14" height="14" fill="none" stroke="currentColor" viewBox="0 0
24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M11 5H6a2 2 0
00-2 2v11a2 2 0 002 2h11a2 2 0 002-2v-5m-1.414-9.414a2 2 0 112.828 2.828L11.828 15H9v-
2.828l8.586-8.586z"/></svg>
                </button>
                <button class="wf-row-delete" onclick="window.adminApp.wfRemoveRow(${idx})"
title="Remove">
                    <svg width="14" height="14" fill="none" stroke="currentColor" viewBox="0 0
24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 7l-.867
12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1
```

```
0 00-1 1v3M4 7h16"/></svg>
                </button>
            </div>
        </td>
    </tr>`;
}).join('');

// Add totals row (sum unique taskIds to avoid double counting)
const taskOverheads = {};
const taskProdValues = {};

rosterRows.forEach(row => {
    if (row.taskId) {
        taskOverheads[row.taskId] = parseFloat(row.totalOverheads) || 0;
        taskProdValues[row.taskId] = (parseFloat(row.prodValueEa) || 0) * (parseFloat(row.qty)
|| 0);
    }
});

const grandTotalOverheads = Object.values(taskOverheads).reduce((sum, val) => sum + val, 0);
const grandTotalProdValue = Object.values(taskProdValues).reduce((sum, val) => sum + val, 0);

if (grandTotalOverheads > 0 || grandTotalProdValue > 0) {
    tbody.innerHTML += `
        <tr class="wf-row-total" style="background-color: #f1f5f9; font-weight: 700;">
            <td colspan="7" class="text-right" style="padding: 12px 15px; color: #475569;
font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.05em; border-top: 2px solid
#e2e8f0;">Grand Totals</td>
            <td class="text-right" style="padding: 12px 15px; color: #0891b2; border-top: 2px
solid #e2e8f0; font-size: 0.9rem;">₹${grandTotalProdValue.toFixed(2)}</td>
            <td class="text-right" style="padding: 12px 15px; color: #0f172a; border-top: 2px
solid #e2e8f0; border-left: 2px solid #e2e8f0; font-size: 0.9rem;">₹
${grandTotalOverheads.toFixed(2)}</td>
            <td colspan="5" style="border-top: 2px solid #e2e8f0;"></td>
        </tr>`;
}
```

};

// ===== ROW UPDATE =====

export const updateRow = (idx, field, value) => { if (rosterRows[idx]) { const empId = rosterRows[idx].employeeId; rosterRows[idx][field] = value;

```
    // Sync inTime/outTime for all rows of the same employee
    if (field === 'inTime' || field === 'outTime') {
        rosterRows.forEach(r => {
            if (r.employeeId === empId) r[field] = value;
        });
        renderTable(); // Re-render to show sync'd times in all rows
    }
```

```
    saveAll(); // Auto-save on inline changes
}
```

};

export const removeRow = (idx) => { if (confirm('Remove this assignment?')) { rosterRows.splice(idx, 1); renderTable(); saveAll(); // Auto-save on removal } };

export const editRow = (idx) => { currentEditIdx = idx; openAssignModal(idx); };

// ===== ASSIGNMENT MODAL =====

export const openAssignModal = (editIdx = -1) => { const members = window.adminApp?.getCurrentMembers ? window.adminApp.getCurrentMembers() : []; const dept = currentWorkflowDept;

```
currentEditIdx = editIdx;
const editData = editIdx >= 0 ? rosterRows[editIdx] : null;

const filteredMembers = (dept !== 'All'
    ? members.filter(m => {
        const memberDept = (m.section || m.department || '').toLowerCase();
        return memberDept.includes(dept.toLowerCase());
    })
    : members).sort((a, b) => {
        const idA = a.employeeId || 'ZZZ';
        const idB = b.employeeId || 'ZZZ';
        return idA.localeCompare(idB);
    });

// Remove existing modal
const existing = document.getElementById('wf-assign-modal');
if (existing) existing.remove();

let modalHtml = `
<div id="wf-assign-modal" class="modal active">
    <div class="modal-backdrop" onclick="document.getElementById('wf-assign-
modal').classList.remove('active'); setTimeout(() => document.getElementById('wf-assign-
modal')?.remove(), 300)"></div>
    <div class="modal-content modal-wide" style="max-width: 900px;">
        <div class="modal-header" style="background: linear-gradient(135deg, #0d9488 0%,
#065f46 100%);">
            <h3 class="modal-title" style="color:white;">${editData ? 'Edit Assignment' :
'Assign Work'}</h3>
            <button class="modal-close" onclick="document.getElementById('wf-assign-
modal').classList.remove('active'); setTimeout(() => document.getElementById('wf-assign-
modal')?.remove(), 300)">
                <svg fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-
linecap="round" stroke-linejoin="round" stroke-width="2" d="M6 18L18 6M6 6l12 12"/></svg>
            </button>
        </div>
        <div class="modal-body wf-compact-modal-body">
            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;">
                <!-- LEFT COLUMN: PEOPLE & TIMING -->
```

```
                <div style="display: flex; flex-direction: column; gap: 0.5rem;">
                    <!-- GROUP: TEAM -->
                    <div class="wf-modal-group">
                        <div class="wf-group-title">
                            <svg width="12" height="12" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M12 4.354a4 4 0 110 5.292M15 21H3v-1a6 6 0 0112 0v1zm0 0h6v-1a6 6 0 00-9-5.197M13 7a4 4 0
11-8 0 4 4 0 018 0z"/></svg>
                            Who is Assigned?
                        </div>
                        <div class="form-group" style="margin-bottom: 0.75rem;">
                            <label class="wf-form-label">Primary Employee</label>
                            <select id="wf-assign-employee" class="form-input" style="padding:
0.4rem 0.6rem; font-size: 0.8rem;">
                                <option value="">-- Select Employee --</option>
                                ${filteredMembers.map(m => `<option value="${m.id}" data-
name="${m.name}" data-empno="${m.employeeId || ''}" data-dept="${m.section || m.department ||
''}" data-role="${m.role || m.designation || ''}" data-overheads="${m.overheads || 0}"
${editData && editData.employeeId === m.id ? 'selected' : ''}>${m.name} (${m.employeeId ||
'N/A'})</option>`).join('')}
                            </select>
                        </div>

                        <div class="inline-section-label" style="margin: 0.5rem 0 0.4rem;
font-size: 0.6rem;">Assigned With (Team)</div>
                        <div style="max-height: 110px; overflow-y: auto; background: #f8fafc;
border: 1px solid #e2e8f0; border-radius: 6px; padding: 0.4rem;">
                            <div class="form-group" style="margin-bottom: 0.4rem;">
                                <input type="text" id="wf-assign-with-filter" class="form-
input" placeholder="Search team members..." style="font-size: 0.7rem; padding: 3px 6px;"
oninput="window.adminApp.wfFilterTeam(this.value)">
                            </div>
                            <div id="wf-assign-with-list" class="wf-team-list" style="grid-
template-columns: repeat(auto-fill, minmax(110px, 1fr)); gap: 0.4rem;">
                                ${filteredMembers.map(m => `
                                    <label class="wf-team-item" style="padding: 4px 8px; font-
size: 0.7rem; gap: 6px;">
                                        <input type="checkbox" name="wf-team-member"
value="${m.id}" data-name="${m.name}" ${editData?.assignedWith?.includes(m.name) ? 'checked' :
''}>
                                        <span>${m.name}</span>
                                    </label>
                                `).join('')}
                            </div>
                        </div>
                    </div>

                    <!-- GROUP: SHIFTS -->
                    <div class="wf-modal-group">
                        <div class="wf-group-title">
                            <svg width="12" height="12" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
```

```
d="M12 8v4l3 3m6-3a9 9 0 11-18 0 9 9 0 0118 0z"/></svg>
                            Employee Shift Details
                        </div>
                        <div class="wf-grid-2">
                            <div class="form-group">
                                <label class="wf-form-label">Shift In</label>
                                <input type="time" id="wf-assign-intime" class="form-input"
style="padding: 0.4rem; font-size: 0.8rem;" value="${editData?.inTime || ''}">
                            </div>
                            <div class="form-group">
                                <label class="wf-form-label">Shift Out</label>
                                <input type="time" id="wf-assign-outtime" class="form-input"
style="padding: 0.4rem; font-size: 0.8rem;" value="${editData?.outTime || ''}">
                            </div>
                        </div>
                    </div>

                    <!-- GROUP: PROGRESS & STATS -->
                    <div class="wf-modal-group">
                        <div class="wf-group-title">
                            <svg width="12" height="12" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M9 7h6m0 10v-3m-3 3h.01M9 17h.01M9 14h.01M12 14h.01M15 11h.01M12 11h.01M9 11h.01M7 21h10a2
2 0 002-2V5a2 2 0 00-2-2H7a2 2 0 00-2 2v14a2 2 0 002 2z"/></svg>
                            Calculated Overheads
                        </div>
                        <div id="wf-overhead-container" style="background: #f8fafc; border:
1px solid #e2e8f0; border-radius: 6px; padding: 0.6rem;">
                            <div id="wf-overhead-list" style="display: flex; flex-direction:
column; gap: 0.3rem; margin-bottom: 0.5rem; max-height: 80px; overflow-y: auto;">
                                <!-- Populated by JS -->
                            </div>
                            <div style="display: flex; justify-content: space-between; border-
top: 1px dashed #cbd5e1; padding-top: 0.4rem; font-weight: 700; font-size: 0.85rem; color:
#0f172a;">
                                <span>Total Overheads:</span>
                                <span id="wf-overhead-total">₹0.00</span>
                            </div>
                        </div>
                    </div>
                </div>

                <!-- RIGHT COLUMN: TASK DETAILS -->
                <div style="display: flex; flex-direction: column; gap: 0.5rem;">
                    <!-- GROUP: WORK -->
                    <div class="wf-modal-group">
                        <div class="wf-group-title">
                            <svg width="12" height="12" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M9 5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2V7a2 2 0 00-2-2h-2M9 5a2 2 0 002 2h2a2 2 0
002-2M9 5a2 2 0 012-2h2a2 2 0 012 2m-3 7h3m-3 4h3m-6-4h.01M9 16h.01"/></svg>
                            What is the Job?
```

```
                        </div>
                        <div class="form-group" style="margin-bottom: 0.75rem;">
                            <label class="wf-form-label">IO # (Auto-fills details)</label>
                            <input type="text" id="wf-assign-io" class="form-input"
style="padding: 0.4rem; font-size: 0.8rem;" placeholder="e.g. 202526-530" list="wf-io-
suggestions" value="${editData?.orderNo || ''}">
                            <datalist id="wf-io-suggestions"></datalist>
                        </div>
                        <div class="form-group" style="margin-bottom: 0.75rem;">
                            <label class="wf-form-label">Task Description *</label>
                            <textarea id="wf-assign-desc" class="form-input" placeholder="Task
details..." style="min-height: 45px; max-height: 80px; padding: 0.4rem; font-size: 0.8rem;
resize: none;">${editData?.description || ''}</textarea>
                        </div>
                        <div class="wf-grid-2">
                            <div class="form-group">
                                <label class="wf-form-label">Drawing No</label>
                                <input type="text" id="wf-assign-drg" class="form-input"
style="padding: 0.4rem; font-size: 0.8rem;" placeholder="DRG #" value="${editData?.drawingNo
|| ''}">
                            </div>
                            <div class="form-group">
                                <label class="wf-form-label">Customer</label>
                                <input type="text" id="wf-assign-customer" class="form-input"
style="padding: 0.4rem; font-size: 0.8rem;" placeholder="Client" value="${editData?.customer
|| ''}">
                            </div>
                        </div>
                    </div>

                    <!-- GROUP: SPECS -->
                    <div class="wf-modal-group">
                        <div class="wf-group-title">
                            <svg width="12" height="12" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 11H5m14 0a2 2 0 012 2v6a2 2 0 01-2 2H5a2 2 0 01-2-2v-6a2 2 0 012-2m14 0V9a2 2 0 00-2-
2M5 11V9a2 2 0 012-2m0 0V5a2 2 0 012-2h6a2 2 0 012 2v2M7 7h10"/></svg>
                            Timeline & Resources
                        </div>
                        <div class="wf-grid-3" style="display: grid; grid-template-columns:
repeat(3, 1fr); gap: 0.5rem; margin-bottom: 0.5rem;">
                            <div class="form-group">
                                <label class="wf-form-label">Start</label>
                                <input type="time" id="wf-assign-workstart" class="form-input"
style="padding: 0.35rem; font-size: 0.75rem;" value="${editData?.workStart || ''}">
                            </div>
                            <div class="form-group">
                                <label class="wf-form-label">End</label>
                                <input type="time" id="wf-assign-workend" class="form-input"
style="padding: 0.35rem; font-size: 0.75rem;" value="${editData?.workEnd || ''}">
                            </div>
                            <div class="form-group">
```

```
                                <label class="wf-form-label">MP</label>
                                <input type="number" id="wf-assign-manpower" class="form-
input" style="padding: 0.35rem; font-size: 0.75rem;" min="1" value="${editData?.manpower ||
'1'}">
                            </div>
                        </div>
                        <div class="wf-grid-3" style="display: grid; grid-template-columns:
repeat(3, 1fr); gap: 0.5rem;">
                            <div class="form-group">
                                <label class="wf-form-label">Total Qty</label>
                                <input type="number" id="wf-assign-qty" class="form-input"
style="padding: 0.35rem; font-size: 0.75rem;" min="0" value="${editData?.qty || ''}"
oninput="window.adminApp.wfCalculateProdValue()">
                            </div>
                            <div class="form-group">
                                <label class="wf-form-label">Prod. Cost/Unit</label>
                                <input type="number" id="wf-assign-prod-cost" class="form-
input" style="padding: 0.35rem; font-size: 0.75rem;" min="0" step="0.01"
value="${editData?.prodValueEa || ''}" oninput="window.adminApp.wfCalculateProdValue()">
                            </div>
                            <div class="form-group">
                                <label class="wf-form-label">Total Prod. Val</label>
                                <input type="text" id="wf-assign-prod-total" class="form-input
computed" style="padding: 0.35rem; font-size: 0.75rem; background: #f1f5f9;" readonly
value="">
                            </div>
                        </div>
                    </div>
                </div>

                <!-- FULL WIDTH: PROGRESS & NOTES -->
                <div class="wf-full">
                    <div class="wf-modal-group" style="margin-bottom: 0;">
                        <div class="wf-group-title">
                            <svg width="12" height="12" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M11 5H6a2 2 0 00-2 2v11a2 2 0 002 2h11a2 2 0 002-2v-5m-1.414-9.414a2 2 0 112.828
2.828L11.828 15H9v-2.828l8.586-8.586z"/></svg>
                            Priority, Status & Notes
                        </div>
                        <div style="display: flex; gap: 1rem; align-items: flex-end;">
                            <div style="display: grid; grid-template-columns: 1fr 1fr; gap:
0.75rem; width: 35%;">
                                <div class="form-group">
                                    <label class="wf-form-label">Priority</label>
                                    <select id="wf-assign-priority" class="form-input"
style="padding: 0.4rem; font-size: 0.8rem;">
                                        <option value="Medium" ${editData?.priority ===
'Medium' ? 'selected' : ''}>Medium</option>
                                        <option value="High" ${editData?.priority === 'High' ?
'selected' : ''}>High</option>
                                        <option value="Low" ${editData?.priority === 'Low' ?
```

```
'selected' : ''}>Low</option>
                                    </select>
                                </div>
                                <div class="form-group">
                                    <label class="wf-form-label">Status</label>
                                    <select id="wf-assign-status" class="form-input"
style="padding: 0.4rem; font-size: 0.8rem;">
                                        <option value="Pending" ${editData?.status ===
'Pending' ? 'selected' : ''}>Pending</option>
                                        <option value="Ongoing" ${editData?.status ===
'Ongoing' ? 'selected' : ''}>Ongoing</option>
                                        <option value="Completed" ${editData?.status ===
'Completed' ? 'selected' : ''}>Completed</option>
                                    </select>
                                </div>
                            </div>
                            <div style="flex: 1;">
                                <div class="form-group">
                                    <label class="wf-form-label">Additional
Instructions</label>
                                    <input type="text" id="wf-assign-notes" class="form-input"
style="padding: 0.4rem; font-size: 0.8rem;" placeholder="e.g. Finish by tomorrow"
value="${editData?.notes || ''}">
                                </div>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        </div>
        <div class="modal-footer" style="border-top: 1px solid #e2e8f0; padding-top: 1rem;">
            <button class="btn btn-secondary" onclick="document.getElementById('wf-assign-
modal').classList.remove('active'); setTimeout(() => document.getElementById('wf-assign-
modal')?.remove(), 300)">Cancel</button>
            <button class="btn btn-primary"
onclick="window.adminApp.wfConfirmAssign()">${editData ? 'Update Assignment' : 'Assign Task'}
</button>
        </div>
    </div>
</div>`;

document.body.insertAdjacentHTML('beforeend', modalHtml);
populateIOSuggestions();
setupIOLookup();
setupEmployeeTimingLookup();
setupOverheadListeners();
calculateProdValue(); // Initial calculation
updateOverheadsDisplay(); // Initial calculation
```

};

export const calculateProdValue = () => { const qty = parseFloat(document.getElementById('wf-assign-qty')?.value) || 0; const cost = parseFloat(document.getElementById('wf-assign-prod-cost')?.value) || 0; const totalEl = document.getElementById('wf-assign-prod-total'); if (totalEl) totalEl.value = (qty * cost).toFixed(2); };

const setupOverheadListeners = () => { const empSelect = document.getElementById('wf-assign-employee'); if (empSelect) { empSelect.addEventListener('change', updateOverheadsDisplay); }

```
const teamList = document.getElementById('wf-assign-with-list');
if (teamList) {
    teamList.addEventListener('change', (e) => {
        if (e.target.name === 'wf-team-member') {
            updateOverheadsDisplay();
        }
    });
}
```

};

export const updateOverheadsDisplay = () => { const members = window.adminApp?.getCurrentMembers ? window.adminApp.getCurrentMembers() : []; const empSelect = document.getElementById('wf-assign-employee'); const primaryId = empSelect?.value;

```
const teamCheckboxes = document.querySelectorAll('input[name="wf-team-member"]:checked');
const teamIds = Array.from(teamCheckboxes).map(cb => cb.value);

const listEl = document.getElementById('wf-overhead-list');
const totalEl = document.getElementById('wf-overhead-total');
if (!listEl || !totalEl) return;

let total = 0;
let html = '';

const allIds = [primaryId, ...teamIds].filter(Boolean);
const uniqueIds = [...new Set(allIds)];

uniqueIds.forEach(id => {
    const m = members.find(m => m.id === id);
    if (m) {
        const oh = parseFloat(m.overheads) || 0;
        total += oh;
        html += `
            <div style="display: flex; justify-content: space-between; font-size: 0.75rem;
color: #64748b;">
                <span>${m.name}</span>
                <span>₹${oh.toFixed(2)}</span>
            </div>`;
    }
});

if (!html) {
    html = '<div style="font-size: 0.75rem; color: #94a3b8; text-align: center;">No employees
selected</div>';
```

```
}

listEl.innerHTML = html;
totalEl.textContent = `₹${total.toFixed(2)}`;
```

};

const populateIOSuggestions = () => { const orders = window.adminApp?.getCurrentOrders ? window.adminApp.getCurrentOrders() : []; const datalist = document.getElementById('wf-io-suggestions'); if (!datalist) return; const pending = orders.filter(o => o.status === 'Pending' || o.status === 'Partially Delivered'); datalist.innerHTML = pending.map(o => `<option value="${o.internalOrderNo}">${o.customer || 'No Customer'} - ${o.description || 'No Description'}</option>`).join(''); };

const setupIOLookup = () => { const ioInput = document.getElementById('wf-assign-io'); if (!ioInput) return;

```
ioInput.addEventListener('input', () => {
    const val = ioInput.value.trim();
    if (!val) return;
    const orders = window.adminApp?.getCurrentOrders ? window.adminApp.getCurrentOrders() :
[];
    const match = orders.find(o => o.internalOrderNo === val);
    if (match) {
        const setVal = (id, v) => { const el = document.getElementById(id); if (el) el.value =
v || ''; };
        setVal('wf-assign-desc', match.description);
        setVal('wf-assign-drg', match.drawingNo);
        setVal('wf-assign-customer', match.customer);
        setVal('wf-assign-qty', match.qty);
        setVal('wf-assign-unit', match.qtyUnit || 'Nos');
        setVal('wf-assign-prod-cost', match.prodValueEa || '');
        calculateProdValue();
    }
});
```

};

const setupEmployeeTimingLookup = () => { const empSelect = document.getElementById('wf-assign-employee'); const inTimeInput = document.getElementById('wf-assign-intime'); const outTimeInput = document.getElementById('wf-assign-outtime');

```
if (!empSelect || !inTimeInput || !outTimeInput) return;

empSelect.addEventListener('change', () => {
    const empId = empSelect.value;
    if (!empId) return;

    // Find if this employee already has an assignment today
    const existing = rosterRows.find(r => r.employeeId === empId);
    if (existing) {
        if (existing.inTime) inTimeInput.value = existing.inTime;
        if (existing.outTime) outTimeInput.value = existing.outTime;
```

```
    }
});
```

};

// ===== CONFIRM ASSIGNMENT =====

export const confirmAssign = () => { const empSelect = document.getElementById('wf-assign-employee'); if (!empSelect || !empSelect.value) { alert('Please select an employee.'); return; }

```
const getVal = id => (document.getElementById(id)?.value || '').trim();

const desc = getVal('wf-assign-desc');
const orderNo = getVal('wf-assign-io');
if (!desc && !orderNo) {
    alert('Please enter a task description or IO number.');
    return;
}

const selectedOption = empSelect.options[empSelect.selectedIndex];
const mainEmpId = empSelect.value;
const mainEmpName = selectedOption.getAttribute('data-name');
const mainEmpNo = selectedOption.getAttribute('data-empno');
const mainEmpDept = selectedOption.getAttribute('data-dept') || currentWorkflowDept;
const mainEmpRole = selectedOption.getAttribute('data-role') || '';

// Get team members
const teamCheckboxes = document.querySelectorAll('input[name="wf-team-member"]:checked');
const teamMembers = Array.from(teamCheckboxes).map(cb => ({
    id: cb.value,
    name: cb.getAttribute('data-name')
})).filter(tm => tm.id !== mainEmpId); // Exclude main employee if selected twice

const taskId = currentEditIdx >= 0 ? rosterRows[currentEditIdx].taskId : crypto.randomUUID();
const modalInTime = getVal('wf-assign-intime');
const modalOutTime = getVal('wf-assign-outtime');

const allAssignees = [
    { id: mainEmpId, name: mainEmpName, empNo: mainEmpNo, dept: mainEmpDept, role:
mainEmpRole, overheads: parseFloat(selectedOption.getAttribute('data-overheads')) || 0 },
    ...teamMembers.map(tm => {
        const m = (window.adminApp.getCurrentMembers()).find(m => m.id === tm.id);
        return {
            id: tm.id,
            name: tm.name,
            empNo: m?.employeeId || '',
            dept: m?.section || m?.department || currentWorkflowDept,
            role: m?.role || m?.designation || '',
            overheads: parseFloat(m?.overheads) || 0
        };
    })
];
```

```
const baseData = {
    type: orderNo ? 'order' : 'adhoc',
    orderNo,
    description: desc,
    drawingNo: getVal('wf-assign-drg'),
    customer: getVal('wf-assign-customer'),
    qty: parseFloat(getVal('wf-assign-qty')) || 0,
    unit: getVal('wf-assign-unit') || 'Nos',
    manpower: parseInt(getVal('wf-assign-manpower')) || 1,
    workStart: getVal('wf-assign-workstart'),
    workEnd: getVal('wf-assign-workend'),
    priority: getVal('wf-assign-priority') || 'Medium',
    notes: getVal('wf-assign-notes'),
    status: getVal('wf-assign-status') || 'Pending',
    prodValueEa: parseFloat(getVal('wf-assign-prod-cost')) || 0,
    taskId: taskId,
    totalOverheads: allAssignees.reduce((sum, a) => sum + (a.overheads || 0), 0)
};

if (currentEditIdx >= 0) {
    // Find ALL rows with this taskId and remove them first (to handle changes in team
members)
    rosterRows = rosterRows.filter(r => r.taskId !== taskId);
}

// Add/Update for all assignees
allAssignees.forEach(assignee => {
    // Other members in this task
    const others = allAssignees.filter(a => a.id !== assignee.id).map(a => a.name).join(', ');

    // Find existing time for THIS assignee in the roster
    const existingRow = rosterRows.find(r => r.employeeId === assignee.id);
    const existingIn = existingRow ? existingRow.inTime : '';
    const existingOut = existingRow ? existingRow.outTime : '';

    // Only the main employee takes the modal's timing.
    // Team members keep their existing timing or stay empty.
    const finalIn = (assignee.id === mainEmpId) ? modalInTime : (existingIn || '');
    const finalOut = (assignee.id === mainEmpId) ? modalOutTime : (existingOut || '');

    const finalData = {
        ...baseData,
        employeeId: assignee.id,
        employeeName: assignee.name,
        employeeNo: assignee.empNo,
        department: assignee.dept,
        role: assignee.role,
        assignedWith: others,
        overheads: assignee.overheads,
        inTime: finalIn,
        outTime: finalOut
    };
```

```
    const lastIdx = rosterRows.map(r => r.employeeId).lastIndexOf(finalData.employeeId);
    if (lastIdx >= 0) {
        rosterRows.splice(lastIdx + 1, 0, finalData);
    } else {
        rosterRows.push(finalData);
    }

    // Sync times for THIS specific employee across all their rows
    if (finalIn || finalOut) {
        rosterRows.forEach(r => {
            if (r.employeeId === assignee.id) {
                if (finalIn) r.inTime = finalIn;
                if (finalOut) r.outTime = finalOut;
            }
        });
    }
});

// Close modal
currentEditIdx = -1;
window.adminApp.wfSaveAll(); // Refresh and save
const modal = document.getElementById('wf-assign-modal');
if (modal) {
    modal.classList.remove('active');
    setTimeout(() => modal.remove(), 300);
}

renderTable();
saveAll(); // Auto-save after addition/edit
```

};

// ===== SAVE =====

export const saveAll = async () => { if (!currentWorkflowDate) return;

```
const notes = (document.getElementById('wf-supervisor-notes')?.value || '').trim();

const grouped = {};
rosterRows.forEach(row => {
    const key = row.employeeId;
    if (!grouped[key]) {
        grouped[key] = {
            employeeId: row.employeeId,
            employeeName: row.employeeName,
            employeeNo: row.employeeNo,
            department: row.department,
            role: row.role,
            tasks: []
        };
    }
    grouped[key].tasks.push({
```

```
        type: row.type,
        orderNo: row.orderNo,
        drawingNo: row.drawingNo,
        description: row.description,
        customer: row.customer,
        qty: row.qty,
        unit: row.unit,
        manpower: row.manpower,
        assignedWith: row.assignedWith,
        inTime: row.inTime,
        outTime: row.outTime,
        workStart: row.workStart,
        workEnd: row.workEnd,
        priority: row.priority,
        notes: row.notes,
        status: row.status,
        overheads: parseFloat(row.overheads) || 0,
        totalOverheads: parseFloat(row.totalOverheads) || 0,
        prodValueEa: parseFloat(row.prodValueEa) || 0,
        taskId: row.taskId
    });
});

const assignments = Object.values(grouped);

if (currentWorkflowDept === 'All') {
    const byDept = {};
    assignments.forEach(a => {
        const dept = a.department || 'Unassigned';
        if (!byDept[dept]) byDept[dept] = [];
        byDept[dept].push(a);
    });

    // Ensure we update (clear) any department that was previously loaded but now has no
assignments
    const allDeptsToUpdate = new Set([...Object.keys(byDept), ...loadedDepartments]);
    for (const dept of allDeptsToUpdate) {
        const deptAssignments = byDept[dept] || [];
        await DB.saveWorkflow(currentWorkflowDate, dept, deptAssignments, notes);
    }
} else {
    await DB.saveWorkflow(currentWorkflowDate, currentWorkflowDept, assignments, notes);
}

// Visual feedback
const saveBtn = document.querySelector('.wf-btn-save');
if (saveBtn) {
    const original = saveBtn.innerHTML;
    saveBtn.innerHTML = `<svg width="16" height="16" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M5 13l4 4L19 7"/></svg> Saved!`;
    saveBtn.style.background = '#059669';
```

```
    setTimeout(() => { if (saveBtn) { saveBtn.innerHTML = original; saveBtn.style.background =
'''; } }, 1500);
}
```

};

// ===== COPY PREVIOUS DAY =====

export const copyPreviousDay = async () => { if (!currentWorkflowDate) return;

```
const prevDate = new Date(currentWorkflowDate);
prevDate.setDate(prevDate.getDate() - 1);
const prevDateStr = prevDate.toISOString().split('T')[0];

if (currentWorkflowDept === 'All') {
    alert('Please select a specific department to copy from previous day.');
    return;
}

const result = await DB.getWorkflow(prevDateStr, currentWorkflowDept);
if (result.data && result.data.assignments && result.data.assignments.length > 0) {
    const totalTasks = result.data.assignments.reduce((sum, a) => sum + (a.tasks?.length ||
0), 0);
    if (!confirm(`Copy ${totalTasks} task(s) from ${prevDateStr}?`)) return;

    rosterRows = [];
    result.data.assignments.forEach(a => {
        (a.tasks || []).forEach(t => {
            rosterRows.push({
                employeeId: a.employeeId,
                employeeName: a.employeeName,
                employeeNo: a.employeeNo,
                department: a.department,
                role: a.role || '',
                ...t,
                status: 'Pending',
                inTime: '',
                outTime: '',
                workStart: '',
                workEnd: ''
            });
        });
    });
    renderTable();
    saveAll(); // Auto-save after copy
} else {
    alert(`No assignments found for ${currentWorkflowDept} on ${prevDateStr}.`);
}
```

};

// ===== UNASSIGNED ALERT =====

const updateUnassignedAlert = () => { const alertEl = document.getElementById('wf-unassigned-alert'); const countEl = document.getElementById('wf-unassigned-count'); if (!alertEl || !countEl) return;

```
const orders = window.adminApp?.getCurrentOrders ? window.adminApp.getCurrentOrders() : [];
const pendingCount = orders.filter(o => o.status === 'Pending').length;

if (pendingCount > 0) {
    countEl.textContent = `${pendingCount} pending order${pendingCount !== 1 ? 's' : ''}`;
    alertEl.classList.remove('hidden');
} else {
    alertEl.classList.add('hidden');
}
```

};

// ===== PRINT =====

export const printWorksheet = () => { if (rosterRows.length === 0) { alert('No assignments to print.'); return; }

```
const date = currentWorkflowDate;
const dept = currentWorkflowDept === 'All' ? 'All Departments' : currentWorkflowDept;
const notes = (document.getElementById('wf-supervisor-notes')?.value || '').trim();

// Count unique employees
const uniqueEmps = new Set(rosterRows.map(r => r.employeeId)).size;

let printHtml = `
<html>
<head>
    <title>Daily Roster - ${dept} - ${date}</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: 'Segoe UI', Arial, sans-serif; padding: 15px; color: #1e293b;
font-size: 11px; }
        .print-header { text-align: center; border-bottom: 3px solid #0d9488; padding-bottom:
10px; margin-bottom: 12px; }
        .print-header h1 { font-size: 16px; color: #0d9488; letter-spacing: 1px; }
        .print-header .print-subtitle { font-size: 12px; color: #64748b; margin-top: 3px; }
        .print-meta { display: flex; justify-content: space-between; font-size: 11px; color:
#475569; margin-bottom: 10px; padding: 6px 10px; background: #f8fafc; border-radius: 4px; }
        table { width: 100%; border-collapse: collapse; font-size: 10px; }
        th { background: #e2e8f0; padding: 5px 6px; text-align: left; font-weight: 700; font-
size: 9px; text-transform: uppercase; letter-spacing: 0.4px; color: #334155; border: 1px solid
#cbd5e1; }
        td { padding: 4px 6px; border: 1px solid #e2e8f0; vertical-align: top; }
        .emp-row { background: #f0fdf4; font-weight: 600; }
        .priority-high { color: #dc2626; font-weight: 700; }
        .priority-medium { color: #d97706; }
        .priority-low { color: #16a34a; }
        .status-done { background: #dcfce7; color: #166534; padding: 1px 6px; border-radius:
3px; font-size: 9px; }
        .status-ongoing { background: #dbeafe; color: #1e40af; padding: 1px 6px; border-
```

```
radius: 3px; font-size: 9px; }
        .status-pending { background: #fff7ed; color: #c2410c; padding: 1px 6px; border-
radius: 3px; font-size: 9px; }
        .supervisor-notes { margin-top: 12px; padding: 8px 10px; background: #f0fdf4; border:
1px solid #bbf7d0; border-radius: 4px; font-size: 11px; }
        .supervisor-notes strong { color: #166534; }
        .footer { margin-top: 20px; text-align: center; font-size: 9px; color: #94a3b8;
border-top: 1px solid #e2e8f0; padding-top: 6px; }
        .signature-area { margin-top: 25px; display: flex; justify-content: space-between; }
        .signature-box { width: 200px; border-top: 1px solid #334155; text-align: center;
padding-top: 4px; font-size: 10px; color: #475569; }
        @media print { body { padding: 8px; } }
    </style>
</head>
<body>
    <div class="print-header">
        <h1>INNOVATIVE ENGINEERING SOLUTIONS</h1>
        <div class="print-subtitle">Daily Roster</div>
    </div>
    <div class="print-meta">
        <span><strong>Date:</strong> ${new Date(date).toLocaleDateString('en-IN', { weekday:
'long', day: '2-digit', month: 'short', year: 'numeric' })}</span>
        <span><strong>Department:</strong> ${dept}</span>
        <span><strong>Employees:</strong> ${uniqueEmps} | <strong>Tasks:</strong>
${rosterRows.length}</span>
    </div>
    <table>
        <thead>
            <tr>
                <th>#</th>
                <th>Employee & Timing</th>
                <th>IO No</th>
                <th>Drawing</th>
                <th>Description</th>
                <th>Customer</th>
                <th>Qty</th>
                <th>Prod. Value</th>
                <th>Total Overhead</th>
                <th>MP</th>
                <th>Assigned With</th>
                <th>Work Duration</th>
                <th>Status</th>
            </tr>
        </thead>
        <tbody>`;

let lastEmpId = '';
let rowNum = 0;

rosterRows.forEach((row) => {
    const isNewEmployee = row.employeeId !== lastEmpId;
    lastEmpId = row.employeeId;
```

```
    if (isNewEmployee) rowNum++;

    const pClass = row.priority === 'High' ? 'priority-high' : row.priority === 'Medium' ?
'priority-medium' : 'priority-low';
    const sClass = row.status === 'Completed' ? 'status-done' : row.status === 'Ongoing' ?
'status-ongoing' : 'status-pending';

    printHtml += `
            <tr class="${isNewEmployee ? 'emp-row' : ''}">
                <td>${isNewEmployee ? rowNum : ''}</td>
                <td>
                    ${isNewEmployee ? `<strong>${row.employeeName}</strong><br>
<small>${row.employeeNo || ''} · ${row.department || ''} · ${row.role || ''}</small><br><div
style="font-size:8px; margin-top:3px; color:#64748b;">In: ${row.inTime || '-'} · Out:
${row.outTime || '-'}</div>` : ''}
                </td>
                <td style="font-weight:600">${row.orderNo || 'Ad-hoc'}</td>
                <td>${row.drawingNo || '-'}</td>
                <td class="${pClass}">${row.description || '-'}</td>
                <td>${row.customer || '-'}</td>
                <td>${row.qty || '-'} ${row.unit || ''}</td>
                <td>${(row.prodValueEa > 0 && row.qty > 0) ? (row.prodValueEa *
row.qty).toFixed(2) : '-'}</td>
                <td>${row.totalOverheads > 0 ? `₹${row.totalOverheads.toFixed(2)}` : '-'}</td>
                <td>${row.manpower || '-'}</td>
                <td>${row.assignedWith || '-'}</td>
                <td>${row.workStart || '-'} to ${row.workEnd || '-'}</td>
                <td><span class="${sClass}">${row.status || 'Pending'}</span></td>
            </tr>`;
});

// Add Grand Total row for Print
const taskOverheads = {};
const taskProdValues = {};

rosterRows.forEach(row => {
    if (row.taskId) {
        taskOverheads[row.taskId] = parseFloat(row.totalOverheads) || 0;
        taskProdValues[row.taskId] = (parseFloat(row.prodValueEa) || 0) * (parseFloat(row.qty)
|| 0);
    }
});

const grandTotalOverheads = Object.values(taskOverheads).reduce((sum, val) => sum + val, 0);
const grandTotalProdValue = Object.values(taskProdValues).reduce((sum, val) => sum + val, 0);

if (grandTotalOverheads > 0 || grandTotalProdValue > 0) {
    printHtml += `
        <tr style="background: #f8fafc; font-weight: 700;">
            <td colspan="7" style="text-align: right; border: 1px solid #cbd5e1;">GRAND
TOTALS</td>
            <td style="border: 1px solid #cbd5e1; text-align: right;">₹
```

```
${grandTotalProdValue.toFixed(2)}</td>
            <td style="border: 1px solid #cbd5e1; text-align: right;">₹
${grandTotalOverheads.toFixed(2)}</td>
            <td colspan="3" style="border: 1px solid #cbd5e1;"></td>
        </tr>`;
}

printHtml += `</tbody></table>`;

if (notes) {
    printHtml += `<div class="supervisor-notes"><strong>Supervisor Notes:</strong> ${notes}
</div>`;
}

printHtml += `
    <div class="signature-area">
        <div class="signature-box">Supervisor Signature</div>
        <div class="signature-box">Manager Signature</div>
    </div>
    <div class="footer">Generated on ${new Date().toLocaleString('en-IN')} · IES Groups Admin
Portal</div>
</body></html>`;

const printWindow = window.open('', '_blank', 'width=900,height=600');
printWindow.document.write(printHtml);
printWindow.document.close();
printWindow.focus();
setTimeout(() => printWindow.print(), 300);
```

};

// ===== TEAM FILTER =====

export const filterTeam = (val) => { const list = document.getElementById('wf-assign-with-list'); if (!list) return; const items = list.querySelectorAll('.wf-team-item'); const search = val.toLowerCase(); items.forEach(item => { const name = item.querySelector('span').textContent.toLowerCase(); if (name.includes(search)) { item.style.display = 'flex'; } else { item.style.display = 'none'; } }); }; `\n \n\n\n### File: e:\re\Innovative Engineering Solutions\assets\admin\js\app.js\n*Description: Admin Core Logic (Routing)*\n\n` javascript\nimport * as Auth from './auth.js'; import * as UI from './ui.js'; import * as DB from './db.js'; import * as Charts from './charts.js'; import * as Monitoring from './monitoring.js'; import * as Inventory from './inventory.js'; import * as Workflow from './workflow.js';

// App State let currentMembers = []; let currentProjects = []; let currentOrders = []; let isTrashView = false; let isProjectTrashView = false; let projectViewMode = localStorage.getItem('projectViewMode') || 'grid'; let currentInventory = []; let currentTransactions = []; let inventoryUnsubscribe = null; let transactionUnsubscribe = null; let isInventoryTrashView = false; let currentInventoryTab = 'master';

// Helper: Member Search Handling function setupMemberSearch(containerId, inputClass, hiddenInputId, onSelectChange) { const container = document.getElementById(containerId); if (!container) return;

```
const tagsContainer = container.querySelector('.search-tags-container');
const input = container.querySelector('.' + inputClass);
const hiddenInput = document.getElementById(hiddenInputId);
```

```
const dropdown = container.querySelector('.search-suggestions-dropdown');

let selectedMembers = [];

// Initialize from hidden input if it has a value
if (hiddenInput && hiddenInput.value) {
    try {
        selectedMembers = JSON.parse(hiddenInput.value);
        renderTags();
    } catch (e) {
        // Fallback for comma-separated string if it's not JSON
        selectedMembers = hiddenInput.value.split(',').map(s => s.trim()).filter(s => s);
        renderTags();
    }
}

function renderTags() {
    if (!tagsContainer) return;
    const currentInput = input.value;
    const tagHTML = selectedMembers.map(m => `
        <span class="member-tag">
            ${m}
            <span class="member-tag-remove" data-member="${m}">&times;</span>
        </span>
    `).join('');

    tagsContainer.innerHTML = tagHTML;
    tagsContainer.appendChild(input);
    input.value = currentInput;

    if (hiddenInput) {
        hiddenInput.value = JSON.stringify(selectedMembers);
        // Trigger change event if needed
        hiddenInput.dispatchEvent(new Event('change'));
    }

    if (onSelectChange) onSelectChange(selectedMembers);
}

function showSuggestions(term) {
    if (!dropdown) return;
    const filtered = currentMembers.filter(m =>
        m.name.toLowerCase().includes(term.toLowerCase()) &&
        !selectedMembers.includes(m.name)
    );

    if (filtered.length === 0 || term === '') {
        dropdown.innerHTML = '';
        dropdown.classList.add('hidden');
        return;
    }
```

```
    dropdown.innerHTML = filtered.map(m => `
        <div class="suggestion-item" data-name="${m.name}">${m.name}</div>
    `).join('');
    dropdown.classList.remove('hidden');
}

input.addEventListener('input', (e) => {
    showSuggestions(e.target.value);
});

input.addEventListener('focus', () => {
    showSuggestions(input.value);
});

// Close dropdown on click outside
document.addEventListener('click', (e) => {
    if (!container.contains(e.target)) {
        dropdown?.classList.add('hidden');
    }
});

container.addEventListener('click', (e) => {
    if (e.target.classList.contains('suggestion-item')) {
        const name = e.target.dataset.name;
        if (!selectedMembers.includes(name)) {
            selectedMembers.push(name);
            input.value = '';
            dropdown.classList.add('hidden');
            renderTags();
        }
    } else if (e.target.classList.contains('member-tag-remove')) {
        const name = e.target.dataset.member;
        selectedMembers = selectedMembers.filter(m => m !== name);
        renderTags();
    } else if (e.target === tagsContainer || e.target.classList.contains('member-tag')) {
        input.focus();
    }
});

// Initial render if members were pre-loaded
renderTags();
```

}

// Global App Object function calculateDashboardStats(orders, selectedMonth = 'all', selectedDept = 'all') { // Filter by month and department const filteredOrders = orders.filter(o => { if (o.isTrash) return false;

```
    const matchesMonth = selectedMonth === 'all' || (o.date &&
o.date.startsWith(selectedMonth));
    const matchesDept = selectedDept === 'all' || o.department === selectedDept;

    return matchesMonth && matchesDept;
```

```
});

const active = filteredOrders.filter(o => o.status === 'Pending');
const delivered = filteredOrders.filter(o => o.status === 'Delivered');

const parseTotal = (val) => {
    if (typeof val === 'number') return val;
    if (typeof val === 'string') return parseFloat(val.replace(/,/g, '')) || 0;
    return 0;
};

// Revenue is now sum of all Pending orders in selection (Using In-house Value only)
const revenue = active.reduce((sum, o) => sum + parseTotal(o.prodValueEa), 0);
const pendingCount = active.length;

// Unassigned orders: Pending orders with no assignment
const unassignedCount = active.filter(o => !o.assignedTo || o.assignedTo.length === 0).length;

const totalCount = active.length + delivered.length;

// Pipeline percentages
const pendingPct = totalCount > 0 ? Math.round((pendingCount / totalCount) * 100) : 0;
const deliveredPct = totalCount > 0 ? Math.round((delivered.length / totalCount) * 100) : 100;

return {
    revenue,
    activeOrders: active.length,
    pendingCount,
    unassignedCount,
    totalMembers: currentMembers.length,
    pendingPct,
    deliveredPct,
    filteredOrders
};
```

}

// Helper: Refresh Dashboard UI function refreshDashboard() { const monthFilter = document.getElementById('dashboard-month-filter'); const deptFilter = document.getElementById('dashboard-dept-filter');

```
const selectedMonth = monthFilter ? monthFilter.value : 'all';
const selectedDept = deptFilter ? deptFilter.value : 'all';

const stats = calculateDashboardStats(currentOrders, selectedMonth, selectedDept);
UI.updateStats(stats);
UI.renderDashboardPendingOrders(stats.filteredOrders);
UI.renderDashboardRecentActivity(stats.filteredOrders);
```

}

// Global App Object window.adminApp = { currentEditingProjectId: null, switchView: (viewName) => { UI.switchView(viewName); if (viewName === 'inventory_management') { window.adminApp.initInventory(); } else if

(inventoryUnsubscribe) { inventoryUnsubscribe(); inventoryUnsubscribe = null; } if (viewName === 'daily_roster') { Workflow.initWorkflowView(); } },

```
refreshDashboard: () => {
    refreshDashboard();
},

// Definitions
rolesList: [
    "Director", "Managing Director", "General Manager",
    "Business Development Manager", "Section Head", "Manager", "Assistant Manager",
    "Senior Engineer", "Design Engineer", "Quality Engineer", "Production Engineer",
    "Supervisor", "Foreman", "Technician", "Operator",
    "CNC Operator", "VMC Operator", "Welder", "Fitter", "Electrician",
    "Accountant", "HR Manager", "HR Executive", "Sales Executive",
    "Office Admin", "Store Keeper", "Helper"
],
selectedRoles: new Set(),

// Multi-Select Helpers
toggleRoleDropdown: () => {
    const options = document.getElementById('role-dropdown-options');
    if (options) options.classList.toggle('hidden');
},

populateRoleOptions: () => {
    const container = document.getElementById('role-dropdown-options');
    if (!container) return;

    container.innerHTML = window.adminApp.rolesList.map(role => `
        <div class="select-option ${window.adminApp.selectedRoles.has(role) ? 'selected' :
''}"
             onclick="window.adminApp.selectRole('${role}')">
            ${role}
            ${window.adminApp.selectedRoles.has(role) ? '<svg class="w-4 h-4 ml-auto text-
emerald-600" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-
linecap="round" stroke-linejoin="round" stroke-width="2" d="M5 13l4 4L19 7"></path></svg>' :
''}
        </div>
    `).join('');
},

selectRole: (role) => {
    if (window.adminApp.selectedRoles.has(role)) {
        window.adminApp.selectedRoles.delete(role);
    } else {
        window.adminApp.selectedRoles.add(role);
    }
    window.adminApp.updateRoleDisplay();
    window.adminApp.populateRoleOptions(); // Re-render to update classes
},
```

```
updateRoleDisplay: () => {
    const display = document.getElementById('role-display-text');
    const input = document.getElementById('roles-input');
    if (!display || !input) return;

    const roles = Array.from(window.adminApp.selectedRoles);
    input.value = JSON.stringify(roles);

    if (roles.length === 0) {
        display.innerHTML = '<span class="text-slate-400">Select Roles...</span>';
    } else {
        display.innerHTML = roles.map(r => `
            <span class="role-tag">${r}
                <span onclick="event.stopPropagation(); window.adminApp.selectRole('${r}')"
class="ml-1 hover:text-red-500 cursor-pointer">&times;</span>
            </span>
        `).join('');
    }
},

// Department Multi-Select - Matching Org Tree + Management
departmentsList: ["Management", "Fabrication", "CNC & VMC", "SPM", "HR"],
selectedDepts: new Set(),

toggleDeptDropdown: () => {
    const options = document.getElementById('dept-dropdown-options');
    if (options) options.classList.toggle('hidden');
},

populateDeptOptions: () => {
    const container = document.getElementById('dept-dropdown-options');
    if (!container) return;

    container.innerHTML = window.adminApp.departmentsList.map(dept => `
        <div class="select-option ${window.adminApp.selectedDepts.has(dept) ? 'selected' :
''}"
             onclick="window.adminApp.selectDept('${dept}')">
            ${dept}
            ${window.adminApp.selectedDepts.has(dept) ? '<svg class="w-4 h-4 ml-auto text-
emerald-600" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-
linecap="round" stroke-linejoin="round" stroke-width="2" d="M5 13l4 4L19 7"></path></svg>' :
''}
        </div>
    `).join('');
},

selectDept: (dept) => {
    if (window.adminApp.selectedDepts.has(dept)) {
        window.adminApp.selectedDepts.delete(dept);
    } else {
        window.adminApp.selectedDepts.add(dept);
    }
```

```
    window.adminApp.updateDeptDisplay();
    window.adminApp.populateDeptOptions();
},

updateDeptDisplay: () => {
    const display = document.getElementById('dept-display-text');
    const input = document.getElementById('departments-input');
    if (!display || !input) return;

    const depts = Array.from(window.adminApp.selectedDepts);
    input.value = JSON.stringify(depts);

    if (depts.length === 0) {
        display.innerHTML = '<span class="text-slate-400">Select Departments...</span>';
    } else {
        display.innerHTML = depts.map(d => `
            <span class="role-tag" style="background: var(--blue-50); color: var(--blue-
700);">${d}
                <span onclick="event.stopPropagation(); window.adminApp.selectDept('${d}')"
class="ml-1 hover:text-red-500 cursor-pointer">&times;</span>
            </span>
        `).join('');
    }
},

populateManagers: (selectedId = null) => {
    const select = document.getElementById('manager-select');
    if (!select) return;

    // Filter out current member if editing (prevent self-reporting)
    const currentMemberId = document.getElementById('memberId-input')?.value;
    const candidates = currentMembers.filter(m => m.id !== currentMemberId);

    select.innerHTML = '<option value="">Select Manager</option>' +
        candidates.map(m => `<option value="${m.id}" ${m.id === selectedId ? 'selected' :
''}>${m.name} (${m.designation || 'Member'})</option>`).join('');
},

openAddMemberModal: (memberId = null) => {
    const modal = document.getElementById('add-member-modal');
    if (!modal) return;

    // Reset form
    const form = document.getElementById('add-member-form');
    if (form) form.reset();

    // Reset hidden ID field
    const hiddenId = document.getElementById('memberId-input');
    if (hiddenId) hiddenId.value = '';

    // Update modal title
    const title = modal.querySelector('.modal-title');
```

```
    if (title) title.textContent = memberId ? 'Edit Team Member' : 'Add Team Member';

    // Initialize Multi-Select for Roles and Departments
    window.adminApp.selectedRoles.clear();
    window.adminApp.selectedDepts.clear();
    if (!memberId) {
        window.adminApp.updateRoleDisplay();
        window.adminApp.updateDeptDisplay();
    }

    window.adminApp.populateRoleOptions();
    window.adminApp.populateDeptOptions();
    window.adminApp.populateManagers();

    window.adminApp.openModal('add-member-modal');
},

editMember: (memberId) => {
    const member = currentMembers.find(m => m.id === memberId);
    if (!member) return;

    // Open modal first
    window.adminApp.openAddMemberModal(memberId);

    // Set hidden ID
    const hiddenId = document.getElementById('memberId-input');
    if (hiddenId) hiddenId.value = memberId;

    // Populate form fields
    const form = document.getElementById('add-member-form');
    if (!form) return;

    const setField = (name, value) => {
        const el = form.querySelector(`[name="${name}"]`);
        if (el) el.value = value || '';
    };

    setField('name', member.name);
    setField('employeeId', member.employeeId);
    setField('email', member.email);
    setField('phone', member.phone);
    setField('section', member.section || member.department);
    setField('status', member.status);
    setField('joiningDate', member.joiningDate);
    setField('overheads', member.overheads || '');

    // Populate Roles
    window.adminApp.selectedRoles.clear();
    if (member.orgRoles && Array.isArray(member.orgRoles)) {
        member.orgRoles.forEach(r => window.adminApp.selectedRoles.add(r));
    } else if (member.role || member.designation) {
        window.adminApp.selectedRoles.add(member.role || member.designation);
```

```
    }
    window.adminApp.updateRoleDisplay();
    window.adminApp.populateRoleOptions();

    // Populate Departments
    window.adminApp.selectedDepts.clear();
    if (member.departments && Array.isArray(member.departments)) {
        member.departments.forEach(d => window.adminApp.selectedDepts.add(d));
    } else if (member.department) {
        window.adminApp.selectedDepts.add(member.department);
    }
    window.adminApp.updateDeptDisplay();
    window.adminApp.populateDeptOptions();

    // Populate Manager
    window.adminApp.populateManagers(member.reportingManagerId);
},

deleteMember: (memberId, memberName) => {
    window.adminApp.showConfirmModal(
        "Delete Member?",
        `Are you sure you want to delete "${memberName}"? This action cannot be undone.`,
        async () => {
            const result = await DB.deleteMember(memberId);
            if (result.error) {
                alert('Error deleting member: ' + result.error);
            }
        }
    );
},

submitMemberForm: async () => {
    const form = document.getElementById('add-member-form');
    if (!form.checkValidity()) {
        form.reportValidity();
        return;
    }

    const formData = new FormData(form);
    const memberId = formData.get('memberId');

    const memberData = {
        name: formData.get('name'),
        employeeId: formData.get('employeeId') || '',
        email: formData.get('email') || '',
        phone: formData.get('phone') || '',

        // Roles Handling
        orgRoles: Array.from(window.adminApp.selectedRoles),
        role: Array.from(window.adminApp.selectedRoles)[0] || '', // Primary Role
        designation: Array.from(window.adminApp.selectedRoles)[0] || '', // Backward Compat
```

```
        // Departments Handling
        departments: Array.from(window.adminApp.selectedDepts),
        section: Array.from(window.adminApp.selectedDepts)[0] || '', // Primary Dept
        department: Array.from(window.adminApp.selectedDepts)[0] || '', // Backward Compat

        status: formData.get('status') || 'Active',
        joiningDate: formData.get('joiningDate') || '',
        overheads: parseFloat(formData.get('overheads')) || 0,
        reportingManagerId: formData.get('reportingManager') || null
    };

    let result;
    if (memberId) {
        result = await DB.updateMember(memberId, memberData);
    } else {
        result = await DB.addMember(memberData);
    }

    if (result.error) {
        alert('Error: ' + result.error);
    } else {
        window.adminApp.closeModal('add-member-modal');
    }
},

openAddProjectModal: (projectId = null) => {
    const modal = document.getElementById('add-project-modal');
    if (!modal) return;

    const form = document.getElementById('add-project-form');
    if (form) form.reset();

    const title = modal.querySelector('.modal-title');
    const submitBtn = form?.querySelector('button[type="submit"]');

    if (projectId) {
        // Edit mode
        const project = currentProjects.find(p => p.id === projectId);
        if (!project) return;

        if (title) title.textContent = 'Edit Project';
        if (submitBtn) submitBtn.textContent = 'Save Changes';

        // Pre-fill
        const setVal = (name, val) => { const el = form.querySelector(`[name="${name}"]`); if
(el) el.value = val || ''; };
        setVal('name', project.name);
        setVal('customerName', project.customerName);
        setVal('jobType', project.jobType);
        setVal('drawingSource', project.drawingSource);
        setVal('expectedCompletion', project.expectedCompletion);
        setVal('internalNotes', project.internalNotes);
```

```
        // Store edit ID
        modal.dataset.editId = projectId;
    } else {
        if (title) title.textContent = 'Initialize New Project';
        if (submitBtn) submitBtn.textContent = 'Initialize Project';
        delete modal.dataset.editId;
    }

    window.adminApp.openModal('add-project-modal');
},

submitProjectForm: async () => {
    const form = document.getElementById('add-project-form');
    if (!form.checkValidity()) {
        form.reportValidity();
        return;
    }

    const submitBtn = form.querySelector('button[type="submit"]');
    if (submitBtn) submitBtn.disabled = true;

    try {
        const formData = new FormData(form);
        const projectData = {
            name: formData.get('name'),
            customerName: formData.get('customerName'),
            jobType: formData.get('jobType'),
            drawingSource: formData.get('drawingSource'),
            expectedCompletion: formData.get('expectedCompletion') || null,
            internalNotes: formData.get('internalNotes') || '',
        };

        const modal = document.getElementById('add-project-modal');
        const editId = modal?.dataset.editId;

        if (editId) {
            // Update existing
            const result = await DB.updateProject(editId, projectData, 'Project Edited');
            if (result.error) {
                alert('Error: ' + result.error);
            } else {
                window.adminApp.closeModal('add-project-modal');
                form.reset();
                delete modal.dataset.editId;
            }
        } else {
            // Create new
            const result = await DB.addProject(projectData);
            if (result.error) {
                alert('Error: ' + result.error);
            } else {
```

```
                window.adminApp.closeModal('add-project-modal');
                form.reset();
            }
        }
    } finally {
        if (submitBtn) submitBtn.disabled = false;
    }
},

filterProjects: () => {
    const searchTerm = document.getElementById('project-search')?.value.toLowerCase() || '';
    const statusFilter = document.getElementById('project-status-filter')?.value || 'all';
    const typeFilter = document.getElementById('project-type-filter')?.value || 'all';

    const filtered = currentProjects.filter(p => {
        // Filter based on trash view
        const isDeleted = !!p.isDeleted;
        if (isProjectTrashView !== isDeleted) return false;

        // Cross-reference drawing number for search
        let drgNo = p.drawingNo || '';
        if (!drgNo && window.adminApp.getCurrentOrders) {
            const orders = window.adminApp.getCurrentOrders();
            const matchingOrder = orders.find(o => o.internalOrderNo === p.projectId);
            if (matchingOrder) drgNo = matchingOrder.drawingNo || '';
        }

        const matchesSearch = (p.name?.toLowerCase().includes(searchTerm) ||
            p.projectId?.toLowerCase().includes(searchTerm) ||
            p.customerName?.toLowerCase().includes(searchTerm) ||
            drgNo.toLowerCase().includes(searchTerm));
        const matchesStatus = statusFilter === 'all' || p.status === statusFilter;
        const matchesType = typeFilter === 'all' || p.jobType === typeFilter;

        return matchesSearch && matchesStatus && matchesType;
    });

    window.adminApp.renderProjectCards(filtered);
    window.adminApp.updateProjectStats();
},

setProjectView: (mode) => {
    window.adminApp.projectViewMode = mode;
    localStorage.setItem('projectViewMode', mode);

    // Update Toggle Buttons
    const gridBtn = document.getElementById('btn-view-grid');
    const listBtn = document.getElementById('btn-view-list');
    if (gridBtn) gridBtn.classList.toggle('active', mode === 'grid');
    if (listBtn) listBtn.classList.toggle('active', mode === 'list');

    window.adminApp.filterProjects();
```

```
},

renderProjectTable: (projects) => {
    const tbody = document.getElementById('project-list-body');
    if (!tbody) return;

    if (projects.length === 0) {
        tbody.innerHTML = '<tr><td colspan="6" class="text-center py-12 text-slate-400">No
projects found.</td></tr>';
        return;
    }

    tbody.innerHTML = projects.map(p => {
        const statusClass = (p.status || 'draft').toLowerCase().replace(/\s+/g, '-');
        const isTrashed = !!p.isDeleted;
        const deliveryDate = p.expectedCompletion ? new
Date(p.expectedCompletion).toLocaleDateString('en-IN', { day: '2-digit', month: 'short' }) :
'N/A';

        // Cross-reference Drawing No if missing
        let drgNo = p.drawingNo || '';
        if (!drgNo && window.adminApp.getCurrentOrders) {
            const orders = window.adminApp.getCurrentOrders();
            const matchingOrder = orders.find(o => o.internalOrderNo === p.projectId);
            if (matchingOrder) drgNo = matchingOrder.drawingNo || '';
        }

        const displayDrg = drgNo ? `<span class="pm-table-project-drg text-xs font-semibold
px-2 py-0.5 rounded ml-2" style="background: var(--brand-50); color: var(--brand-700); border:
1px solid var(--brand-200);">DRG: ${drgNo}</span>` : '';



        const actions = isTrashed ? `
            <button class="pm-action-btn restore-btn" onclick="event.stopPropagation();
window.adminApp.restoreProject('${p.id}')">Restore</button>
            <button class="pm-action-btn" style="color:#ef4444;"
onclick="event.stopPropagation();
window.adminApp.permanentDeleteProject('${p.id}')">Delete</button>
        ` : `
            <div class="flex gap-2 justify-end">
                <button class="pm-action-btn" onclick="event.stopPropagation();
window.adminApp.editProject('${p.id}')">Edit</button>
                <button class="pm-action-btn deep-dive" onclick="event.stopPropagation();
window.adminApp.viewProjectDetails('${p.id}')">Report</button>
            </div>
        `;

        return `
            <tr class="${isTrashed ? 'opacity-50' : ''}">
                <td>
                    <div class="pm-table-project-info">
                        <span class="pm-table-project-name">${p.name}</span>
```

```
                        <div class="flex items-center mt-1">
                            <span class="pm-table-project-id">${p.projectId}</span>
                            ${displayDrg}
                        </div>
                    </div>
                </td>
                <td><span class="pm-table-customer">${p.customerName}</span></td>
                <td><span class="version-tag">${p.jobType || 'N/A'}</span></td>
                <td><span class="status-badge ${statusClass}">${p.status}</span></td>
                <td>
                    <div class="pm-table-timeline">
                        <span class="pm-table-date-label">Delivery</span>
                        <span class="font-bold text-slate-700">${deliveryDate}</span>
                    </div>
                </td>
                <td class="text-right">${actions}</td>
            </tr>
        `;
    }).join('');
},

updateProjectStats: () => {
    const active = currentProjects.filter(p => !p.isDeleted);
    const completed = active.filter(p => p.status === 'Completed');
    const trashed = currentProjects.filter(p => !!p.isDeleted);

    const setEl = (id, val) => { const el = document.getElementById(id); if (el)
el.textContent = val; };
    setEl('pm-stat-total', active.length);
    setEl('pm-stat-active', active.length - completed.length);
    setEl('pm-stat-completed', completed.length);
    setEl('pm-stat-trash', trashed.length);

    // Trash count badge
    const trashCount = document.getElementById('pm-trash-count');
    if (trashCount) {
        trashCount.textContent = trashed.length;
        trashCount.style.display = trashed.length > 0 ? 'inline-flex' : 'none';
    }
},

toggleProjectTrash: () => {
    isProjectTrashView = !isProjectTrashView;

    const toggleBtn = document.getElementById('pm-trash-toggle-btn');
    const toggleLabel = document.getElementById('pm-trash-toggle-label');
    const trashBanner = document.getElementById('pm-trash-banner');
    const filterBar = document.getElementById('pm-filter-bar');
    const newBtn = document.getElementById('pm-new-project-btn');

    if (isProjectTrashView) {
        if (toggleBtn) toggleBtn.classList.add('active');
```

```
        if (toggleLabel) toggleLabel.textContent = 'Active Projects';
        if (trashBanner) trashBanner.style.display = 'flex';
        if (filterBar) filterBar.style.display = 'none';
        if (newBtn) newBtn.style.display = 'none';
    } else {
        if (toggleBtn) toggleBtn.classList.remove('active');
        if (toggleLabel) toggleLabel.textContent = 'Trash';
        if (trashBanner) trashBanner.style.display = 'none';
        if (filterBar) filterBar.style.display = '';
        if (newBtn) newBtn.style.display = '';
    }

    window.adminApp.filterProjects();
},

trashProject: (projectId) => {
    window.adminApp.showConfirmModal(
        "Move to Trash?",
        "This project will be moved to the trash. You can restore it later.",
        async () => {
            const result = await DB.softDeleteProject(projectId);
            if (result.error) alert('Error: ' + result.error);
        }
    );
},

restoreProject: async (projectId) => {
    const result = await DB.restoreProject(projectId);
    if (result.error) alert('Error restoring: ' + result.error);
},

permanentDeleteProject: (projectId) => {
    window.adminApp.showConfirmModal(
        "Permanently Delete?",
        "This project will be permanently deleted. This cannot be undone.",
        async () => {
            const result = await DB.deleteProject(projectId);
            if (result.error) alert('Error: ' + result.error);
        }
    );
},

editProject: (projectId) => {
    window.adminApp.openAddProjectModal(projectId);
},

// === INVENTORY MANAGEMENT ===
initInventory: () => {
    if (inventoryUnsubscribe) return;
    inventoryUnsubscribe = Inventory.getInventory((items) => {
        currentInventory = items;
        window.adminApp.renderInventoryList(items);
```

```
        window.adminApp.updateInventoryStats(items);
    });
},

toggleInventoryTrash: () => {
    isInventoryTrashView = !isInventoryTrashView;
    const btn = document.getElementById('inventory-trash-btn');
    const badge = document.querySelector('.pm-header-badge.inventory');

    if (isInventoryTrashView) {
        btn.classList.add('active');
        btn.innerHTML = `
            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M11
15l-3-3m0 0l3-3m-3 3h8M3 12a9 9 0 1118 0 9 9 0 01-18 0z" />
            </svg>
            <span>Back</span>
        `;
        if (badge) {
            badge.textContent = '📦 Deleted Items (Trash)';
            badge.classList.add('deleted');
        }
    } else {
        btn.classList.remove('active');
        btn.innerHTML = `
            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19
7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-
1h-4a1 1 0 00-1 1v3M4 7h16" />
            </svg>
            <span>Trash</span>
        `;
        if (badge) {
            badge.textContent = 'Stock Control';
            badge.classList.remove('deleted');
        }
    }
    window.adminApp.renderInventoryList(currentInventory);
},

trashInventoryItem: async (id, name) => {
    if (!confirm(`Move "${name}" to trash?`)) return;
    const result = await Inventory.softDeleteInventoryItem(id);
    if (result.error) alert('Error: ' + result.error);
},

restoreInventoryItem: async (id) => {
    const result = await Inventory.restoreInventoryItem(id);
    if (result.error) alert('Error: ' + result.error);
},

permanentDeleteInventoryItem: async (id, name) => {
```

```
    if (!confirm(`Permanently delete "${name}"? This action cannot be undone.`)) return;
    const result = await Inventory.permanentDeleteInventoryItem(id);
    if (result.error) alert('Error: ' + result.error);
},



renderInventoryList: (items) => {
    const body = document.getElementById('inventory-list-body');
    const table = body.closest('table');
    const headerRow = table ? table.querySelector('thead tr') : null;
    if (!body) return;

    // Filter items based on trash view
    const displayItems = items.filter(item => !!item.isDeleted === isInventoryTrashView);

    // Adjust Table Header for Trash View
    if (headerRow) {
        if (isInventoryTrashView) {
            headerRow.innerHTML = `
                <th class="cr-emerald-bg">Item Details</th>
                <th class="cr-emerald-bg">Category</th>
                <th class="cr-emerald-bg">Last Stock</th>
                <th class="cr-emerald-bg">Deleted On</th>
                <th class="cr-emerald-bg text-right">Actions</th>
            `;
        } else {
            headerRow.innerHTML = `
                <th class="cr-emerald-bg">Item Details</th>
                <th class="cr-emerald-bg">Category</th>
                <th class="cr-emerald-bg">Location</th>
                <th class="cr-emerald-bg text-center">Current Stock</th>
                <th class="cr-emerald-bg">Status</th>
                <th class="cr-emerald-bg text-right">Actions</th>
            `;
        }
    }

    if (displayItems.length === 0) {
        const colspan = isInventoryTrashView ? 5 : 6;
        body.innerHTML = `
            <tr>
                <td colspan="${colspan}" class="p-12 text-center text-slate-400">
                    <div class="flex flex-col items-center gap-2">
                        <span class="text-2xl">${isInventoryTrashView ? '🗑️' : '📦'}</span>
                        <p class="italic">No ${isInventoryTrashView ? 'deleted' : 'inventory'}
items found.</p>
                    </div>
                </td>
            </tr>
        `;
        return;
```

```
    }

    body.innerHTML = displayItems.map(item => {
        const isLow = item.currentStock <= item.minimumLevel && item.currentStock > 0;
        const isOut = item.currentStock <= 0;

        let statusBadge = `<span class="badge badge-success">In Stock</span>`;
        if (isOut) statusBadge = `<span class="badge" style="background: #fef2f2; color:
#ef4444; border: 1px solid #fee2e2;">Out of Stock</span>`;
        else if (isLow) statusBadge = `<span class="badge badge-warning">Low Stock</span>`;

        // Thumbnail Logic
        let thumb = `
            <div class="inventory-icon-placeholder">
                <span>${item.category === 'Tool' ? '🔧' : (item.category === 'Raw Material' ?
'🧱' : '📦')}</span>
            </div>
        `;
        if (item.photoUrl) {
            const safeName = (item.name || '').replace(/'/g, "\\'");
            thumb = `<img src="${item.photoUrl}" class="inventory-thumb cursor-pointer
hover:ring-2 hover:ring-teal-500 transition-all" alt="${item.name}"
onclick="event.stopPropagation(); window.adminApp.openPhotoViewer('${item.photoUrl}',
'${safeName}')">`;
        }

        if (isInventoryTrashView) {
            return `
                <tr class="border-b border-slate-50 hover:bg-slate-50 transition-colors">
                    <td class="p-3">
                        <div class="flex items-center gap-3">
                            ${thumb}
                            <div>
                                <div class="font-bold text-slate-700">${item.name}</div>
                            </div>
                        </div>
                    </td>
                    <td class="p-3 text-slate-500 font-medium">${item.category}</td>
                    <td class="p-3 text-slate-700 font-bold">${item.currentStock} ${item.unit}
</td>
                    <td class="p-3 text-slate-400 text-xs">${item.updatedAt ? new
Date(item.updatedAt.seconds * 1000).toLocaleDateString() : 'Recently'}</td>
                    <td class="p-3 text-right">
                        <div class="flex justify-end gap-2">
                            <button class="btn btn-ghost btn-sm text-green-600"
onclick="window.adminApp.restoreInventoryItem('${item.id}')" title="Restore">Restore</button>
                            <button class="btn btn-ghost btn-sm text-red-600"
onclick="window.adminApp.permanentDeleteInventoryItem('${item.id}', '${item.name}')"
title="Delete Permanently">Delete</button>
                        </div>
                    </td>
                </tr>
```

```
            `;
        }

        return `
            <tr class="border-b border-slate-50 hover:bg-slate-50 transition-colors">
                <td class="p-3">
                    <div class="flex items-center gap-3">
                        ${thumb}
                        <div>
                            <div class="font-bold text-slate-700">${item.name}</div>
                        </div>
                    </div>
                </td>
                <td class="p-3 text-slate-500 font-medium">${item.category}</td>
                <td class="p-3 text-slate-500">${item.location || '-'}</td>
                <td class="p-3 text-center">
                    <div class="font-bold text-slate-700 text-lg">${item.currentStock}</div>
                    <div class="text-[10px] text-slate-400 uppercase font-bold">${item.unit}
</div>
                </td>
                <td class="p-3">${statusBadge}</td>
                <td class="p-3 text-right">
                    <div class="flex justify-end gap-2">
                        <button class="pm-c-primary-btn"
onclick='window.adminApp.openAdjustStockModal("${item.id}")'>🔄 Adjust</button>
                        <button class="action-btn delete"
onclick="window.adminApp.trashInventoryItem('${item.id}', '${item.name}')" title="Move to
Trash">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2" d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-
6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" />
                            </svg>
                        </button>
                    </div>
                </td>
            </tr>
        `;
    }).join('');
},

updateInventoryStats: (items) => {
    const total = items.length;
    const low = items.filter(i => i.currentStock <= i.minimumLevel).length;

    const setEl = (id, val) => {
        const el = document.getElementById(id);
        if (el) el.textContent = val;
    };

    setEl('inv-stat-total', total);
    setEl('inv-stat-low', low);
```

```
    // More stats can be added if price is tracked
},

openAddInventoryModal: () => {
    const form = document.getElementById('add-inventory-form');
    if (form) form.reset();

    // Populate Order ID dropdown (Internal Orders)
    const orderSelect = form.querySelector('select[name="orderId"]');
    if (orderSelect) {
        const internalOrders = window.adminApp.getCurrentOrders ?
window.adminApp.getCurrentOrders() : [];
        orderSelect.innerHTML = '<option value="">-- No Order ID --</option>' +
            internalOrders.map(o => {
                const id = o.internalOrderNo || o.id;
                return `<option value="${id}">${id} - ${o.customer || 'Order'}</option>`;
            }).join('');
    }

    // Hide photo section initially
    const photoSection = document.getElementById('inv-photo-section');
    if (photoSection) photoSection.classList.add('hidden');

    // Reset preview
    const preview = document.getElementById('inv-photo-preview');
    const placeholder = document.getElementById('inv-photo-preview-placeholder');
    if (preview) preview.classList.add('hidden');
    if (placeholder) placeholder.classList.remove('hidden');

    window.adminApp.openModal('add-inventory-modal');
},

onInventoryCategoryChange: (category) => {
    const photoSection = document.getElementById('inv-photo-section');
    if (!photoSection) return;

    if (category === 'Tool') {
        photoSection.classList.remove('hidden');
    } else {
        photoSection.classList.add('hidden');
    }
},

handleInventoryPhotoSelect: (event) => {
    const file = event.target.files[0];
    if (!file) return;

    // Validation: 1MB limit
    if (file.size > 1024 * 1024) {
        alert("File too large. Max size is 1MB.");
        event.target.value = '';
        return;
```

```
    }

    // Preview
    const reader = new FileReader();
    reader.onload = (e) => {
        const preview = document.getElementById('inv-photo-preview');
        const placeholder = document.getElementById('inv-photo-preview-placeholder');
        if (preview) {
            preview.src = e.target.result;
            preview.classList.remove('hidden');
        }
        if (placeholder) placeholder.classList.add('hidden');
    };
    reader.readAsDataURL(file);
},

handleAddInventoryItem: async (event) => {
    event.preventDefault();
    const formData = new FormData(event.target);

    const itemData = {
        name: formData.get('name'),
        price: parseFloat(formData.get('price')) || 0,
        category: formData.get('category'),
        unit: formData.get('unit'),
        currentStock: parseInt(formData.get('currentStock')) || 0,
        minimumLevel: parseInt(formData.get('minimumLevel')) || 0,
        location: formData.get('location') || '',
        orderId: formData.get('orderId') || null
    };

    const submitBtn = event.target.querySelector('button[type="submit"]');
    const originalText = submitBtn.innerHTML;

    try {
        submitBtn.disabled = true;
        submitBtn.innerHTML = "Saving...";

        const { id, error } = await Inventory.addInventoryItem(itemData);

        if (error) throw new Error(error);

        // If it's a tool and has a photo selected
        const photoInput = document.getElementById('inv-photo-input');
        if (itemData.category === 'Tool' && photoInput.files[0]) {
            submitBtn.innerHTML = "Uploading Photo...";
            const photoResult = await Inventory.uploadToolPhoto(id, photoInput.files[0]);
            if (photoResult.error) {
                alert("Item saved, but photo upload failed: " + photoResult.error);
            }
        }
```

```
        window.adminApp.closeModal('add-inventory-modal');
    } catch (err) {
        alert("Failed to add item: " + err.message);
    } finally {
        submitBtn.disabled = false;
        submitBtn.innerHTML = originalText;
    }
},

openAdjustStockModal: (itemId) => {
    const item = currentInventory.find(i => i.id === itemId);
    if (!item) return;

    const form = document.getElementById('adjust-stock-form');
    if (form) {
        form.reset();
        // Set default price from item
        const priceInput = form.querySelector('input[name="price"]');
        if (priceInput) priceInput.value = item.price || 0;
    }

    document.getElementById('adjust-item-id').value = item.id;
    document.getElementById('adjust-item-name').value = item.name;
    document.getElementById('adjust-item-name-text').textContent = item.name;
    document.getElementById('adjust-current-stock-text').textContent = `${item.currentStock}
${item.unit}`;

    // Populate Order ID dropdown
    const orderSelect = document.getElementById('adjust-project-select');
    if (orderSelect) {
        const internalOrders = window.adminApp.getCurrentOrders ?
window.adminApp.getCurrentOrders() : [];
        orderSelect.innerHTML = '<option value="">-- No Order ID --</option>' +
            internalOrders.map(o => {
                const id = o.internalOrderNo || o.id;
                return `<option value="${id}">${id} - ${o.customer || 'Order'}</option>`;
            }).join('');
    }

    // Hide project section by default (only for OUT)
    document.getElementById('adjust-project-section').classList.add('hidden');

    window.adminApp.openModal('adjust-stock-modal');
},

onStockActionChange: (action) => {
    const projectSection = document.getElementById('adjust-project-section');
    if (projectSection) {
        if (action === 'OUT') {
            projectSection.classList.remove('hidden');
        } else {
            projectSection.classList.add('hidden');
```

```
        }
    }
},

handleAdjustStock: async (event) => {
    event.preventDefault();
    const formData = new FormData(event.target);

    const itemId = formData.get('itemId');
    const itemName = formData.get('itemName');
    const type = formData.get('type');
    const quantity = parseInt(formData.get('quantity'));
    const reason = formData.get('reason');
    const orderId = formData.get('orderId'); // Renamed from projectId
    const unitPrice = parseFloat(formData.get('price')) || 0;
    const performedBy = formData.get('performedBy') || 'Admin';

    const submitBtn = document.getElementById('adjust-stock-submit');
    const originalText = submitBtn.innerHTML;

    try {
        submitBtn.disabled = true;
        submitBtn.innerHTML = "Updating...";

        // Get category for transaction record
        const item = currentInventory.find(i => i.id === itemId);
        const category = item ? (item.category || 'General') : 'General';

        const result = await Inventory.updateStock(itemId, itemName, type, quantity, reason,
orderId, unitPrice, performedBy, category);
        if (!result.success) throw new Error(result.error);

        window.adminApp.closeModal('adjust-stock-modal');
    } catch (err) {
        alert("Update failed: " + err.message);
    } finally {
        submitBtn.disabled = false;
        submitBtn.innerHTML = originalText;
    }
},

switchInventoryTab: (tab) => {
    currentInventoryTab = tab;
    const masterView = document.getElementById('subview-inventory-master');
    const ledgerView = document.getElementById('subview-inventory-ledger');
    const masterTabs = document.querySelectorAll('#tab-inventory-master');
    const ledgerTabs = document.querySelectorAll('#tab-inventory-ledger');
    const masterActions = document.getElementById('inventory-master-actions');
    const ledgerActions = document.getElementById('inventory-ledger-actions');

    if (tab === 'master') {
        masterView.classList.remove('hidden');
```

```
        ledgerView.classList.add('hidden');
        masterTabs.forEach(t => t.classList.add('active'));
        ledgerTabs.forEach(t => t.classList.remove('active'));
        masterActions.classList.remove('hidden');
        ledgerActions.classList.add('hidden');
    } else {
        masterView.classList.add('hidden');
        ledgerView.classList.remove('hidden');
        masterTabs.forEach(t => t.classList.remove('active'));
        ledgerTabs.forEach(t => t.classList.add('active'));
        masterActions.classList.add('hidden');
        ledgerActions.classList.remove('hidden');
        window.adminApp.loadInventoryTransactions();
    }

    // Apply current filters to the new tab
    window.adminApp.filterInventory();
},

printInventoryLedger: () => {
    const printDate = document.getElementById('ledger-print-date');
    if (printDate) printDate.textContent = new Date().toLocaleString();

    window.print();
},

toggleInventoryTransactions: () => {
    window.adminApp.switchInventoryTab('ledger');
},

loadInventoryTransactions: async () => {
    // Use a persistent listener for transactions if not already set
    if (!transactionUnsubscribe) {
        transactionUnsubscribe = Inventory.getInventoryTransactions((transactions) => {
            currentTransactions = transactions;
            window.adminApp.renderInventoryTransactions(transactions);
        });
    }
},

renderInventoryTransactions: (transactions) => {
    const body = document.getElementById('inventory-ledger-body');
    if (!body) return;

    if (!transactions || transactions.length === 0) {
        body.innerHTML = '<tr><td colspan="9" class="p-8 text-center text-slate-400 italic">No
transactions found.</td></tr>';
        return;
    }

    body.innerHTML = transactions.map(t => {
        const date = t.timestamp?.toDate ? t.timestamp.toDate() : (t.timestamp?.seconds ? new
```

```
Date(t.timestamp.seconds * 1000) : new Date());
        const formattedDate = date.toLocaleDateString();
        const formattedTime = date.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit'
});

        const unitPrice = t.unitPrice || 0;
        const totalCost = t.totalCost || (t.quantity * unitPrice);

        return `
            <tr class="border-b border-slate-50 hover:bg-slate-50 transition-colors">
                <td class="p-3">
                    <div class="text-slate-800 font-bold text-xs">${formattedDate}</div>
                    <div class="text-slate-400 text-[10px] uppercase">${formattedTime}</div>
                </td>
                <td class="p-3 font-medium text-slate-700">${t.itemName}</td>
                <td class="p-3 text-slate-500 font-medium text-xs">${t.category || 'General'}
</td>
                <td class="p-3"><span class="inv-trans-type ${t.type.toLowerCase()}">${t.type}
</span></td>
                <td class="p-3 text-center font-bold ${t.type === 'IN' ? 'text-green-600' :
'text-red-600'}">${t.type === 'IN' ? '+' : '-'}${t.quantity}</td>
                <td class="p-3 font-mono text-slate-600 text-xs">₹
${unitPrice.toLocaleString('en-IN', { minimumFractionDigits: 2 })}</td>
                <td class="p-3 font-bold text-slate-800 text-xs">₹
${totalCost.toLocaleString('en-IN', { minimumFractionDigits: 2 })}</td>
                <td class="p-3 text-slate-500 text-xs truncate max-w-[150px]"
title="${t.reason || ''}">${t.reason || '-'}</td>
                <td class="p-3 text-teal-600 font-bold text-xs">${t.orderId || '-'}</td>
            </tr>
        `;
    }).join('');
},

filterInventory: () => {
    const searchTerm = document.getElementById('inv-search')?.value.toLowerCase();
    const categoryFilter = document.getElementById('inv-category-filter')?.value;
    const statusFilter = document.getElementById('inv-status-filter')?.value;

    if (currentInventoryTab === 'master') {
        const filteredItems = currentInventory.filter(item => {
            const matchesSearch = !searchTerm ||
                item.name.toLowerCase().includes(searchTerm) ||
                (item.location && item.location.toLowerCase().includes(searchTerm)) ||
                (item.lastReason && item.lastReason.toLowerCase().includes(searchTerm));

            const matchesCategory = categoryFilter === 'all' || item.category ===
categoryFilter;

            let matchesStatus = true;
            if (statusFilter !== 'all') {
                const isLow = item.currentStock <= item.minimumLevel;
                if (statusFilter === 'In Stock') matchesStatus = item.currentStock > 0 &&
```

```
!isLow;
                else if (statusFilter === 'Low Stock') matchesStatus = isLow &&
item.currentStock > 0;
                else if (statusFilter === 'Out of Stock') matchesStatus = item.currentStock
=== 0;
            }

            return matchesSearch && matchesCategory && matchesStatus;
        });
        window.adminApp.renderInventoryList(filteredItems);
    } else {
        const filteredTrans = currentTransactions.filter(t => {
            const matchesSearch = !searchTerm ||
                t.itemName.toLowerCase().includes(searchTerm) ||
                (t.reason && t.reason.toLowerCase().includes(searchTerm)) ||
                (t.orderId && t.orderId.toLowerCase().includes(searchTerm));

            const matchesCategory = categoryFilter === 'all' || t.category === categoryFilter;

            return matchesSearch && matchesCategory;
        });
        window.adminApp.renderInventoryTransactions(filteredTrans);
    }
},

openPhotoViewer: (url, name) => {
    const img = document.getElementById('inventory-viewer-img');
    const title = document.getElementById('inventory-viewer-title');
    if (img) img.src = url;
    if (title) title.textContent = name || 'Item Detail';
    window.adminApp.openModal('inventory-image-viewer');
},

// === CONTRACT REVIEW ===
toggleContractReview: () => {
    const section = document.getElementById('contract-review-section');
    if (section) section.classList.toggle('open');
},

loadContractReview: async (projectId) => {
    const project = currentProjects.find(p => p.id === projectId);
    if (!project) return;

    // Store current project ID for save
    const section = document.getElementById('contract-review-section');
    if (section) {
        section.dataset.projectId = projectId;
        section.classList.remove('open'); // Collapse on load
    }

    // Reset status
    const status = document.getElementById('cr-save-status');
```

```
    if (status) status.textContent = 'Loading...';

    // Load saved data from Firestore
    const result = await DB.getContractReview(projectId);
    let reviewData = result.data || {};

    // Auto-fill defaults if not present
    if (!reviewData.reviewNo) reviewData.reviewNo = `CR-${project.projectId || projectId}`;
    if (!reviewData.deliveryDate && project.expectedCompletion) reviewData.deliveryDate =
project.expectedCompletion;
    if (!reviewData.internalDate) reviewData.internalDate = new
Date().toISOString().split('T')[0];

    // Populate Header fields
    const setVal = (id, val) => {
        const el = document.getElementById(id);
        if (el && val) el.value = val;
    };

    setVal('cr-review-no', reviewData.reviewNo);
    setVal('cr-internal-date', reviewData.internalDate);
    setVal('cr-po-no', reviewData.poNo);
    setVal('cr-delivery-date', reviewData.deliveryDate);
    setVal('cr-important-instructions', reviewData.importantInstructions);

    // Map old checklist format to new dynamic format if needed
    let mappedChecklist = reviewData.checklistDynamic || {};

    if (reviewData.checklist && Object.keys(mappedChecklist).length === 0) {
        // Migration/fallback from old structure - optional based on how data was saved
previously
        // Assuming legacy items might need manual re-mapping if important, else start fresh
    }

    // Render the dynamic checklist grid with current data mapping
    window.adminApp.renderContractReview(mappedChecklist);

    if (status) {
        status.textContent = result.data ? '✓ Loaded from saved review' : '';
        setTimeout(() => { if (status) status.textContent = ''; }, 3000);
    }
},



printContractReview: () => {
    const section = document.getElementById('contract-review-section');
    if (section) section.classList.add('open');
    // Populate print header date
    const printDate = document.getElementById('cr-print-date');
    if (printDate) {
        printDate.textContent = new Date().toLocaleDateString('en-IN', { day: '2-digit',
```

```
month: 'short', year: 'numeric' });
    }
    setTimeout(() => window.print(), 200);
},

renderProjectCards: (projects) => {
    const grid = document.getElementById('project-grid');
    const listView = document.getElementById('project-list-view');
    if (!grid || !listView) return;

    const mode = window.adminApp.projectViewMode || 'grid';

    if (mode === 'list') {
        grid.classList.add('hidden');
        listView.classList.remove('hidden');
        window.adminApp.renderProjectTable(projects);
        return;
    } else {
        grid.classList.remove('hidden');
        listView.classList.add('hidden');
    }

    if (projects.length === 0) {
        const msg = isProjectTrashView ? 'Trash is empty.' : 'No projects found matching your
filters.';
        const sub = isProjectTrashView ? 'Deleted projects will appear here.' : 'Try adjusting
your search or create a new project.';
        grid.innerHTML = `
            <div class="pm-empty-state">
                <div class="pm-empty-icon">
                    <svg fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-
linecap="round" stroke-linejoin="round" stroke-width="1.5" d="M19 11H5m14 0a2 2 0 012 2v6a2 2
0 01-2 2H5a2 2 0 01-2-2v-6a2 2 0 012-2m14 0V9a2 2 0 00-2-2M5 11V9a2 2 0 012-2m0 0V5a2 2 0 012-
2h6a2 2 0 012 2v2M7 7h10" /></svg>
                </div>
                <div class="pm-empty-title">${msg}</div>
                <div class="pm-empty-text">${sub}</div>
            </div>
        `;
        return;
    }

    grid.innerHTML = projects.map(p => {
        const rawStatus = p.status || 'Draft';
        const statusClass = rawStatus.toLowerCase().replace(/\s+/g, '-');
        const statusSlug = `st-${statusClass}`;
        const isTrashed = !!p.isDeleted;
        const startDate = p.createdAt ? new Date(p.createdAt.seconds *
1000).toLocaleDateString('en-IN', { day: '2-digit', month: 'short' }) : 'N/A';
        const deliveryDate = p.expectedCompletion ? new
Date(p.expectedCompletion).toLocaleDateString('en-IN', { day: '2-digit', month: 'short' }) :
'N/A';
```

```
        const isContractFiled = p.contractFiled || ['Approved', 'In Progress',
'Completed'].includes(p.status);
        const contractStatusHtml = isContractFiled ?
            '<span class="pm-c-contract-val"><svg style="width:13px;height:13px;" fill="none"
stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2.5" d="M5 13l4 4L19 7" /></svg> Filed</span>' :
            `<span class="pm-c-contract-val pending">${rawStatus}</span>`;

        // NEW: Fetch Drawing Number from Internal Order if missing on project
        const allOrders = window.adminApp.getCurrentOrders ?
window.adminApp.getCurrentOrders() : [];
        const matchingOrder = allOrders.find(o => o.internalOrderNo === p.projectId);
        const displayDrg = p.drawingNo || (matchingOrder ? matchingOrder.drawingNo : '');

        const actionButtons = isTrashed ? `
            <button class="pm-c-icon-btn" onclick="event.stopPropagation();
window.adminApp.restoreProject('${p.id}')" title="Restore">
                <svg style="width:16px;height:16px;" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M3 10h10a8 8 0 018 8v2M3 10l6 6m-6-6l6-6" /></svg>
            </button>
            <button class="pm-c-icon-btn trash" onclick="event.stopPropagation();
window.adminApp.permanentDeleteProject('${p.id}')" title="Delete Permanently">
                <svg style="width:16px;height:16px;" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M6 18L18 6M6 6l12 12" /></svg>
            </button>
        ` : `
            <button class="pm-c-icon-btn" onclick="event.stopPropagation();
window.adminApp.editProject('${p.id}')" title="Edit">
                <svg style="width:16px;height:16px;" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M11 5H6a2 2 0 00-2 2v11a2 2 0 002 2h11a2 2 0 002-2v-5m-1.414-9.414a2 2 0 112.828
2.828L11.828 15H9v-2.828l8.586-8.586z" /></svg>
            </button>
            <button class="pm-c-icon-btn trash" onclick="event.stopPropagation();
window.adminApp.trashProject('${p.id}')" title="Trash">
                <svg style="width:16px;height:16px;" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1
0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" /></svg>
            </button>
        `;

        return `
            <div class="pm-card-compact ${statusSlug} ${isTrashed ? 'trashed' : ''}">
                <div class="pm-c-accent ${statusClass}"></div>
                <div class="pm-c-body">
                    <div class="pm-c-header">
                        <div class="pm-c-title-group">
                            <span class="pm-c-title">${p.name}</span>
```

```
                            <span class="pm-c-subtitle">${p.customerName || 'N/A'}</span>
                        </div>
                        <div class="pm-c-id-group">
                            <span class="pm-c-id-pill">${p.projectId}</span>
                            ${displayDrg ? `<span class="pm-c-drg">DRG: ${displayDrg}</span>`
: ''}
                        </div>
                    </div>
                    <div class="pm-c-status-row">
                        <span class="pm-c-label">Status</span>
                        <span class="status-badge ${statusClass}">${p.status || 'Draft'}
</span>
                        ${p.jobType ? `<span class="pm-c-type-badge">${p.jobType}</span>` :
''}
                    </div>
                </div>
                <div class="pm-c-dates">
                    <div class="pm-c-date-cell">
                        <span class="pm-c-label">Start Date</span>
                        <span class="pm-c-value">${startDate}</span>
                    </div>
                    <div class="pm-c-date-cell">
                        <span class="pm-c-label">Delivery</span>
                        <span class="pm-c-value">${deliveryDate}</span>
                    </div>
                </div>
                <div class="pm-c-footer">
                    <div class="pm-c-contract-status">
                        <span class="pm-c-label">Contract</span>
                        ${contractStatusHtml}
                    </div>
                    <div class="pm-c-actions">
                        ${actionButtons}
                        <button class="pm-c-primary-btn" onclick="event.stopPropagation();
window.adminApp.viewProjectDetails('${p.id}')">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24"><path
stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M9 5l7 7-7 7" /></svg>
                            View Report
                        </button>
                    </div>
                </div>
            </div>
        `;
    }).join('');
},

getStatusColorClass: (status) => {
    switch (status) {
        case 'Draft': return 'border-slate-300';
        case 'Under Review': return 'border-amber-400';
        case 'Approved': return 'border-blue-500';
        case 'In Progress': return 'border-teal-500';
```

```
        case 'Completed': return 'border-emerald-600';
        default: return 'border-slate-200';
    }
},

viewProjectDetails: async (id) => {
    window.adminApp.currentEditingProjectId = id;
    const project = currentProjects.find(p => p.id === id);
    if (!project) return;

    window.adminApp.switchView('project_detail');
    window.adminApp.switchDeepDiveTab('review');

    // Populate Basic Info
    const idDisplay = document.getElementById('detail-project-id');
    const nameDisplay = document.getElementById('detail-project-name');
    const statusDisplay = document.getElementById('detail-project-status');

    if (idDisplay) idDisplay.textContent = project.projectId;
    if (nameDisplay) nameDisplay.textContent = project.name;
    if (statusDisplay) {
        const span = statusDisplay.querySelector('span');
        if (span) span.textContent = project.status || 'Draft';
        const statusCls = (project.status || 'Draft').toLowerCase().replace(/\s+/g, '-');
        statusDisplay.className = `status-badge ${statusCls} flex items-center gap-1 group`;
    }

    // Side Info
    const infoCustomer = document.getElementById('info-customer');
    const infoJobType = document.getElementById('info-job-type');
    const infoDrgSource = document.getElementById('info-drg-source');
    const infoExpectedDate = document.getElementById('info-expected-date');
    const infoNotes = document.getElementById('info-notes');

    if (infoCustomer) infoCustomer.textContent = project.customerName || '-';
    if (infoJobType) infoJobType.textContent = project.jobType || '-';
    if (infoDrgSource) infoDrgSource.textContent = project.drawingSource || '-';
    if (infoExpectedDate) infoExpectedDate.textContent = project.expectedCompletion || 'Not
Set';
    if (infoNotes) infoNotes.textContent = project.internalNotes || 'No internal notes.';



    // Subscribe to Project Sub-collections (Files & Logs)
    DB.subscribeToProjectFiles(id, (files) => {
        window.adminApp.renderProjectFiles(files);
    });

    DB.subscribeToProjectAuditLogs(id, (logs) => {
        window.adminApp.renderProjectAuditLogs(logs);
    });
```

```
    // Load Contract Review form for this project
    window.adminApp.loadContractReview(id);
},

renderProgressDots: (project) => {
    const stages = ['Intake', 'Planning', 'Design', 'Production', 'Quality', 'Delivery',
'Closure'];
    const currentIdx = stages.indexOf(project.currentStage);
    const container = document.getElementById('dd-progress-dots');
    if (!container) return;

    let html = '';
    stages.forEach((stage, idx) => {
        if (idx > 0) {
            html += `<span class="dd-dot-line ${idx <= currentIdx ? 'completed' : ''}">
</span>`;
        }
        let cls = 'dd-dot';
        if (idx === currentIdx) cls += ' active';
        else if (idx < currentIdx) cls += ' completed';
        html += `<span class="${cls}" title="${stage}"></span>`;
    });
    container.innerHTML = html;
},

switchDeepDiveTab: (tabName) => {
    // Update tab buttons
    document.querySelectorAll('.dd-tab').forEach(t => {
        t.classList.toggle('active', t.dataset.tab === tabName);
    });
    // Update panels
    document.querySelectorAll('.dd-panel').forEach(p => {
        p.classList.toggle('active', p.id === `dd-panel-${tabName}`);
    });
},

renderStageAction: (project) => {
    const stages = {
        'Intake': {
            description: 'Review initial metadata and drawing source requirements.',
            buttons: [
                { label: 'Submit for Planning', class: 'btn-primary', action:
`window.adminApp.transitionStage('${project.id}', 'Planning')` }
            ]
        },
        'Planning': {
            description: 'Define timelines, resources, and production strategy.',
            buttons: [
                { label: 'Request Design Approval', class: 'btn-primary', action:
`window.adminApp.submitApprovalRequest('${project.id}', 'Planning')` }
            ]
        },
```

```
        'Design': {
            description: 'Complete engineering designs and release drawings for production.',
            buttons: [
                { label: 'Submit for Production', class: 'btn-primary', action:
`window.adminApp.transitionStage('${project.id}', 'Production')` }
            ]
        },
        'Production': {
            description: 'Manufacturing phase. Track shop floor progress and machine
utilization.',
            buttons: [
                { label: 'Mark Production Complete', class: 'btn-primary', action:
`window.adminApp.transitionStage('${project.id}', 'Quality')` }
            ]
        },
        'Quality': {
            description: 'Post-production inspection and regulatory compliance checks.',
            buttons: [
                { label: 'Pass Quality Check', class: 'btn-primary', action:
`window.adminApp.transitionStage('${project.id}', 'Delivery')` }
            ]
        },
        'Delivery': {
            description: 'Packaging, logistics, and customer handover.',
            buttons: [
                { label: 'Mark as Delivered', class: 'btn-primary', action:
`window.adminApp.transitionStage('${project.id}', 'Closure')` }
            ]
        },
        'Closure': {
            description: 'Final audit, documentation filing, and project completion.',
            buttons: [
                { label: 'Close Project', class: 'btn-primary', action:
`window.adminApp.completeProject('${project.id}')` }
            ]
        }
    };

    const config = stages[project.currentStage] || stages['Intake'];

    const stagePill = document.getElementById('dd-stage-pill');
    const stageDesc = document.getElementById('dd-stage-desc');
    const stageActions = document.getElementById('dd-stage-actions');

    if (stagePill) stagePill.textContent = project.currentStage;
    if (stageDesc) stageDesc.textContent = config.description;
    if (stageActions) {
        stageActions.innerHTML = config.buttons.map(btn => `
            <button class="btn ${btn.class} btn-sm" onclick="${btn.action}">${btn.label}
</button>
        `).join('');
    }
```

```
},

transitionStage: async (projectId, nextStage) => {
    const result = await DB.updateProject(projectId, { currentStage: nextStage },
`Transitioned to ${nextStage}`);
    if (result.error) {
        alert('Error transitioning stage: ' + result.error);
    } else {
        // Re-render handled by real-time subscription
    }
},

renderProjectFiles: (files) => {
    const body = document.getElementById('project-files-body');
    if (!body) return;

    if (files.length === 0) {
        body.innerHTML = '<tr><td colspan="5" class="p-8 text-center text-slate-400 italic">No
files uploaded yet.</td></tr>';
        return;
    }

    body.innerHTML = files.map(f => `
        <tr class="border-b border-slate-50 hover:bg-slate-50 transition-colors">
            <td class="p-3 font-medium text-slate-700">${f.name}</td>
            <td class="p-3 text-slate-500">${f.category}</td>
            <td class="p-3 text-center"><span class="version-tag">v${f.version}</span></td>
            <td class="p-3 text-slate-500">${f.uploadedBy || 'System'}</td>
            <td class="p-3 text-right">
                <div class="flex justify-end gap-1">
                    <button class="btn btn-ghost btn-sm text-teal-600"
onclick="window.open('${f.url}')">View</button>
                    <button class="btn btn-ghost btn-sm text-rose-500"
onclick="window.adminApp.confirmDeleteFile('${f.id}', '${f.url}')">Delete</button>
                </div>
            </td>
        </tr>
    `).join('');
},

renderProjectAuditLogs: (logs) => {
    const container = document.getElementById('audit-log-container');
    if (!container) return;

    if (logs.length === 0) {
        container.innerHTML = '<div class="text-center py-4 text-slate-400 italic">No audit
history found.</div>';
        return;
    }

    container.innerHTML = logs.map(l => {
        const date = l.timestamp?.toDate ? l.timestamp.toDate() : new Date();
```

```
        const timeStr = date.toLocaleDateString() + ' • ' + date.toLocaleTimeString([], {
hour: '2-digit', minute: '2-digit' });

        return `
            <div class="dd-timeline-item">
                <div class="dd-timeline-title">${l.action}</div>
                <div class="dd-timeline-desc">${l.details}</div>
                <div class="dd-timeline-time">${timeStr} by ${l.user || 'System'}</div>
            </div>
        `;
    }).join('');
},

moveProject: async (projectId, newStatus) => {
    const result = await DB.updateProjectStatus(projectId, newStatus);
    if (result.error) {
        alert('Error moving project: ' + result.error);
    } else {
        const isMilestone = ['Approved', 'In Progress', 'Completed'].includes(newStatus);
        const detail = `Status moved to ${newStatus}${isMilestone ? ' (Contract Auto-filed)' :
''}`;
        await DB.addAuditLog(projectId, 'Status Updated', detail);
    }
},

deleteProject: (projectId) => {
    window.adminApp.trashProject(projectId);
},

openAddOrderModal: () => {
    window.adminApp.openModal('add-order-modal');
},

toggleStatusMenu: (e) => {
    e.stopPropagation();
    const menu = document.getElementById('dd-status-menu');
    if (menu) menu.classList.toggle('hidden');
},

updateProjectStatus: async (newStatus) => {
    const id = window.adminApp.currentEditingProjectId;
    if (!id) return;

    const result = await DB.updateProjectStatus(id, newStatus);
    if (result.error) {
        alert('Error updating status: ' + result.error);
    } else {
        // UI update for badge
        const statusDisplay = document.getElementById('detail-project-status');
        if (statusDisplay) {
            const span = statusDisplay.querySelector('span');
            if (span) span.textContent = newStatus;
```

```
            statusDisplay.className = `status-badge ${newStatus.toLowerCase().replace(/\s+/g,
'-')} flex items-center gap-1 group`;
        }
        const menu = document.getElementById('dd-status-menu');
        if (menu) menu.classList.add('hidden');

        // Audit Log
        const isMilestone = ['Approved', 'In Progress', 'Completed'].includes(newStatus);
        const detail = `Status changed to ${newStatus}${isMilestone ? ' (Contract Auto-filed)'
: ''}`;
        await DB.addAuditLog(id, 'Status Updated', detail);
    }
},

prepareAddOrder: () => {
    const form = document.getElementById('add-order-form');
    if (form) form.reset();
    const hidden = document.getElementById('orderId-input');
    if (hidden) hidden.value = '';

    // Set default date to today
    const dateInput = form.querySelector('[name="date"]');
    if (dateInput) {
        dateInput.value = new Date().toISOString().split('T')[0];
    }

    // Auto-generate next Order ID (YYYYYY-NNN)
    const now = new Date();
    const yr = now.getFullYear();
    const mo = now.getMonth(); // 0-indexed
    // Financial year: April (month 3) to March
    const fyStart = mo >= 3 ? yr : yr - 1;
    const fyEnd = fyStart + 1;
    // e.g. 2025-2026 → "202526-"
    const prefix = `${fyStart}${String(fyEnd).slice(-2)}-`;
    let maxNum = 0;
    currentOrders.forEach(o => {
        const io = o.internalOrderNo || '';
        if (io.startsWith(prefix)) {
            const num = parseInt(io.replace(prefix, ''), 10);
            if (!isNaN(num) && num > maxNum) maxNum = num;
        }
    });
    const nextNum = String(maxNum + 1).padStart(3, '0');
    const orderNoInput = form.querySelector('[name="internalOrderNo"]');
    if (orderNoInput) orderNoInput.value = `${prefix}${nextNum}`;

    // Setup auto-calculation for Total
    const qtyInput = document.getElementById('order-qty');
    const valueInput = document.getElementById('order-value');
    const totalInput = document.getElementById('order-total');
```

```
    const calculateTotal = () => {
        const qty = parseFloat(qtyInput?.value) || 0;
        const value = parseFloat(valueInput?.value) || 0;
        const total = qty * value;
        if (totalInput) {
            totalInput.value = total > 0 ? total.toLocaleString('en-IN', {
minimumFractionDigits: 2, maximumFractionDigits: 2 }) : '';
        }
    };

    if (qtyInput) qtyInput.addEventListener('input', calculateTotal);
    if (valueInput) valueInput.addEventListener('input', calculateTotal);

    window.adminApp.openAddOrderModal();
},

// New: Delivery Modal Logic
openAddDeliveryModal: (existingOrder = null) => {
    const form = document.getElementById('add-delivery-form');
    if (form) form.reset();

    const hidden = document.getElementById('delivery-orderId-input');
    if (hidden) hidden.value = existingOrder ? existingOrder.id : '';

    // Clear/Populate the IO lookup field
    const ioLookup = document.getElementById('delivery-io-lookup');
    if (ioLookup) {
        ioLookup.value = existingOrder ? (existingOrder.internalOrderNo || '') : '';
    }

    // Populate datalist with all active/delivered IO numbers for autocomplete
    const datalist = document.getElementById('delivery-io-suggestions');
    if (datalist) {
        const activeOrders = currentOrders.filter(o =>
            (o.status === 'Pending' || o.status === 'Portion Delivered' || o.status ===
'Delivered' || (existingOrder && o.internalOrderNo === existingOrder.internalOrderNo)) &&
            o.internalOrderNo &&
            (!o.entryType || o.entryType !== 'delivery_report')
        );
        datalist.innerHTML = activeOrders.map(o =>
            `<option value="${o.internalOrderNo}">${o.customer || ''} - ${o.description || ''}
(${o.status})</option>`
        ).join('');
    }

    if (existingOrder) {
        // Populate fields directly if editing
        const setVal = (name, val) => {
            const el = form.querySelector(`[name="${name}"]`);
            if (el) el.value = val !== undefined && val !== null ? val : '';
        };
        setVal('date', existingOrder.deliveryDateActual || existingOrder.date);
```

```
        setVal('customer', existingOrder.customer);
        setVal('description', existingOrder.description);
        setVal('drawingNo', existingOrder.drawingNo);
        setVal('department', existingOrder.department);
        setVal('dcNo', existingOrder.dcNo);
        setVal('qty', existingOrder.deliveryQty || existingOrder.qty);
        setVal('total', existingOrder.total || '');
        setVal('labourCost', existingOrder.labourCost || 0);
        setVal('billNo', existingOrder.billNo || '');
        setVal('manpower', existingOrder.manpower || '');
        setVal('qtyUnit', existingOrder.qtyUnit || 'Nos');
    } else {
        const dateInput = form.querySelector('[name="date"]');
        if (dateInput) dateInput.value = new Date().toISOString().split('T')[0];
    }

    window.adminApp.openModal('add-delivery-modal');
},

// Lookup: Auto-fill delivery form from an existing Internal Order
lookupOrderForDelivery: (ioNo) => {
    if (!ioNo || !ioNo.trim()) return;
    const order = currentOrders.find(o => o.internalOrderNo === ioNo.trim());
    if (!order) return;

    const form = document.getElementById('add-delivery-form');
    if (!form) return;

    // Auto-fill fields (all remain editable)
    const setVal = (name, val) => {
        const el = form.querySelector(`[name="${name}"]`);
        if (el && val) el.value = val;
    };

    setVal('customer', order.customer);
    setVal('description', order.description);
    setVal('drawingNo', order.drawingNo);
    setVal('qty', order.qty);
    setVal('qtyUnit', order.qtyUnit);
    setVal('department', order.department);
},

submitDeliveryForm: async () => {
    const form = document.getElementById('add-delivery-form');
    if (!form.checkValidity()) {
        form.reportValidity();
        return;
    }

    const formData = new FormData(form);
    const orderId = formData.get('orderId');
```

```
    // Construct object compatible with DB.addOrder
    const orderData = {
        date: formData.get('date'),
        customer: formData.get('customer'),
        drawingNo: formData.get('drawingNo') || '',
        description: formData.get('description') || '',
        internalOrderNo: formData.get('internalOrderNo')?.trim() || '',
        qty: parseFloat(formData.get('qty')) || 0,
        total: parseFloat(formData.get('total')) || 0, // Manual Delivery Value
        department: formData.get('department') || '',
        labourCost: parseFloat(formData.get('labourCost')) || 0,
        manpower: parseFloat(formData.get('manpower')) || 0,

        // Delivery Specifics
        deliveryDateActual: formData.get('date'), // Same as entry date for direct delivery
        deliveryQty: parseFloat(formData.get('qty')) || 0,
        dcNo: formData.get('dcNo') || '',
        billNo: formData.get('billNo') || '',
        status: 'Delivered', // Force status
        entryType: 'delivery_report', // Flag to distinguish from Internal Orders

        // Defaults for unused fields
        saleValueEa: 0,
        prodValueEa: 0,
        priority: 'Medium',
        drgAvail: 'n',
        rawAvail: 'n',
        finishAvail: 'n'
    };

    let result;
    if (orderId) {
        result = await DB.updateOrder(orderId, orderData);
    } else {
        result = await DB.addOrder(orderData);
    }

    // Automated Sync: Update original Internal Order if IO Number is linked
    if (!result.error && orderData.internalOrderNo) {
        const ioNo = orderData.internalOrderNo;
        const originalOrder = currentOrders.find(o =>
            o.internalOrderNo === ioNo && (!o.entryType || o.entryType !== 'delivery_report')
        );

        if (originalOrder) {
            // 1. Calculate Total Delivered Quantity across all delivery report entries
            const allDeliveries = currentOrders.filter(o =>
                o.entryType === 'delivery_report' &&
                o.internalOrderNo === ioNo &&
                o.id !== orderId // Exclude old version if editing
            );
```

```
            const totalDelivered = allDeliveries.reduce((sum, o) => sum +
(parseFloat(o.deliveryQty) || 0), 0) + orderData.deliveryQty;

            // 2. Pool DC Numbers
            let allDCs = allDeliveries.map(o => o.dcNo?.trim()).filter(dc => dc);
            if (orderData.dcNo?.trim() && !allDCs.includes(orderData.dcNo.trim())) {
                allDCs.push(orderData.dcNo.trim());
            }
            const pooledDCs = allDCs.join(', ');

            // 3. Update Status based on completion
            const orderedQty = parseFloat(originalOrder.qty) || 0;
            let newStatus = originalOrder.status;

            if (totalDelivered >= orderedQty && orderedQty > 0) {
                newStatus = 'Delivered';
            } else if (totalDelivered > 0) {
                newStatus = 'Portion Delivered';
            } else {
                newStatus = 'Pending';
            }

            console.log(`Automated Sync: Updating IO ${ioNo} ->
${totalDelivered}/${orderedQty} (${newStatus})`);

            await DB.updateOrder(originalOrder.id, {
                deliveryQty: totalDelivered,
                dcNo: pooledDCs,
                status: newStatus
            });
        }
    }

    if (result.error) {
        alert('Error: ' + result.error);
    } else {
        window.adminApp.closeModal('add-delivery-modal');
    }
},

submitApprovalRequest: async (projectId, stage) => {
    const result = await DB.submitApproval(projectId, stage, {
        status: 'Approved', // For now, auto-approving or just recording
        approver: 'Admin',
        notes: `Auto-approved for ${stage} gate.`
    });

    if (result.error) {
        alert('Approval error: ' + result.error);
    } else {
        // Transition to next stage automatically if approved
        const stages = ['Intake', 'Planning', 'Design', 'Production', 'Quality', 'Delivery',
```

```
'Closure'];
        const currentIdx = stages.indexOf(stage);
        if (currentIdx < stages.length - 1) {
            window.adminApp.transitionStage(projectId, stages[currentIdx + 1]);
        }
    }
},

completeProject: async (projectId) => {
    const result = await DB.updateProject(projectId, {
        status: 'Completed',
        progress: 100
    }, 'Project marked as Completed.');

    if (result.error) {
        alert('Error completing project: ' + result.error);
    }
},

lockProject: async (projectId) => {
    const project = currentProjects.find(p => p.id === projectId);
    const newLockState = !project.isLocked;

    const result = await DB.updateProject(projectId, { isLocked: newLockState }, newLockState
? 'Project Locked' : 'Project Unlocked');
    if (result.error) {
        alert('Error locking/unlocking project: ' + result.error);
    }
},

openProjectSettings: () => {
    alert('Project Settings coming soon!');
},

openUploadModal: () => {
    const fileInput = document.getElementById('project-file-input');
    if (fileInput) fileInput.click();
},

handleFileSelection: async (event) => {
    const file = event.target.files[0];
    if (!file) return;

    // Validation: 500KB limit
    const maxSize = 500 * 1024; // 500KB
    if (file.size > maxSize) {
        alert(`File is too large (${(file.size / 1024).toFixed(1)}KB). Max size is 500KB.`);
        event.target.value = '';
        return;
    }

    const projectIdText = document.getElementById('detail-project-id')?.textContent;
```

```
    // Search in currentProjects for the project with that ID
    const activeProject = currentProjects.find(p => p.projectId === projectIdText);

    if (!activeProject) {
        alert("Error: Active project context not found.");
        return;
    }

    const uploadBtn = event.target.parentElement.querySelector('button');
    const originalHtml = uploadBtn ? uploadBtn.innerHTML : 'Upload';

    try {
        if (uploadBtn) {
            uploadBtn.disabled = true;
            uploadBtn.innerHTML = "Uploading...";
        }

        const result = await DB.uploadProjectFile(activeProject.id, file, 'Admin');

        if (result.error) {
            alert("Upload failed: " + result.error);
        }
    } catch (error) {
        console.error("Upload Error:", error);
        alert("Upload failed: " + error.message);
    } finally {
        if (uploadBtn) {
            uploadBtn.disabled = false;
            uploadBtn.innerHTML = originalHtml;
        }
        event.target.value = '';
    }
},

confirmDeleteFile: async (fileId, fileUrl) => {
    if (!confirm("Are you sure you want to permanently delete this file? This cannot be
undone.")) return;

    const projectIdText = document.getElementById('detail-project-id')?.textContent;
    const activeProject = currentProjects.find(p => p.projectId === projectIdText);

    if (!activeProject) {
        alert("Error: Active project context not found.");
        return;
    }

    try {
        const result = await DB.deleteProjectFile(activeProject.id, fileId, fileUrl);
        if (result.success) {
            // Success message or toast could go here
            console.log("File deleted successfully");
        } else {
```

```
            alert("Deletion failed: " + result.error);
        }
    } catch (error) {
        console.error("Delete Error:", error);
        alert("Deletion failed: " + error.message);
    }
},

// Modal Helpers
openModal: (id) => {
    const modal = document.getElementById(id);
    if (modal) {
        modal.classList.remove('hidden');
        // Small delay to allow display:block to apply before adding active class for
transition
        setTimeout(() => modal.classList.add('active'), 10);
    }
},

closeModal: (id) => {
    const modal = document.getElementById(id);
    if (modal) {
        modal.classList.remove('active');
        // Wait for transition to finish before hiding
        setTimeout(() => modal.classList.add('hidden'), 300);
    }
},

toggleSidebarGroup: (header) => {
    const group = header.parentElement;
    if (group) {
        group.classList.toggle('collapsed');
    }
},

viewMemberWorkload: (memberId) => {
    const member = currentMembers.find(m => m.id === memberId);
    if (!member) return;

    // Filter orders assigned to this member (include all for summary stats)
    const memberTasks = currentOrders.filter(o =>
        o.assignedTo && Array.isArray(o.assignedTo) && o.assignedTo.includes(memberId) &&
        !o.isTrash && !o.deleted
    );

    UI.renderMemberWorkload(member, memberTasks);
    window.adminApp.openModal('member-workload-modal');
},

printMemberWorkload: () => {
    window.print();
},
```

```
getCurrentMembers: () => currentMembers,
getCurrentOrders: () => currentOrders,

// Custom Confirm Modal
showConfirmModal: (title, message, onConfirm) => {
    const modal = document.getElementById('confirm-modal');
    const titleEl = document.getElementById('confirm-title');
    const msgEl = document.getElementById('confirm-message');
    const yesBtn = document.getElementById('confirm-yes-btn');

    if (!modal || !titleEl || !msgEl || !yesBtn) {
        if (confirm(message)) onConfirm();
        return;
    }

    titleEl.textContent = title;
    msgEl.textContent = message;

    // Clone button to remove old event listeners
    const newBtn = yesBtn.cloneNode(true);
    yesBtn.parentNode.replaceChild(newBtn, yesBtn);

    newBtn.onclick = () => {
        window.adminApp.closeModal('confirm-modal');
        onConfirm();
    };

    window.adminApp.openModal('confirm-modal');
},

editOrder: (id) => {
    const order = currentOrders.find(o => o.id === id);
    if (!order) return;

    if (order.entryType === 'delivery_report') {
        // Use specialized delivery modal population
        window.adminApp.openAddDeliveryModal(order);
    } else {
        // Use Standard Modal
        Monitoring.populateForm(order);
    }
},

toggleDeliveryTrash: () => {
    const btn = document.getElementById('delivery-trash-btn');
    if (!btn) return;

    // Toggle global trash state (reusing existing state for simplicity or adding specific
one)
    // For now, let's use a specific flag for delivery report trash if needed,
    // OR reuse isTrashView if we want a global toggle.
```

```
    // User requested "Trash box" in Delivery Report specifically.

    // Let's implement a specific toggle for Delivery Report visualization
    if (btn.classList.contains('btn-danger')) {
        // Turn OFF Trash
        btn.innerHTML = '<svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1
0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16"></path></svg> Trash';
        btn.classList.remove('btn-danger');
        btn.classList.add('btn-secondary');

        // Hide Empty Trash Btn
        const emptyBtn = document.getElementById('delivery-empty-trash-btn');
        if (emptyBtn) emptyBtn.classList.add('hidden');

        Monitoring.setDeliveryTrashMode(false);
    } else {
        // Turn ON Trash
        btn.innerHTML = '<svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M10 19l-7-7m0 0l7-7m-7 7h18"></path></svg> Back to Report';
        btn.classList.remove('btn-secondary');
        btn.classList.add('btn-danger');

        // Show Empty Trash Btn
        const emptyBtn = document.getElementById('delivery-empty-trash-btn');
        if (emptyBtn) emptyBtn.classList.remove('hidden');

        Monitoring.setDeliveryTrashMode(true);
    }

    // Re-render
    const weekPicker = document.getElementById('delivery-week-picker');
    if (weekPicker && weekPicker.value) {
        Monitoring.renderDeliveryReport(weekPicker.value);
    }
},

emptyTrash: (type) => {
    window.adminApp.showConfirmModal(
        "Empty Trash?",
        `Are you sure you want to permanently delete ALL ${type === 'delivery' ? 'delivery
report' : 'internal order'} items in the trash? This cannot be undone.`,
        async () => {
            try {
                const result = await DB.emptyTrash(type);
                if (result.error) {
                    alert("Error emptying trash: " + result.error);
                } else {
                    // Refresh views
                    if (type === 'delivery') {
```

```
                        const weekPicker = document.getElementById('delivery-week-picker');
                        if (weekPicker && weekPicker.value) {
                            Monitoring.renderDeliveryReport(weekPicker.value);
                        }
                    }
                }
            } catch (e) {
                console.error("Empty trash failed:", e);
                alert("Failed to empty trash.");
            }
        }
    );
},

softDeleteOrder: (id) => {
    window.adminApp.showConfirmModal(
        "Move to Trash?",
        "Are you sure you want to move this order to trash?",
        async () => {
            try {
                console.log("Attempting soft delete for:", id);
                const result = await DB.softDeleteOrder(id);
                if (result.error) {
                    alert("Error moving order to trash: " + result.error);
                } else {
                    console.log("Soft delete successful");
                }
            } catch (e) {
                console.error("Soft delete failed:", e);
                alert("Failed to move order to trash. Please try again.");
            }
        }
    );
},

restoreOrder: (id) => {
    DB.restoreOrder(id);
},

changeOrderStatus: async (id, newStatus) => {
    try {
        await DB.updateOrder(id, { status: newStatus });
        const select = document.querySelector(`select[data-order-id="${id}"]`);
        if (select) {
            select.className = 'status-select ' +
                (newStatus === 'Delivered' ? 'status-delivered' : 'status-pending');
        }
    } catch (e) {
        console.error('Failed to update status:', e);
        alert('Failed to update status');
    }
},
```

```
permanentDeleteOrder: (id) => {
    window.adminApp.showConfirmModal(
        "Delete Permanently?",
        "This action cannot be undone. The order will be permanently removed.",
        async () => {
            try {
                const result = await DB.permanentDeleteOrder(id);
                if (result.error) {
                    alert("Error deleting order: " + result.error);
                }
            } catch (e) {
                console.error("Permanent delete failed:", e);
                alert("Failed to delete order permanently. Please try again.");
            }
        }
    );
},

toggleTrashMode: () => {
    isTrashView = !isTrashView;
    Monitoring.setTrashMode(isTrashView);

    const btn = document.getElementById('trash-toggle-btn');
    if (btn) {
        if (isTrashView) {
            btn.innerHTML = '<svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M10 19l-7-7m0 0l7-7m-7 7h18"></path></svg> Back to Orders';
            btn.classList.remove('btn-secondary');
            btn.classList.add('btn-danger');
        } else {
            btn.innerHTML = '<svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1
0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16"></path></svg> Trash';
            btn.classList.add('btn-secondary');
            btn.classList.remove('btn-danger');
        }
    }

    // Re-subscribe with correct filter
    if (window.unsubscribeOrders) window.unsubscribeOrders();

    window.unsubscribeOrders = DB.subscribeToOrders((orders) => {
        currentOrders = orders;
        if (!document.getElementById('view-monitoring').classList.contains('hidden')) {
            Monitoring.renderTable(orders);
        }
    }, isTrashView);
},
```

```
switchTeamTab: (tab) => {
    const memberView = document.getElementById('subview-members');
    const hierView = document.getElementById('subview-hierarchy');
    const memberBtn = document.getElementById('tab-members');
    const hierBtn = document.getElementById('tab-hierarchy');

    if (tab === 'members') {
        if (memberView) memberView.classList.remove('hidden');
        if (hierView) hierView.classList.add('hidden');
        if (memberBtn) memberBtn.classList.add('active');
        if (hierBtn) hierBtn.classList.remove('active');
    } else {
        if (memberView) memberView.classList.add('hidden');
        if (hierView) hierView.classList.remove('hidden');
        if (memberBtn) memberBtn.classList.remove('active');
        if (hierBtn) hierBtn.classList.add('active');

        // Trigger render
        Charts.renderHierarchy(currentMembers || [], 'hierarchy-container');
    }
},

renderTeamView: () => {
    Charts.renderHierarchy(currentMembers || [], 'hierarchy-container');
},

renderMonitoring: () => {
    if (currentOrders && currentOrders.length > 0) {
        Monitoring.renderTable(currentOrders);
    }
},

// Workflow accessors
getCurrentMembers: () => currentMembers,
getCurrentOrders: () => currentOrders,

// Daily Roster
renderWorkflowView: () => Workflow.initWorkflowView(),
wfOpenAssignModal: () => Workflow.openAssignModal(),
wfConfirmAssign: () => Workflow.confirmAssign(),
wfUpdateRow: (idx, field, value) => Workflow.updateRow(idx, field, value),
wfEditRow: (idx) => Workflow.editRow(idx),
wfFilterTeam: (val) => Workflow.filterTeam(val),
wfRemoveRow: (idx) => Workflow.removeRow(idx),
wfSaveAll: () => Workflow.saveAll(),
wfCopyPreviousDay: () => Workflow.copyPreviousDay(),
wfPrint: () => Workflow.printWorksheet(),
wfCalculateProdValue: () => Workflow.calculateProdValue(),

setPageSize: (size) => {
    Monitoring.setPageSize(size);
},
```

```
sort: (key) => {
    Monitoring.sort(key);
},

exportToPDF: () => {
    Monitoring.exportToPDF(currentOrders);
},

// New: Delivery Report Helpers
getCurrentOrders: () => {
    return currentOrders;
},

printDeliveryReport: () => {
    const tableHtml = document.getElementById('delivery-report-table')?.outerHTML || '';
    const totalItems = document.getElementById('report-total-items')?.textContent || '0';
    const totalValue = document.getElementById('report-total-value')?.textContent || '₹0';
    const totalManpower = document.getElementById('report-total-manpower')?.textContent ||
'0';
    const dateRangeText = document.getElementById('delivery-week-range')?.textContent ||
'Delivery Report';

    const printWindow = window.open('', '_blank');
    const html = `
        <!DOCTYPE html>
        <html>
        <head>
            <title>Delivery Report Print</title>
            <style>
                body { font-family: Arial, sans-serif; padding: 20px; color: #000; }
                .header { text-align: center; margin-bottom: 20px; }
                .header h1 { margin: 0; font-size: 24px; color: #333; }
                .header p { margin: 5px 0; color: #666; font-weight: bold; text-transform:
uppercase;}
                .summary { display: flex; justify-content: space-around; margin-bottom: 20px;
padding: 15px; background: #f8f9fa; border: 1px solid #e2e8f0; border-radius: 8px;}
                .stat { text-align: center; }
                .stat-label { font-size: 12px; text-transform: uppercase; color: #64748b;
font-weight: bold;}
                .stat-value { font-size: 22px; font-weight: bold; margin-top: 5px; color:
#0f172a;}
                table { width: 100%; border-collapse: collapse; font-size: 11px; margin-top:
10px; table-layout: fixed; word-wrap: break-word;}
                th, td { border: 1px solid #cbd5e1; padding: 8px 6px; text-align: left;
vertical-align: middle;}
                th { background-color: #f1f5f9; font-weight: bold; color: #334155; text-
transform: uppercase; font-size: 10px;}
                .text-center { text-align: center; }

                /* Hide internal UI buttons and Action column */
                .no-print, th:last-child, td:last-child { display: none !important; }
```

```
                button, .btn { display: none !important; }

                @media print {
                    @page { size: A4 landscape; margin: 10mm; }
                    body { padding: 0; }
                    .summary { background: #f8f9fa !important; border-color: #cbd5e1
!important; -webkit-print-color-adjust: exact; print-color-adjust: exact; }
                    th { background-color: #f1f5f9 !important; -webkit-print-color-adjust:
exact; print-color-adjust: exact; }
                }
            </style>
        </head>
        <body>
            <div class="header">
                <h1>Delivery Report</h1>
                <p>${dateRangeText}</p>
            </div>
            <div class="summary">
                <div class="stat">
                    <div class="stat-label">Delivered Items</div>
                    <div class="stat-value">${totalItems}</div>
                </div>
                <div class="stat">
                    <div class="stat-label">Total Value</div>
                    <div class="stat-value">${totalValue}</div>
                </div>
                <div class="stat">
                    <div class="stat-label">Total Manpower</div>
                    <div class="stat-value">${totalManpower}</div>
                </div>
            </div>
            ${tableHtml}
            <script>
                window.onload = () => {
                    setTimeout(() => {
                        window.print();
                    }, 500);
                };
            </script>
        </body>
        </html>
    `;
    printWindow.document.write(html);
    printWindow.document.close();
},

saveManpower: (date, value) => {
    const numericVal = parseFloat(value.replace(/[^0-9.]/g, '')) || 0;
    DB.saveDailyStat(date, 'manpower', numericVal).then(res => {
        if (res.error) console.error(res.error);
        else console.log('Manpower saved for', date);
    });
```

```
},

// ============================================
// PENDING ASSIGNMENT FUNCTIONS
// ============================================

renderPendingAssignment: () => {
    const view = document.getElementById('view-pending_assignment');
    if (!view || view.classList.contains('hidden')) return;

    const tbody = document.getElementById('pending-assignment-body');
    const countBadge = document.getElementById('pending-assignment-count');
    if (!tbody) return;

    // Get filters
    const deptFilter = document.getElementById('pending-filter-department')?.value || '';
    const assignedFilter = document.getElementById('pending-filter-assigned')?.value || '';
    const priorityFilter = document.getElementById('pending-filter-priority')?.value || '';

    // Filter pending orders
    let pendingOrders = currentOrders.filter(o =>
        o.status === 'Pending' && !o.deleted
    );

    // Apply department filter
    if (deptFilter) {
        pendingOrders = pendingOrders.filter(o => o.department === deptFilter);
    }

    // Apply assigned filter
    if (assignedFilter === 'assigned') {
        pendingOrders = pendingOrders.filter(o => o.assignedTo && o.assignedTo.length > 0);
    } else if (assignedFilter === 'unassigned') {
        pendingOrders = pendingOrders.filter(o => !o.assignedTo || o.assignedTo.length === 0);
    }

    // Apply priority filter
    if (priorityFilter) {
        pendingOrders = pendingOrders.filter(o => (o.priority || 'normal') ===
priorityFilter);
    }

    // Get sort option
    const sortBy = document.getElementById('pending-sort-by')?.value || 'priority';

    // Sort based on selection
    pendingOrders.sort((a, b) => {
        if (sortBy === 'priority') {
            // Urgent first, then by due date
            const priorityA = a.priority === 'urgent' ? 0 : 1;
            const priorityB = b.priority === 'urgent' ? 0 : 1;
            if (priorityA !== priorityB) return priorityA - priorityB;
```

```
            // Then by due date
            const dateA = new Date(a.estimatedCompletion || '2099-12-31');
            const dateB = new Date(b.estimatedCompletion || '2099-12-31');
            return dateA - dateB;
        } else if (sortBy === 'dueDate') {
            const dateA = new Date(a.estimatedCompletion || '2099-12-31');
            const dateB = new Date(b.estimatedCompletion || '2099-12-31');
            return dateA - dateB;
        } else if (sortBy === 'orderId') {
            const idA = a.internalOrderNo || a.id || '';
            const idB = b.internalOrderNo || b.id || '';
            return idA.localeCompare(idB);
        }
        return 0;
    });

    // Update count badge
    if (countBadge) {
        countBadge.textContent = `${pendingOrders.length} orders`;
    }

    // Build table rows
    if (pendingOrders.length === 0) {
        tbody.innerHTML = '<tr><td colspan="8" class="text-center py-8 text-slate-400">No
pending orders found</td></tr>';
        return;
    }

    // Build member dropdown options
    const memberOptions = currentMembers.map(m =>
        `<option value="${m.id}">${m.name}</option>`
    ).join('');

    tbody.innerHTML = pendingOrders.map(order => {
        const isUrgent = order.priority === 'urgent';
        const assignedList = (order.assignedTo || []).map(id => {
            return {
                id: id,
                name: currentMembers.find(m => m.id === id)?.name || id
            };
        });

        // Format due date for input (YYYY-MM-DD)
        const dueDateValue = order.estimatedCompletion
            ? new Date(order.estimatedCompletion).toISOString().split('T')[0]
            : '';

        const rowClass = isUrgent ? 'pending-row-urgent' : 'pending-row-normal';

        return `
            <tr class="pending-assignment-row ${rowClass}">
                <td style="text-align: center; vertical-align: middle;">
```

```
                    <button onclick="window.adminApp.togglePriority('${order.id}')"
                            class="priority-toggle ${isUrgent ? 'is-urgent' : ''}"
                            title="${isUrgent ? 'Mark as Normal' : 'Mark as Urgent'}">
                        ${isUrgent ? '🔥' : '🟣'}
                    </button>
                </td>
                <td><span class="order-id-badge">${order.internalOrderNo || order.id}</span>
</td>
                <td style="font-weight: 500;">${order.customer || '-'}</td>
                <td>${order.description || '-'}</td>
                <td><span style="font-weight: 600; color: var(--brand-600);">${order.drawingNo
|| '-'}</span></td>
                <td style="font-weight: 600; text-align: center;">${order.qty || '-'}</td>
                <td style="text-align: center;">${order.qtyUnit || '-'}</td>
                <td>
                    <input type="date" class="table-form-input"
                           value="${dueDateValue}"
                           onchange="window.adminApp.updateDueDate('${order.id}', this.value)"
                           title="Click to set/change due date">
                </td>
                <td class="assign-cell">
                    <select class="assign-dropdown"
                            onchange="window.adminApp.updateAssignment('${order.id}',
this.value)"
                            id="assign-${order.id}">
                        <option value="">+ Add</option>
                        ${memberOptions}
                    </select>
                    ${assignedList.length > 0 ? `
                    <div class="assign-chips">
                        ${assignedList.map(m => {
            const initials = m.name.split(' ').map(w => w[0]).join('').toUpperCase().slice(0,
2);
            return `
                            <span class="assign-chip">
                                <span class="assign-chip-avatar">${initials}</span>
                                <span class="assign-chip-name">${m.name}</span>
                                <button class="assign-chip-remove"

onclick="window.adminApp.removeAssignment('${order.id}', '${m.id}')"
                                        title="Remove">×</button>
                            </span>`;
        }).join('')}
                    </div>` : ''}
                </td>
                <td>
                    <input type="text" class="table-form-input"
                           placeholder="Add remarks..."
                           value="${order.remarks || ''}"
                           onblur="window.adminApp.saveRemarks('${order.id}', this.value)"
                           id="remarks-${order.id}">
                </td>
```

```
            </tr>
        `;
    }).join('');


},

updateAssignment: async (orderId, memberId) => {
    if (!memberId) return;

    const order = currentOrders.find(o => o.id === orderId);
    if (!order) return;

    const currentAssigned = order.assignedTo || [];
    if (!currentAssigned.includes(memberId)) {
        currentAssigned.push(memberId);
    }

    // Log history
    const historyEntry = {
        action: 'assigned',
        memberId: memberId,
        memberName: currentMembers.find(m => m.id === memberId)?.name || memberId,
        timestamp: new Date().toISOString(),
        assignedBy: 'admin' // TODO: Get current user
    };

    const assignmentHistory = order.assignmentHistory || [];
    assignmentHistory.push(historyEntry);

    // Update in Firestore
    const result = await DB.updateOrder(orderId, {
        assignedTo: currentAssigned,
        assignmentHistory: assignmentHistory
    });

    if (!result.error) {
        window.adminApp.renderPendingAssignment();
    }
},

removeAssignment: async (orderId, memberId) => {
    const order = currentOrders.find(o => o.id === orderId);
    if (!order) return;

    const currentAssigned = (order.assignedTo || []).filter(id => id !== memberId);
    const memberName = currentMembers.find(m => m.id === memberId)?.name || memberId;

    // Log history
    const historyEntry = {
        action: 'removed',
        memberId: memberId,
```

```
        memberName: memberName,
        timestamp: new Date().toISOString(),
        assignedBy: 'admin'
    };

    const assignmentHistory = order.assignmentHistory || [];
    assignmentHistory.push(historyEntry);

    // Update in Firestore
    const result = await DB.updateOrder(orderId, {
        assignedTo: currentAssigned,
        assignmentHistory: assignmentHistory
    });

    if (!result.error) {
        window.adminApp.renderPendingAssignment();
    }
},

saveRemarks: async (orderId, remarks) => {
    const result = await DB.updateOrder(orderId, {
        remarks: remarks
    });

    if (result.error) {
        console.error('Failed to save remarks:', result.error);
    }
},

updateDueDate: async (orderId, dateValue) => {
    const result = await DB.updateOrder(orderId, {
        estimatedCompletion: dateValue
    });

    if (!result.error) {
        // Refresh to re-sort if needed
        window.adminApp.renderPendingAssignment();
    } else {
        console.error('Failed to save due date:', result.error);
    }
},

togglePriority: async (orderId) => {
    const order = currentOrders.find(o => o.id === orderId);
    if (!order) return;

    const newPriority = order.priority === 'urgent' ? 'normal' : 'urgent';

    const result = await DB.updateOrder(orderId, {
        priority: newPriority
    });
```

```
    if (!result.error) {
        window.adminApp.renderPendingAssignment();
    }
},


generatePendingReport: () => {
    // Get currently displayed pending orders
    const pendingOrders = currentOrders.filter(o => o.status === 'Pending' && !o.deleted);

    if (pendingOrders.length === 0) {
        alert('No pending orders to generate report.');
        return;
    }

    // Calculate stats
    const total = pendingOrders.length;
    const urgent = pendingOrders.filter(o => o.priority === 'urgent').length;
    const assigned = pendingOrders.filter(o => o.assignedTo && o.assignedTo.length >
0).length;
    const unassigned = total - assigned;

    // Employee workload
    const workload = {};
    pendingOrders.forEach(o => {
        (o.assignedTo || []).forEach(id => {
            const name = currentMembers.find(m => m.id === id)?.name || id;
            workload[name] = (workload[name] || 0) + 1;
        });
    });

    // Generate print content
    const today = new Date().toLocaleDateString('en-IN', {
        day: '2-digit', month: 'long', year: 'numeric'
    });

    const reportHTML = `
        <html>
        <head>
            <title>Pending Orders Report - ${today}</title>
            <style>
                body { font-family: Arial, sans-serif; padding: 20px; }
                h1 { text-align: center; color: #1e293b; }
                .summary { display: flex; gap: 20px; margin-bottom: 20px; }
                .stat-box { background: #f1f5f9; padding: 15px; border-radius: 8px; flex: 1;
text-align: center; }
                .stat-box h3 { margin: 0; font-size: 24px; color: #0d9488; }
                .stat-box p { margin: 5px 0 0; color: #64748b; }
                table { width: 100%; border-collapse: collapse; margin-top: 20px; }
                th, td { border: 1px solid #e2e8f0; padding: 8px; text-align: left; font-size:
12px; }
                th { background: #1e293b; color: white; }
```

```
                .urgent { background: #fef2f2; }
                .workload { margin-top: 20px; }
                .workload-item { padding: 8px 0; border-bottom: 1px solid #e2e8f0; }
            </style>
        </head>
        <body>
            <h1>📋 IES Groups - Pending Orders Report</h1>
            <p style="text-align: center; color: #64748b;">${today}</p>

            <div class="summary">
                <div class="stat-box"><h3>${total}</h3><p>Total Pending</p></div>
                <div class="stat-box"><h3 style="color: #ef4444;">${urgent}</h3><p>Urgent</p>
</div>
                <div class="stat-box"><h3 style="color: #22c55e;">${assigned}</h3>
<p>Assigned</p></div>
                <div class="stat-box"><h3 style="color: #f59e0b;">${unassigned}</h3>
<p>Unassigned</p></div>
            </div>

            <h2>Employee Workload</h2>
            <div class="workload">
                ${Object.entries(workload).map(([name, count]) =>
        `<div class="workload-item"><strong>${name}</strong>: ${count} orders</div>`
    ).join('')}
                ${Object.keys(workload).length === 0 ? '<p>No assignments yet</p>' : ''}
            </div>

            <h2>Order Details</h2>
            <table>
                <thead>
                    <tr>
                        <th>Priority</th>
                        <th>Order ID</th>
                        <th>Customer</th>
                        <th>Description</th>
                        <th>Drg No</th>
                        <th>Qty</th>
                        <th>Unit</th>
                        <th>Due Date</th>
                        <th>Assigned To</th>
                        <th>Remarks</th>
                    </tr>
                </thead>
                <tbody>
                    ${pendingOrders.map(o => {
        const assignedNames = (o.assignedTo || []).map(id =>
            currentMembers.find(m => m.id === id)?.name || id
        ).join(', ') || 'Unassigned';
        const dueDate = o.estimatedCompletion
            ? new Date(o.estimatedCompletion).toLocaleDateString('en-IN')
            : '-';
        return `
```

```
                            <tr class="${o.priority === 'urgent' ? 'urgent' : ''}">
                                <td>${o.priority === 'urgent' ? '🔴 Urgent' : '⚪ Normal'}
</td>
                                <td>${o.internalOrderNo || o.id}</td>
                                <td>${o.customer || '-'}</td>
                                <td>${o.description || '-'}</td>
                                <td>${o.drawingNo || '-'}</td>
                                <td>${o.qty || '-'}</td>
                                <td>${o.qtyUnit || '-'}</td>
                                <td>${dueDate}</td>
                                <td>${assignedNames}</td>
                                <td>${o.remarks || '-'}</td>
                            </tr>
                        `;
    }).join('')}
                </tbody>
            </table>

            <script>window.print();</script>
        </body>
        </html>
    `;

    // Open print window
    const printWindow = window.open('', '_blank');
    printWindow.document.write(reportHTML);
    printWindow.document.close();
},

// ============================================
// DAILY REPORTS FUNCTIONS
// ============================================

renderReports: async () => {
    const tbody = document.getElementById('reports-body');
    if (!tbody) return;

    tbody.innerHTML = '<tr><td colspan="7" class="text-center py-8 text-slate-400">Loading
reports...</td></tr>';

    const reports = await DB.getReports(30);

    if (reports.length === 0) {
        tbody.innerHTML = '<tr><td colspan="7" class="text-center py-8 text-slate-400">No
reports yet. Click "Generate Today\'s Report" to create one.</td></tr>';
        return;
    }

    tbody.innerHTML = reports.map(report => {
        const displayDate = new Date(report.date).toLocaleDateString('en-IN', {
            weekday: 'short', day: '2-digit', month: 'short', year: 'numeric'
        });
```

```
        const generatedAt = report.createdAt?.toDate
            ? report.createdAt.toDate().toLocaleString('en-IN')
            : (report.updatedAt?.toDate ? report.updatedAt.toDate().toLocaleString('en-IN') :
'Unknown');

        return `
            <tr>
                <td style="font-weight: 600;">${displayDate}</td>
                <td style="text-align: center;">${report.totalOrders || 0}</td>
                <td style="text-align: center; color: #ef4444;">${report.urgent || 0}</td>
                <td style="text-align: center; color: #22c55e;">${report.assigned || 0}</td>
                <td style="text-align: center; color: #f59e0b;">${report.unassigned || 0}</td>
                <td style="font-size: 0.75rem; color: #64748b;">${generatedAt}</td>
                <td style="text-align: center;">
                    <button class="btn btn-ghost btn-icon"
                            onclick="window.adminApp.printSavedReport('${report.date}')"
                            title="Print Report">
                        🖨
                    </button>
                </td>
            </tr>
        `;
    }).join('');
},

generateAndSaveReport: async () => {
    const today = new Date();
    const dateStr = today.toISOString().split('T')[0]; // YYYY-MM-DD

    // 1. Active Internal Orders (Excluding delivered and trash)
    const activeOrdersSnapshot = currentOrders.filter(o =>
        o.status !== 'Delivered' && !o.deleted && o.entryType !== 'delivery_report'
    ).map(o => ({
        id: o.id || '',
        internalOrderNo: o.internalOrderNo || '',
        drawingNo: o.drawingNo || '',
        description: o.description || '',
        customer: o.customer || '',
        qty: o.qty || '',
        qtyUnit: o.qtyUnit || '',
        priority: o.priority || 'normal',
        status: o.status || 'Pending',
        estimatedCompletion: o.estimatedCompletion || ''
    }));

    // 2. Pending Assignments (Subset of active with no members assigned)
    const pendingAssignmentsSnapshot = currentOrders.filter(o =>
        o.status === 'Pending' && !o.deleted && (!o.assignedTo || o.assignedTo.length === 0)
    ).map(o => ({
        id: o.id || '',
        internalOrderNo: o.internalOrderNo || '',
```

```
        drawingNo: o.drawingNo || '',
        description: o.description || '',
        customer: o.customer || '',
        priority: o.priority || 'normal',
        estimatedCompletion: o.estimatedCompletion || ''
    }));

    // 3. Today's Delivery Report
    const deliverySnapshot = currentOrders.filter(o =>
        o.status === 'Delivered' && o.deliveryDateActual === dateStr && o.entryType ===
'delivery_report'
    ).map(o => ({
        internalOrderNo: o.internalOrderNo || '',
        customer: o.customer || '',
        description: o.description || '',
        drawingNo: o.drawingNo || '',
        deliveryQty: o.deliveryQty || o.qty || 0,
        dcNo: o.dcNo || '-',
        total: o.total || 0
    }));

    // 4. Daily Roster (Today's workflow from all departments)
    const rosterData = await DB.getWorkflowsForDate(dateStr);
    const rosterSnapshot = rosterData.map(wf => ({
        department: wf.department,
        assignments: (wf.assignments || []).map(a => ({
            employeeName: a.employeeName,
            employeeNo: a.employeeNo,
            tasks: (a.tasks || []).map(t => ({
                orderNo: t.orderNo,
                description: t.description,
                status: t.status,
                workDuration: `${t.workStart || ''} to ${t.workEnd || ''}`
            }))
        })),
        notes: wf.supervisorNotes || ''
    }));

    const reportData = {
        date: dateStr,
        totalOrders: activeOrdersSnapshot.length,
        urgent: activeOrdersSnapshot.filter(o => o.priority === 'urgent').length,
        assigned: activeOrdersSnapshot.length - pendingAssignmentsSnapshot.length,
        unassigned: pendingAssignmentsSnapshot.length,
        activeOrdersSnapshot,
        pendingAssignmentsSnapshot,
        deliverySnapshot,
        rosterSnapshot,
        generatedAt: new Date().toISOString()
    };

    const result = await DB.saveReport(reportData);
```

```
    if (result.success) {
        alert(`Report for ${dateStr} saved successfully!`);
        window.adminApp.renderReports();
    } else {
        alert('Failed to save report: ' + (result.error || 'Unknown error'));
    }
},

printSavedReport: async (dateStr) => {
    const report = await DB.checkTodayReport(dateStr);
    if (!report) {
        alert('Report not found.');
        return;
    }

    const displayDate = new Date(dateStr).toLocaleDateString('en-IN', {
        day: '2-digit', month: 'long', year: 'numeric'
    });

    const reportHTML = `
        <html>
        <head>
            <title>IES Groups - Daily Report - ${displayDate}</title>
            <style>
                body { font-family: 'Segoe UI', Arial, sans-serif; padding: 20px; color:
#1e293b; line-height: 1.5; }
                .header { text-align: center; border-bottom: 2px solid #0d9488; padding-
bottom: 10px; margin-bottom: 20px; }
                .header h1 { margin: 0; color: #0d9488; font-size: 24px; text-transform:
uppercase; }
                .header p { margin: 5px 0 0; color: #64748b; font-weight: bold; }

                .section-title { background: #f1f5f9; padding: 8px 12px; border-left: 4px
solid #0d9488; margin: 25px 0 15px; font-size: 16px; font-bold; text-transform: uppercase; }

                .summary-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap:
15px; margin-bottom: 25px; }
                .stat-box { background: white; border: 1px solid #e2e8f0; padding: 12px;
border-radius: 8px; text-align: center; }
                .stat-box h3 { margin: 0; font-size: 20px; color: #0d9488; }
                .stat-box p { margin: 4px 0 0; font-size: 11px; color: #64748b; text-
transform: uppercase; font-weight: bold; }

                table { width: 100%; border-collapse: collapse; margin-bottom: 20px; table-
layout: fixed; }
                th, td { border: 1px solid #e2e8f0; padding: 8px 10px; text-align: left; font-
size: 11px; word-wrap: break-word; }
                th { background: #f8fafc; color: #475569; font-weight: bold; text-transform:
uppercase; border-bottom: 2px solid #e2e8f0; }

                .badge { padding: 2px 6px; border-radius: 4px; font-size: 10px; font-weight:
```

```
bold; }
                .badge-urgent { background: #fef2f2; color: #ef4444; }
                .badge-normal { background: #f1f5f9; color: #64748b; }

                .roster-card { border: 1px solid #e2e8f0; border-radius: 8px; margin-bottom:
15px; page-break-inside: avoid; }
                .roster-header { background: #f8fafc; border-bottom: 1px solid #e2e8f0;
padding: 6px 12px; font-weight: bold; font-size: 12px; color: #0d9488; }
                .roster-body { padding: 10px; }

                .footer { margin-top: 40px; font-size: 10px; color: #94a3b8; text-align:
center; border-top: 1px solid #f1f5f9; padding-top: 10px; }
                @media print {
                    .page-break { page-break-before: always; }
                    body { padding: 0; }
                }
            </style>
        </head>
        <body>
            <div class="header">
                <h1>Innovative Engineering Solutions Groups</h1>
                <p>Daily Operations Report - ${displayDate}</p>
            </div>

            <div class="summary-grid">
                <div class="stat-box"><h3>${report.totalOrders}</h3><p>Active Orders</p></div>
                <div class="stat-box"><h3>${report.unassigned}</h3><p>Unassigned</p></div>
                <div class="stat-box"><h3>${report.urgent}</h3><p>Urgent Items</p></div>
                <div class="stat-box"><h3>${report.deliverySnapshot?.length || 0}</h3><p>Items
Delivered</p></div>
            </div>

            <!-- SECTION 1: DAILY ROSTER -->
            <div class="section-title">1. Daily Roster (Work Assignments)</div>
            ${(report.rosterSnapshot || []).length > 0 ? (report.rosterSnapshot ||
[]).map(dept => `
                <div class="roster-card">
                    <div class="roster-header">${dept.department} Division</div>
                    <div class="roster-body">
                        <table>
                            <thead>
                                <tr>
                                    <th style="width: 25%;">Employee</th>
                                    <th style="width: 15%;">Order No</th>
                                    <th>Task Description</th>
                                    <th style="width: 15%;">Status</th>
                                    <th style="width: 15%;">Duration</th>
                                </tr>
                            </thead>
                            <tbody>
                                ${dept.assignments.map(a => `
                                    ${a.tasks.map((t, idx) => `
```

```
                                        <tr>
                                            ${idx === 0 ? `<td rowspan="${a.tasks.length}">
<strong>${a.employeeName}</strong><br><small>${a.employeeNo}</small></td>` : ''}
                                            <td>${t.orderNo || '-'}</td>
                                            <td>${t.description}</td>
                                            <td style="text-align:center;">${t.status}</td>
                                            <td style="text-align:center;">${t.workDuration}
</td>
                                        </tr>
                                    `).join('')}
                                `).join('')}
                            </tbody>
                        </table>
                        ${dept.notes ? `<p style="font-size: 11px; color: #64748b; font-style:
italic;"><strong>Note:</strong> ${dept.notes}</p>` : ''}
                    </div>
                </div>
            `).join('') : '<p style="text-align:center; color:#94a3b8; font-size:12px;">No
roster data recorded for today.</p>'}

            <div class="page-break"></div>

            <!-- SECTION 2: DELIVERY REPORT -->
            <div class="section-title">2. Items Delivered Today</div>
            ${(report.deliverySnapshot || []).length > 0 ? `
            <table>
                <thead>
                    <tr>
                        <th style="width: 12%;">Order No</th>
                        <th style="width: 20%;">Customer</th>
                        <th>Description</th>
                        <th style="width: 15%;">Drawing No</th>
                        <th style="width: 10%; text-align:right;">Qty</th>
                        <th style="width: 12%;">DC No</th>
                    </tr>
                </thead>
                <tbody>
                    ${report.deliverySnapshot.map(o => `
                        <tr>
                            <td><strong>${o.internalOrderNo}</strong></td>
                            <td>${o.customer}</td>
                            <td>${o.description}</td>
                            <td>${o.drawingNo}</td>
                            <td style="text-align:right;">${o.deliveryQty}</td>
                            <td>${o.dcNo}</td>
                        </tr>
                    `).join('')}
                </tbody>
            </table>
            ` : '<p style="text-align:center; color:#94a3b8; font-size:12px;">No deliveries
recorded today.</p>'}
```

```
            <!-- SECTION 3: PENDING ASSIGNMENTS -->
            <div class="section-title">3. Pending Assignments (Priority Action)</div>
            ${(report.pendingAssignmentsSnapshot || []).length > 0 ? `
            <table>
                <thead>
                    <tr>
                        <th style="width: 10%;">Priority</th>
                        <th style="width: 15%;">Order No</th>
                        <th style="width: 20%;">Customer</th>
                        <th>Description</th>
                        <th style="width: 15%;">Target Date</th>
                    </tr>
                </thead>
                <tbody>
                    ${report.pendingAssignmentsSnapshot.map(o => `
                        <tr>
                            <td style="text-align:center;"><span class="badge
badge-${o.priority.toLowerCase()}">${o.priority.toUpperCase()}</span></td>
                            <td><strong>${o.internalOrderNo || o.id}</strong></td>
                            <td>${o.customer}</td>
                            <td>${o.description}</td>
                            <td>${o.estimatedCompletion ? new
Date(o.estimatedCompletion).toLocaleDateString('en-IN') : '-'}</td>
                        </tr>
                    `).join('')}
                </tbody>
            </table>
            ` : '<p style="text-align:center; color:#94a3b8; font-size:12px;">All pending
orders have been assigned.</p>'}

            <div class="page-break"></div>

            <!-- SECTION 4: FULL INTERNAL ORDER STATUS -->
            <div class="section-title">4. Active Internal Orders (Backlog)</div>
            <table>
                <thead>
                    <tr>
                        <th style="width: 12%;">Order No</th>
                        <th style="width: 15%;">Customer</th>
                        <th>Description</th>
                        <th style="width: 15%;">Drawing No</th>
                        <th style="width: 10%; text-align:right;">Qty</th>
                        <th style="width: 12%;">Status</th>
                    </tr>
                </thead>
                <tbody>
                    ${(report.activeOrdersSnapshot || []).map(o => `
                        <tr>
                            <td><strong>${o.internalOrderNo}</strong></td>
                            <td>${o.customer}</td>
                            <td>${o.description}</td>
                            <td>${o.drawingNo}</td>
```

```
                            <td style="text-align:right;">${o.qty} ${o.qtyUnit}</td>
                            <td style="text-align:center;"><span class="badge badge-
normal">${o.status}</span></td>
                        </tr>
                    `).join('')}
                </tbody>
            </table>

            <div class="footer">
                Report generated on ${new Date(report.generatedAt).toLocaleString('en-IN')} |
IES Groups Management System
            </div>
        </body>
        </html>
    `;

    const printWin = window.open('', '', 'width=1000,height=800');
    printWin.document.write(reportHTML);
    printWin.document.close();

    // Wait for fonts/images and then print
    setTimeout(() => {
        printWin.print();
    }, 500);
},

checkAutoGenerateReport: async () => {
    // Get current IST time using Intl.DateTimeFormat
    const now = new Date();
    const istFormatter = new Intl.DateTimeFormat('en-CA', {
        timeZone: 'Asia/Kolkata',
        year: 'numeric',
        month: '2-digit',
        day: '2-digit',
        hour: '2-digit',
        hour12: false
    });

    const parts = istFormatter.formatToParts(now);
    const getTimePart = (type) => parts.find(p => p.type === type).value;

    const hours = parseInt(getTimePart('hour'));
    const todayStr = `${getTimePart('year')} -${getTimePart('month')} -${getTimePart('day')}
`;

    // Check if it's past 7 PM IST (19:00)
    if (hours >= 19) {
        // Check if report for today already exists
        const existingReport = await DB.checkTodayReport(todayStr);

        if (!existingReport) {
            console.log('Auto-generating report for', todayStr);
```

```
            // Wait a bit for orders to be loaded if they aren't already
            setTimeout(async () => {
                if (currentOrders && currentOrders.length > 0) {
                    await window.adminApp.generateAndSaveReport();
                    console.log('Auto-report generated successfully');
                }
            }, 3000);
        } else {
            console.log('Report for today already exists');
        }
    }
}
```

};

// DOM Ready document.addEventListener('DOMContentLoaded', () => {

```
// Setup UI Listeners
UI.setupNavigation();

const monitoringView = document.getElementById('view-monitoring');
const deliveryView = document.getElementById('view-delivery_report');

// Dashboard Filter Listener
const dashMonthFilter = document.getElementById('dashboard-month-filter');
if (dashMonthFilter) {
    dashMonthFilter.addEventListener('change', refreshDashboard);
}

// Load Members
DB.subscribeToMembers((members) => {
    currentMembers = members;
    const memberView = document.getElementById('subview-members');
    if (memberView && !memberView.classList.contains('hidden')) {
        UI.renderMemberList(members);
    }
    refreshDashboard();
});

// Load Projects for Kanban / Modern View
DB.subscribeToProjects((projects) => {
    currentProjects = projects;
    const view = document.getElementById('view-project_management');
    if (view && !view.classList.contains('hidden')) {
        window.adminApp.filterProjects();
    }
    window.adminApp.updateProjectStats();
});

// Load Orders
window.unsubscribeOrders = DB.subscribeToOrders((orders) => {
```

```
    // Mock Data Injection if empty (for Demo/Dev)
    if (!orders || orders.length < 5) {
        console.log("Injecting Mock Orders for Demo...");
        const mockOrders = Array.from({ length: 15 }).map((_, i) => ({
            id: `mock - ${i} `,
            internalOrderNo: `2026-02 - ${500 + i} `,
            customer: ['Baliga', 'Bray Controls', 'Flowserve', 'L&T'][Math.floor(Math.random()
* 4)],
            description: `Machining of ${['Valve Body', 'Flange', 'Shaft', 'Housing']
[Math.floor(Math.random() * 4)]} `,
            date: new Date(Date.now() - Math.floor(Math.random() * 86400000 *
3)).toISOString().split('T')[0], // Last 3 days
            delDate: new Date(Date.now() + Math.floor(Math.random() * 86400000 *
10)).toISOString().split('T')[0],
            status: 'Pending',
            createdAt: { seconds: Date.now() / 1000 - (i * 3600) } // Staggered times
        }));
        currentOrders = [...orders, ...mockOrders];
    } else {
        currentOrders = orders;
    }

    // One-time migration: convert any "In Progress" orders to "Pending"
    currentOrders.forEach(o => {
        if (o.status === 'In Progress' && o.id) {
            DB.updateOrder(o.id, { status: 'Pending' });
            o.status = 'Pending';
        }
    });

    if (monitoringView && !monitoringView.classList.contains('hidden')) {
        Monitoring.renderTable(currentOrders);
    }

    // --- ADDED: Live Update for Pending Assignment ---
    const pendingView = document.getElementById('view-pending_assignment');
    if (pendingView && !pendingView.classList.contains('hidden')) {
        window.adminApp.renderPendingAssignment();
    }

    if (deliveryView && !deliveryView.classList.contains('hidden')) {
        const weekPicker = document.getElementById('delivery-week-picker');
        if (weekPicker && weekPicker.value) {
            Monitoring.renderDeliveryReport(weekPicker.value);
        }
    }
    refreshDashboard();
}, false);

// Filter Listeners
// Filter Listeners
const monthFromInput = document.getElementById('order-month-from');
```

```
const monthToInput = document.getElementById('order-month-to');
const searchInput = document.getElementById('order-search-filter');

if (monthFromInput && monthToInput) {
    // Initialize from Persistence (via Monitoring module state)
    // If Monitoring.getFilters().monthFrom is '' (show all), input should be ''
    const currentFilters = Monitoring.getFilters();
    monthFromInput.value = currentFilters.monthFrom || '';
    monthToInput.value = currentFilters.monthTo || '';

    const updateFilters = () => {
        Monitoring.setFilters(monthFromInput.value, monthToInput.value, undefined);
        Monitoring.renderTable(currentOrders);
    };

    monthFromInput.addEventListener('change', updateFilters);
    monthToInput.addEventListener('change', updateFilters);
}

if (searchInput) {
    searchInput.addEventListener('input', (e) => {
        Monitoring.setFilters(undefined, undefined, e.target.value);
        Monitoring.renderTable(currentOrders);
    });
}

// Form Submissions
const orderForm = document.getElementById('add-order-form');
if (orderForm) {
    orderForm.addEventListener('submit', (e) => {
        e.preventDefault();
        Monitoring.handleAddOrder();
    });
}

const memberForm = document.getElementById('add-member-form');
if (memberForm) {
    memberForm.addEventListener('submit', (e) => {
        e.preventDefault();
        window.adminApp.submitMemberForm();
    });
}

const projectForm = document.getElementById('add-project-form');
if (projectForm) {
    projectForm.addEventListener('submit', (e) => {
        e.preventDefault();
        window.adminApp.submitProjectForm();
    });
}

// Login Form
```

```
const loginForm = document.getElementById('login-form');
if (loginForm) {
    loginForm.addEventListener('submit', async (e) => {
        e.preventDefault();
        const email = document.getElementById('email-address').value;
        const password = document.getElementById('password').value;

        try {
            const result = await Auth.login(email, password);
            if (result.error) {
                UI.showLoginError(result.error);
            } else {
                // Force View Switch
                const authDiv = document.getElementById('auth-container');
                const dashDiv = document.getElementById('dashboard-container');
                const emailSpan = document.getElementById('user-email-display');

                if (authDiv) authDiv.style.display = 'none';
                if (dashDiv) {
                    dashDiv.style.display = 'flex';
                    dashDiv.classList.remove('hidden');
                }
                if (emailSpan && result.user) emailSpan.textContent = result.user.email;

                // Refresh dashboard after login to use real data
                refreshDashboard();
            }
        } catch (err) {
            console.error(err);
            UI.showLoginError(err.message);
        }
    });
}

// Logout
const logoutBtn = document.getElementById('logout-btn');
if (logoutBtn) {
    logoutBtn.addEventListener('click', async () => {
        await Auth.logout();
    });
}

// Navigation click handler for view renders
document.querySelectorAll('.nav-link').forEach(link => {
    link.addEventListener('click', () => {
        const view = link.dataset.view;

        setTimeout(() => {
            if (view === 'team_org') {
                UI.renderMemberList(currentMembers);
            } else if (view === 'project_management') {
                window.adminApp.filterProjects();
```

```
            } else if (view === 'overview') {
                refreshDashboard();
            } else if (view === 'monitoring') {
                Monitoring.renderTable(currentOrders);
            } else if (view === 'delivery_report') {
                // Initialize with current week if not set
                const weekPicker = document.getElementById('delivery-week-picker');
                if (weekPicker && !weekPicker.value) {
                    const today = new Date();

                    // Get ISO week number
                    const date = new Date(today.getTime());
                    date.setHours(0, 0, 0, 0);
                    date.setDate(date.getDate() + 3 - (date.getDay() + 6) % 7);
                    const week1 = new Date(date.getFullYear(), 0, 4);
                    const weekNum = 1 + Math.round(((date.getTime() - week1.getTime()) /
86400000 - 3 + (week1.getDay() + 6) % 7) / 7);

                    const year = date.getFullYear();
                    weekPicker.value = `${year}-W${weekNum.toString().padStart(2, '0')}`;
                    Monitoring.renderDeliveryReport(weekPicker.value);
                } else if (weekPicker && weekPicker.value) {
                    // Re-render if already set (e.g. data updated)
                    Monitoring.renderDeliveryReport(weekPicker.value);
                }
            } else if (view === 'pending_assignment') {
                // Render pending assignment table
                window.adminApp.renderPendingAssignment();
            } else if (view === 'reports') {
                // Render daily reports list
                window.adminApp.renderReports();
            } else if (view === 'inventory_management') {
                // No action needed for static Coming Soon page for now,
                // UI.switchView(view) handles toggling the hidden class.
            }

        }, 50);
    });
});

// Delivery Report Date Picker
const weekPicker = document.getElementById('delivery-week-picker');
if (weekPicker) {
    weekPicker.addEventListener('change', (e) => {
        const monthPicker = document.getElementById('delivery-month-picker');
        if (monthPicker) monthPicker.value = ''; // Clear month if week is picked
        Monitoring.renderDeliveryReport(e.target.value);
    });
}

const deliveryMonthPicker = document.getElementById('delivery-month-picker');
if (deliveryMonthPicker) {
```

```
    deliveryMonthPicker.addEventListener('change', (e) => {
        const weekPicker = document.getElementById('delivery-week-picker');
        if (weekPicker) weekPicker.value = ''; // Clear week if month is picked
        Monitoring.renderDeliveryReport(undefined, e.target.value);
    });
}

// Pending Assignment Filter Listeners
const pendingFilters = ['pending-filter-department', 'pending-filter-assigned', 'pending-
filter-priority', 'pending-sort-by'];
pendingFilters.forEach(filterId => {
    const filter = document.getElementById(filterId);
    if (filter) {
        filter.addEventListener('change', () => {
            window.adminApp.renderPendingAssignment();
        });
    }
});

// Auth State Observer
Auth.subscribeToAuthChanges((user) => {
    const authDiv = document.getElementById('auth-container');
    const dashDiv = document.getElementById('dashboard-container');
    const emailSpan = document.getElementById('user-email-display');

    if (user) {
        if (authDiv) authDiv.style.display = 'none';
        if (dashDiv) {
            dashDiv.style.display = 'flex';
            dashDiv.classList.remove('hidden');
        }
        if (emailSpan) emailSpan.textContent = user.email;

        // Subscribe to Projects
        DB.subscribeToProjects((projects) => {
            currentProjects = projects;

            // Update Customer filter in Project Management
            const customerFilter = document.getElementById('project-customer-filter');
            if (customerFilter) {
                const customers = [...new Set(projects.map(p => p.customerName).filter(c =>
c))];
                const currentVal = customerFilter.value;
                customerFilter.innerHTML = '<option value="all">All Customers</option>' +
                    customers.map(c => `< option value = "${c}" > ${c}</option > `).join('');
                customerFilter.value = currentVal;
            }

            // Refresh UI based on active view
            const activeView = UI.getActiveView();
            if (activeView === 'project_management') {
                window.adminApp.filterProjects();
```

```
            } else if (activeView === 'project_detail') {
                // Re-render detail view if the project exists
                const detailId = document.getElementById('detail-project-id')?.textContent;
                // Note: detailId might be the projectId (IES-...) not the Firebase doc id.
                // We need a way to track which internal ID is active.
                // Let's assume we can find it in currentProjects or via a global state.
                const activeProject = projects.find(p => p.projectId === detailId);
                if (activeProject) {


                    // Update basic info too in case it changed
                    const statusDisplay = document.getElementById('detail-project-status');
                    if (statusDisplay) {
                        statusDisplay.textContent = activeProject.status;
                        statusDisplay.className = `status - badge
${activeProject.status?.toLowerCase().replace(' ', '-')} `;
                    }
                }
            }
        });

        // Ensure dashboard is ready immediately
        refreshDashboard();

        // Auto-generate report if past 7 PM IST
        window.adminApp.checkAutoGenerateReport();
    } else {
        if (authDiv) authDiv.style.display = 'flex';
        if (dashDiv) dashDiv.style.display = 'none';
    }
});
```

});

// --- Contract Review Logic ---

const checklistItems = [{ id: 'item_1', label: 'Product Description' }, { id: 'item_2', label: 'BOM' }, { id: 'item_3', label: 'Qty' }, { id: 'item_4', label: 'Price Acceptance' }, { id: 'item_5', label: 'MOC' }, { id: 'item_6', label: 'Payment Terms' }, { id: 'item_7', label: 'Approved Drawing' }, { id: 'item_8', label: '3rd Party Inspection' }, { id: 'item_9', label: 'Delivery Date' }, { id: 'item_10', label: 'LD clause' }];

window.adminApp.renderContractReview = (reviewData = {}) => { const container = document.getElementById('cr-excel-checklist'); if (!container) return;

```
const data = reviewData || {};
let html = '';

const renderRow = (item, index) => {
    const itemObj = data[item.id] || {};
    const isCustom = item.isCustom;

    let labelEl = item.label;
    if (isCustom) {
```

```
        labelEl = `< input type = "text" class="cr-master-input cr-custom-label w-full font-
bold px-3 py-2 text-slate-700" value = "${itemObj.customLabel || ''}" data - item -
id="${item.id}" > `;
    } else {
        labelEl = `< span class="px-3 block w-full py-2 font-bold text-slate-700" >
${item.label}</span > `;
    }

    const req = itemObj.req || '';
    const out = itemObj.out || '';

    const cell = (type, val, currentVal, extraVal) => {
        const isActive = type === 'out' && val === 'more' ? extraVal === 'true' : currentVal
=== val;
        const activeClass = isActive ? `active - ${val} ` : '';
        const tick = isActive ? '✓' : 'o';

        return `
< td class="p-0 select-none" onclick = "window.adminApp.setReviewItem('${item.id}', '${type}',
'${val}')" >
    <div class="outcome-tile ${activeClass}" data-opt="${val}">
        <span class="cr-tick">${tick}</span>
    </div>
            </td >
`;
    };

    const deleteBtn = isCustom ? `< button class="cr-item-delete-btn text-red-400 hover:text-
red-600 px-3 flex-shrink-0" onclick = "window.adminApp.removeReviewItem('${item.id}')" title =
"Remove Item" > <svg width="14" height="14" fill="none" stroke="currentColor" viewBox="0 0 24
24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 7l-.867
12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1
0 00-1 1v3M4 7h16" /></svg></button > ` : '';

    return `
< tr class="cr-item-row" data - item - id="${item.id}" data - req - val="${req}" data - out -
val="${out}" data - more - val="${itemObj.more || 'false'}" >
            <td class="cr-item-index text-center uppercase tracking-tighter">${index + 1}</td>
            <td class="p-0">
                <div class="flex items-center h-full">
                    <div class="flex-grow">${labelEl}</div>
                    ${deleteBtn}
                </div>
            </td>
            ${cell('req', 'yes', req)}
            ${cell('req', 'no', req)}
            ${cell('out', 'ok', out)}
            ${cell('out', 'nok', out)}
            ${cell('out', 'na', out)}
            ${cell('out', 'more', out, itemObj.more)}
```

${itemObj.remarks || ''} `; };

```
let currentIndex = 0;
checklistItems.forEach(item => {
    html += renderRow(item, currentIndex++);
});

Object.keys(data).forEach(key => {
    if (key.startsWith('custom_')) {
        html += renderRow({ id: key, isCustom: true }, currentIndex++);
    }
});

container.innerHTML = html;
```

};

window.adminApp.addCustomContractReviewItem = () => { const tbody = document.getElementById('cr-excel-checklist'); if (!tbody) return;

```
const rowCount = tbody.querySelectorAll('.cr-item-row').length;
const newId = 'custom_' + Date.now();

const labelEl = `< input type = "text" class="cr-master-input cr-custom-label w-full font-bold
px-3 py-2 text-slate-700" value = "" data - item - id="${newId}" > `;

const cell = (type, val) => {
    return `
< td class="p-0 select-none" onclick = "window.adminApp.setReviewItem('${newId}', '${type}',
'${val}')" >
    <div class="outcome-tile" data-opt="${val}">
        <span class="cr-tick">o</span>
    </div>
        </td >
`;
};

const deleteBtn = `< button class="cr-item-delete-btn text-red-400 hover:text-red-600 px-3
flex-shrink-0" onclick = "window.adminApp.removeReviewItem('${newId}')" title = "Remove Item"
> <svg width="14" height="14" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path
stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 7l-.867 12.142A2 2 0
0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4
7h16" /></svg></button > `;

const tr = document.createElement('tr');
tr.className = 'cr-item-row';
tr.dataset.itemId = newId;

tr.innerHTML = `
< td class="cr-item-index text-center uppercase tracking-tighter" > ${rowCount + 1}</td >
    <td class="p-0">
        <div class="flex items-center h-full">
            <div class="flex-grow">${labelEl}</div>
            ${deleteBtn}
```

```
        </div>
    </td>
    ${cell('req', 'yes')}
    ${cell('req', 'no')}
    ${cell('out', 'ok')}
    ${cell('out', 'nok')}
    ${cell('out', 'na')}
    ${cell('out', 'more')}
```

`; tr.dataset.moreVal = 'false';

```
tbody.appendChild(tr);
```

};

window.adminApp.removeReviewItem = (itemId) => { const row = document.querySelector(`.cr - item - row[data - item - id="${itemId}"]`); if (row) { row.remove(); // Update indices const indices = document.querySelectorAll('.cr-item-index'); indices.forEach((td, idx) => { td.textContent = idx + 1; }); } };

window.adminApp.setReviewItem = (itemId, type, val) => { const row = document.querySelector(`.cr - item - row[data - item - id="${itemId}"]`); if (!row) return;

```
if (type === 'out' && val === 'more') {
    // Toggle 'more' (Clarity) independently
    const cellTd = row.children[7]; // 8th column (index 7) is 'more'
    const tile = cellTd?.querySelector('.outcome-tile');
    if (!tile) return;

    const tickEl = tile.querySelector('.cr-tick');
    const isActive = row.dataset.moreVal === 'true';

    if (isActive) {
        tile.classList.remove('active-more');
        if (tickEl) tickEl.textContent = 'o';
        row.dataset.moreVal = 'false';
    } else {
        tile.classList.add('active-more');
        if (tickEl) tickEl.textContent = '✓';
        row.dataset.moreVal = 'true';
    }
    return;
}

const targetIdx = type === 'req' ? 2 : 4;
const optionsCount = type === 'req' ? 2 : 3; // For 'out', only OK, NOK, NA (indices 4, 5, 6)

for (let i = 0; i < optionsCount; i++) {
    const cellTd = row.children[targetIdx + i];
    const tile = cellTd.querySelector('.outcome-tile');
    if (!tile) continue;

    const opt = tile.dataset.opt;
```

```
    const tickEl = tile.querySelector('.cr-tick');

    if (opt === val) {
        const isCurrentlyActive = row.dataset[`${type} Val`] === val;
        if (isCurrentlyActive) {
            // Toggle OFF
            tile.classList.remove(`active - ${val} `);
            if (tickEl) tickEl.textContent = 'o';
            row.dataset[`${type} Val`] = '';
        } else {
            // Switch ON
            tile.classList.add(`active - ${val} `);
            if (tickEl) tickEl.textContent = '✓';
            row.dataset[`${type} Val`] = val;
        }
    } else {
        // Force others OFF
        tile.classList.remove(`active - ${opt} `);
        if (tickEl) tickEl.textContent = 'o';
    }
}
```

};

window.adminApp.loadContractReview = async (projectId) => { const project = currentProjects.find(p => p.id === projectId); if (!project) return;

```
const status = document.getElementById('cr-save-status');
if (status) status.textContent = 'Loading...';

try {
    // Load from DB instead of project object!
    const result = await DB.getContractReview(projectId);
    const reviewData = result.data || {};

    // Helper to safely set value
    const setVal = (id, val) => {
        const el = document.getElementById(id) || document.querySelector(`[data - field=
"${id}"]`);
        if (el) el.value = val || '';
    };

    // Auto-fill defaults
    const reviewNo = reviewData.reviewNo || `CR - ${project.projectId || projectId} `;
    const today = new Date().toISOString().split('T')[0];

    // Find matching internal order for auto-population
    const allOrders = window.adminApp.getCurrentOrders ? window.adminApp.getCurrentOrders() :
[];
    const matchingOrder = allOrders.find(o => o.internalOrderNo === project.projectId);

    // Customer Data — pull from saved review, then matching internal order, then project
```

```
    setVal('cr-po-no', reviewData.poNo || (matchingOrder ? matchingOrder.poNo : '') ||
project.poNumber || '');
    setVal('cr-drawing-no', reviewData.drawingNo || (matchingOrder ? matchingOrder.drawingNo :
'') || project.drawingNo || '');
    setVal('cr-date', reviewData.date || today);
    setVal('cr-delivery-date', reviewData.deliveryDate || project.expectedCompletion || '');
    setVal('cr-contact-person', reviewData.contactPerson || '');
    setVal('cr-phone', reviewData.phone || '');

    // Internal Order Data
    setVal('cr-review-no', reviewNo);
    setVal('cr-io-number', project.projectId || '');
    setVal('cr-internal-date', reviewData.internalDate || today);
    setVal('cr-accountability', reviewData.accountability || '');
    setVal('cr-team', reviewData.team || '');
    setVal('cr-team-leader', reviewData.teamLeader || '');
    setVal('cr-members', reviewData.members || '');

    // Instructions
    setVal('cr-important-instructions', reviewData.instructions ||
reviewData.importantInstructions || '');

    // 6M Grid
    const mGrid = ['matl', 'machine', 'man', 'method', 'measure', 'tools'];
    mGrid.forEach(key => {
        const data = reviewData.sixM?.[key] || { cmt: '' };
        setVal(`cmt - ${key} `, data.cmt);
    });

    // Decisions
    setVal('cr-decision-cap', reviewData.decisionCap || '');
    setVal('cr-decision-oa', reviewData.decisionOa || '');
    setVal('cr-prepared-by', reviewData.preparedBy || '');
    setVal('cr-reviewed-by', reviewData.reviewedBy || '');
    setVal('cr-approved-by', reviewData.approvedBy || '');

    // Render Checklist Table
    const checklistItems = reviewData.items || reviewData.checklistDynamic || {};
    window.adminApp.renderContractReview(checklistItems);

    // Initialize Search Components
    setupMemberSearch('cr-search-accountability', 'search-input-inline', 'cr-accountability');
    setupMemberSearch('cr-search-team-leader', 'search-input-inline', 'cr-team-leader');
    setupMemberSearch('cr-search-members', 'search-input-inline', 'cr-members');
    setupMemberSearch('cr-search-prepared', 'search-input-inline', 'cr-prepared-by');
    setupMemberSearch('cr-search-reviewed', 'search-input-inline', 'cr-reviewed-by');
    setupMemberSearch('cr-search-approved', 'search-input-inline', 'cr-approved-by');

    if (status) {
        status.textContent = result.data ? '✓ Loaded' : '';
        setTimeout(() => { if (status) status.textContent = ''; }, 3000);
    }
```

```
} catch (e) {
    console.error("Error loading contract review:", e);
    if (status) status.textContent = 'Error loading';
}
```

};

window.adminApp.saveContractReview = async (action) => { const projectId = document.getElementById('detail-project-id')?.textContent; const project = currentProjects.find(p => p.projectId === projectId); if (!project) return; const projectDocId = project.id;

```
// Helper to get value
const getVal = (id) => {
    const el = document.getElementById(id) || document.querySelector(`[data - field=
"${id}"]`);
    return el ? el.value : '';
};

const reviewData = {
    status: action === 'finalize' ? 'Finalized' : 'Draft',
    updatedAt: new Date().toISOString(),

    // Customer Data
    poNo: getVal('cr-po-no'),
    drawingNo: getVal('cr-drawing-no'),
    date: getVal('cr-date'),
    deliveryDate: getVal('cr-delivery-date'),
    contactPerson: getVal('cr-contact-person'),
    phone: getVal('cr-phone'),

    // Internal Order Data
    reviewNo: getVal('cr-review-no'),
    internalDate: getVal('cr-internal-date'),
    accountability: getVal('cr-accountability'),
    team: getVal('cr-team'),
    teamLeader: getVal('cr-team-leader'),
    members: getVal('cr-members'),

    // Instructions
    instructions: getVal('cr-important-instructions'),
    importantInstructions: getVal('cr-important-instructions'),

    // 6M Grid
    sixM: {},

    // Decisions
    decisionCap: getVal('cr-decision-cap'),
    decisionOa: getVal('cr-decision-oa'),
    preparedBy: getVal('cr-prepared-by'),
    reviewedBy: getVal('cr-reviewed-by'),
    approvedBy: getVal('cr-approved-by'),
```

```
    items: {},
    checklistDynamic: {}
};

const mGrid = ['matl', 'machine', 'man', 'method', 'measure', 'tools'];
mGrid.forEach(key => {
    reviewData.sixM[key] = {
        cmt: getVal(`cmt - ${key} `)
    };
});

// Gather Checklist Items
const rows = document.querySelectorAll('.cr-item-row');
rows.forEach(row => {
    const id = row.dataset.itemId;
    const customInput = row.querySelector('.cr-custom-label');
    const customLabel = customInput ? customInput.value : '';
    const remarks = row.querySelector('.cr-remarks-input')?.value || '';

    const itemData = {
        req: row.dataset.reqVal || '',
        out: row.dataset.outVal || '',
        more: row.dataset.moreVal || 'false',
        remarks: remarks,
        customLabel: customLabel
    };

    reviewData.items[id] = itemData;
    reviewData.checklistDynamic[id] = itemData; // Backward compatibility
});

const statusSpan = document.getElementById('cr-save-status');
if (statusSpan) statusSpan.textContent = 'Saving...';

try {
    const isFiled = action === 'finalize';

    // Save to the actual subcollection
    await DB.saveContractReview(projectDocId, reviewData);

    // Update project with filed status
    await DB.updateProject(projectDocId, {
        contractFiled: isFiled
    }, 'Contract Review ' + (isFiled ? 'Finalized' : 'Draft Saved'));

    if (statusSpan) {
        statusSpan.textContent = `Saved at ${new Date().toLocaleTimeString()} `;
        setTimeout(() => statusSpan.textContent = '', 3000);
    }
    if (isFiled) {
        alert('Contract Review Finalized!');
        window.adminApp.moveProject(projectDocId, 'Planning');
```

```
    }
} catch (e) {
    console.error(e);
    if (statusSpan) statusSpan.textContent = 'Error saving!';
    alert('Failed to save review: ' + e.message);
}
```

};

// Initialize view on load document.addEventListener('DOMContentLoaded', () => { // Close status menu on click outside document.addEventListener('click', (e) => { const menu = document.getElementById('dd-status-menu'); const badge = document.getElementById('detail-project-status'); if (menu && !menu.classList.contains('hidden')) { if (!menu.contains(e.target) && !badge.contains(e.target)) { menu.classList.add('hidden'); } } });

```
setTimeout(() => {
    const savedMode = localStorage.getItem('projectViewMode') || 'grid';
    window.adminApp.setProjectView(savedMode);

    // Handle initial view logic
    const activeView = UI.getActiveView();
    if (activeView === 'inventory_management') {
        console.log("Auto-initializing inventory on load");
        window.adminApp.initInventory();
    }

    // Initialize Monitoring logic
    Monitoring.setupCostCalculation();
}, 100);
```

}); \n `\n\n\n### File: e:\re\Innovative Engineering Solutions\assets\admin\css\pm-theme.css\n*Description: Admin Styles (Subsets)*\n\n` css\n/* == PROJECT MANAGEMENT MODULE — THEME OVERRIDE == */

:root { --pm-bg: #f8fafc; --pm-card-bg: #ffffff; --pm-text-main: #0f172a; --pm-text-muted: #64748b; --pm-text-light: #94a3b8; --pm-border: #e2e8f0; --pm-border-hover: #cbd5e1; --pm-card-hover: #f1f5f9; --pm-shadow: 0 4px 12px rgba(0, 0, 0, 0.04); --pm-shadow-hover: 0 8px 24px rgba(0, 0, 0, 0.08);

```
/* Vibrant accents */
--pm-accent-primary: #10b981;
/* Emerald */
--pm-accent-secondary: #0ea5e9;
/* Sky */
--pm-accent-gradient: linear-gradient(135deg, #10b981, #0ea5e9);
```

}

/* == APPLY VARIABLES == */

@media screen {

```
/* Global Section Background */
#view-project_management {
    background-color: var(--pm-bg);
    min-height: 100%;
}

/* When viewing deep dive, the main background matches */
#view-project_detail {
    background-color: var(--pm-bg) !important;
    color: var(--pm-text-main) !important;
}

#view-project_management .pm-header,
#view-project_detail .dd-header {
    border-color: var(--pm-border);
}

#view-project_management .pm-header-title h2,
#view-project_detail .dd-project-name {
    color: var(--pm-text-main);
}

/* Stats Ribbon & Custom Stats */
#view-project_management .pm-stat-card {
    background-color: var(--pm-card-bg);
    border-color: var(--pm-border);
}

#view-project_management .pm-stat-card:hover {
    box-shadow: var(--pm-shadow-hover);
    transform: translateY(-2px);
    background-color: var(--pm-card-hover);
}

#view-project_management .pm-stat-value {
    color: var(--pm-text-main);
}
```

}

/* Enhancing default standard colorful UI */ .pm-stat-card { border-radius: 16px; background: var(--pm-card-bg); border: 1px solid var(--pm-border); transition: all 300ms cubic-bezier(0.4, 0, 0.2, 1); }

.pm-stat-card:hover { box-shadow: var(--pm-shadow-hover); transform: translateY(-3px); border-color: var(--pm-border-hover); background: var(--pm-card-hover); }

/* Premium Card Enhancements */ .pm-card { border-radius: 16px; box-shadow: var(--pm-shadow); transition: all 300ms cubic-bezier(0.4, 0, 0.2, 1); }

.pm-card:hover { box-shadow: var(--pm-shadow-hover); transform: translateY(-4px) scale(1.01); }

/* == PRINT STYLES FOR PROJECT MANAGEMENT == */ @media print {

```
/* Force light theme colors */
#view-project_management,
#cr-excel-checklist thead th {
    background: transparent !important;
    color: white !important;
    text-transform: uppercase;
    font-size: 0.725rem;
    letter-spacing: 0.1em;
    padding: 1.15rem 0.5rem !important;
    text-align: center !important;
    border-color: rgba(255, 255, 255, 0.15) !important;
    border-top: 1px solid rgba(255, 255, 255, 0.1);
}

#view-project_management,
#view-project_detail {
    background-color: white !important;
    color: black !important;
}

#view-project_management *:not(.cr-emerald-bg):not(.cr-emerald-header),
#view-project_detail *:not(.cr-emerald-bg):not(.cr-emerald-header) {
    background-color: transparent !important;
    color: black !important;
    border-color: #cbd5e1 !important;
    box-shadow: none !important;
}

/* Keep icons/badges slightly colored but mostly printable */
.pm-card-accent {
    display: none !important;
}

.theme-toggle-btn,
.pm-header-actions button,
#sidebar {
    display: none !important;
}
```

}\n \n\n\n### File: e:\re\Innovative Engineering Solutions\admin.html\n*Description: Admin Dashboard Layout Structure*\n\n html\n

```
<!-- Fonts -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
    href="https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700&family=Poppins:wght@600;700;800&display=swap"
    rel="stylesheet">

<!-- Tailwind CSS (Local) -->
```

```
<link rel="stylesheet" href="assets/admin/css/tailwind.css">

<!-- D3.js for Charts -->
<script src="https://d3js.org/d3.v7.min.js"></script>

<!-- PDF Export Library -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf-
autotable/3.5.31/jspdf.plugin.autotable.min.js"></script>

<script src="https://www.gstatic.com/firebasejs/10.7.1/firebase-app.js" type="module">
</script>

<!-- Admin Styles -->
<link rel="stylesheet" href="assets/admin/css/admin.css?v=1">
<link rel="stylesheet" href="assets/admin/css/delivery-modal.css">
<link rel="stylesheet" href="assets/admin/css/project.css">
<link rel="stylesheet" href="assets/admin/css/pm-theme.css">
<style>
    /* Modal Redesign - 2 Column Layout */
    .modal-wide {
        max-width: 1000px !important;
    }

    .modal-body.modal-2col-layout {
        display: grid !important;
        grid-template-columns: 1fr 1fr !important;
        gap: 2rem !important;
        align-items: start !important;
    }

    .modal-col-left,
    .modal-col-right {
        display: flex;
        flex-direction: column;
        gap: 0.8rem;
        /* Reduced from 1.5rem */
        min-width: 0;
    }

    /* Close Button Placement */
    .modal-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

    .modal-close {
        background: none;
        border: none;
        cursor: pointer;
        color: #fff;
```

```
}

.modal-close svg {
    width: 24px;
    height: 24px;
}

@media (max-width: 768px) {
    .modal-body.modal-2col-layout {
        grid-template-columns: 1fr !important;
    }
}

/* Premium Production Cost Card */
/* Polished Modal Inputs */
.form-input {
    height: 42px;
    /* Standardize height */
    border: 1px solid #cbd5e1;
    border-radius: 6px;
    padding: 0 1rem;
    transition: all 0.2s;
}

.form-input:focus {
    border-color: #6366f1;
    box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.1);
}

/* Subtle Production Cost Group */
.production-cost-group {
    background-color: #f8fafc;
    border-radius: 8px;
    padding: 0.75rem;
    /* Reduced from 1rem */
    margin-top: 0.5rem;
    /* Reduced from 0.75rem */
    border: 1px dashed #cbd5e1;
}

.production-cost-label {
    font-size: 0.75rem;
    font-weight: 600;
    color: #64748b;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    margin-bottom: 0.4rem;
    /* Reduced from 0.75rem */
}

/* Checkbox Grid Polish */
.avail-checks {
```

```
        display: block !important;
        background-color: #f8fafc;
        padding: 0.75rem;
        /* Reduced from 1rem */
        margin-top: 0.5rem;
        /* Reduced from 0.75rem */
        border: 1px dashed #cbd5e1;
        /* Match Production Cost Group */
    }

    .check-item {
        display: flex;
        align-items: center;
        gap: 0.5rem;
        font-size: 0.875rem;
        color: #475569;
        cursor: pointer;
    }

    .check-item input {
        width: 1.125rem;
        height: 1.125rem;
        accent-color: #10b981;
    }

    /* Section Spacing */
    .inline-section-label {
        font-size: 0.75rem;
        font-weight: 700;
        color: #94a3b8;
        text-transform: uppercase;
        letter-spacing: 0.1em;
        margin-bottom: 0.4rem;
        /* Reduced from 0.5rem */
        margin-top: 0.6rem;
        /* Reduced from 0.75rem */
        display: flex;
        align-items: center;
    }

    .inline-section-label::after {
        content: '';
        flex: 1;
        height: 1px;
        background: #e2e8f0;
        margin-left: 0.75rem;
    }

    /* Remove top margin for first section */
    .modal-col-left .w-full:first-child .inline-section-label,
    .modal-col-right .w-full:first-child .inline-section-label {
        margin-top: 0;
```

```
}

/* Member Search Component */
.member-search-container {
    position: relative;
    width: 100%;
}

.search-tags-container {
    display: flex;
    flex-wrap: wrap;
    gap: 0.4rem;
    padding: 0.5rem;
    border: 1px solid #cbd5e1;
    border-radius: 6px;
    background: #fff;
    min-height: 42px;
    cursor: text;
    transition: all 0.2s;
}

.search-tags-container:focus-within {
    border-color: #6366f1;
    box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.1);
}

.member-tag {
    display: inline-flex;
    align-items: center;
    gap: 0.25rem;
    background: #f1f5f9;
    color: #475569;
    padding: 0.15rem 0.5rem;
    border-radius: 4px;
    font-size: 0.75rem;
    font-weight: 500;
    border: 1px solid #e2e8f0;
}

.member-tag-remove {
    cursor: pointer;
    color: #94a3b8;
    font-size: 1rem;
    line-height: 1;
}

.member-tag-remove:hover {
    color: #ef4444;
}

.search-input-inline {
    flex: 1;
```

```
        min-width: 120px;
        border: none;
        outline: none;
        font-size: 0.875rem;
        padding: 0.2rem 0;
        background: transparent;
    }

    .search-suggestions-dropdown {
        position: absolute;
        top: 100%;
        left: 0;
        right: 0;
        background: #fff;
        border: 1px solid #e2e8f0;
        border-top: none;
        border-radius: 0 0 6px 6px;
        box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
        max-height: 200px;
        overflow-y: auto;
        z-index: 50;
    }

    .suggestion-item {
        padding: 0.6rem 0.8rem;
        font-size: 0.875rem;
        color: #475569;
        cursor: pointer;
        transition: background 0.15s;
    }

    .suggestion-item:hover {
        background: #f8fafc;
    }

    .suggestion-item.selected {
        background: #f1f5f9;
        color: #6366f1;
        font-weight: 500;
    }
</style>
```

```
<!-- ============================================
     AUTH CONTAINER (Login Page)
     ============================================ -->
<div id="auth-container">
    <div class="login-card">
        <img src="assets/logo.png" alt="Logo" class="login-logo">
        <h1 class="login-title">Admin Portal</h1>
        <p class="login-subtitle">Sign in to access the dashboard</p>

        <form id="login-form">
```

```
            <div class="form-group">
                <label class="form-label" for="email-address">Email Address</label>
                <input type="email" id="email-address" name="email" class="form-input"
                    placeholder="admin@company.com" required autocomplete="email">
            </div>
            <div class="form-group">
                <label class="form-label" for="password">Password</label>
                <input type="password" id="password" name="password" class="form-input"
                    placeholder="Enter your password" required autocomplete="current-
password">
            </div>
            <div class="form-group" style="display: flex; align-items: center; gap: 0.5rem;">
                <input type="checkbox" id="remember-me" name="remember-me"
                    style="width: 16px; height: 16px; accent-color: #10b981;">
                <label for="remember-me" style="font-size: 0.875rem; color: #475569;">Remember
me</label>
            </div>
            <button type="submit" class="btn btn-primary" style="width: 100%; margin-top:
0.5rem;">
                <svg width="18" height="18" fill="none" stroke="currentColor" viewBox="0 0 24
24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                        d="M11 16l-4-4m0 0l4-4m-4 4h14m-5 4v1a3 3 0 01-3 3H6a3 3 0 01-3-3V7a3
3 0 013-3h7a3 3 0 013 3v1" />
                </svg>
                Sign In
            </button>
            <div id="login-error" class="login-error hidden"></div>
        </form>
    </div>
</div>

<!-- ============================================
     DASHBOARD CONTAINER
     ============================================ -->
<div id="dashboard-container" class="hidden">

    <!-- Sidebar -->
    <aside id="sidebar">
        <div class="sidebar-header">
            <span class="brand-text">IES Groups<br><small>Admin Portal</small></span>
            <button id="sidebar-toggle" class="sidebar-toggle"
                onclick="document.getElementById('dashboard-
container').classList.toggle('sidebar-collapsed')"
                title="Toggle Sidebar">
                <svg width="18" height="18" fill="none" stroke="currentColor" viewBox="0 0 24
24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M15 19l-7-7 7-7" />
                </svg>
            </button>
        </div>
```

```
        <nav class="sidebar-nav">
            <div class="sidebar-group">
                <div class="sidebar-group-header"
onclick="window.adminApp.toggleSidebarGroup(this)">
                    <span>General</span>
                    <svg class="group-toggle-icon" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 9l-7 7-7-7" />
                    </svg>
                </div>
                <ul class="nav-list">
                    <li class="nav-item">
                        <a href="#" class="nav-link active" data-view="overview">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M4 6a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2H6a2 2 0
01-2-2V6zM14 6a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2h-2a2 2 0 01-2-2V6zM4 16a2 2 0 012-
2h2a2 2 0 012 2v2a2 2 0 01-2 2H6a2 2 0 01-2-2v-2zM14 16a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2
2h-2a2 2 0 01-2-2v-2z" />
                            </svg>
                            Dashboard
                        </a>
                    </li>
                </ul>
            </div>

            <div class="sidebar-group">
                <div class="sidebar-group-header"
onclick="window.adminApp.toggleSidebarGroup(this)">
                    <span>Operations</span>
                    <svg class="group-toggle-icon" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 9l-7 7-7-7" />
                    </svg>
                </div>
                <ul class="nav-list">
                    <li class="nav-item">
                        <a href="#" class="nav-link" data-view="monitoring">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M9 5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2V7a2 2
0 00-2-2h-2M9 5a2 2 0 002 2h2a2 2 0 002-2M9 5a2 2 0 012-2h2a2 2 0 012 2m-3 7h3m-3 4h3m-6-
4h.01M9 16h.01" />
                            </svg>
                            Internal Orders
                        </a>
                    </li>
```

```
                    <li class="nav-item">
                        <a href="#" class="nav-link" data-view="pending_assignment">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M9 5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2V7a2 2
0 00-2-2h-2M9 5a2 2 0 002 2h2a2 2 0 002-2M9 5a2 2 0 012-2h2a2 2 0 012 2m-6 9l2 2 4-4" />
                            </svg>
                            Pending Assignment
                        </a>
                    </li>
                    <li class="nav-item">
                        <a href="#" class="nav-link" data-view="daily_roster">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 002-2V7a2 2 0 00-2-
2H5a2 2 0 00-2 2v12a2 2 0 002 2z" />
                            </svg>
                            Daily Roster
                        </a>
                    </li>
                    <li class="nav-item">
                        <a href="#" class="nav-link" data-view="delivery_report">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M9 17v-2m3 2v-4m3 4v-6m2 10H7a2 2 0 01-2-2V5a2 2 0 012-
2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z" />
                            </svg>
                            Delivery Report
                        </a>
                    </li>
                </ul>
            </div>

            <div class="sidebar-group">
                <div class="sidebar-group-header"
onclick="window.adminApp.toggleSidebarGroup(this)">
                    <span>Planning & Assets</span>
                    <svg class="group-toggle-icon" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 9l-7 7-7-7" />
                    </svg>
                </div>
                <ul class="nav-list">
                    <li class="nav-item">
                        <a href="#" class="nav-link" data-view="project_management">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
```

```
                                    d="M19 11H5m14 0a2 2 0 012 2v6a2 2 0 01-2 2H5a2 2 0 01-2-
2v-6a2 2 0 012-2m14 0V9a2 2 0 00-2-2M5 11V9a2 2 0 012-2m0 0V5a2 2 0 012-2h6a2 2 0 012 2v2M7
7h10" />
                            </svg>
                            Project Management
                        </a>
                    </li>
                    <li class="nav-item">
                        <a href="#" class="nav-link" data-view="inventory_management">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M20 7l-8-4-8 4m16 0l-8 4m8-4v10l-8 4m0-10L4 7m8 4v10M4
7v10l8 4" />
                            </svg>
                            Inventory Management
                        </a>
                    </li>
                </ul>
            </div>

            <div class="sidebar-group">
                <div class="sidebar-group-header"
onclick="window.adminApp.toggleSidebarGroup(this)">
                    <span>Management</span>
                    <svg class="group-toggle-icon" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 9l-7 7-7-7" />
                    </svg>
                </div>
                <ul class="nav-list">
                    <li class="nav-item">
                        <a href="#" class="nav-link" data-view="team_org">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 4.354a4 4 0 110 5.292M15 21H3v-1a6 6 0 0112 0v1zm0
0h6v-1a6 6 0 00-9-5.197M13 7a4 4 0 11-8 0 4 4 0 018 0z" />
                            </svg>
                            Team &amp;amp; Organization
                        </a>
                    </li>
                    <li class="nav-item">
                        <a href="#" class="nav-link" data-view="reports">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M9 12h6m-6 4h6m2 5H7a2 2 0 01-2-2V5a2 2 0 012-2h5.586a1
1 0 01.707.293l5.414 5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z" />
                            </svg>
                            Daily Reports
```

```
                        </a>
                    </li>
                </ul>
            </div>
        </nav>

        <div class="sidebar-footer">
            <div class="user-info">
                <div class="user-avatar">A</div>
                <div class="user-details">
                    <span class="user-label">Signed in as</span>
                    <span class="user-email" id="user-email-display">admin@company.com</span>
                </div>
            </div>
            <button id="logout-btn">
                <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                        d="M17 16l4-4m0 0l-4-4m4 4H7m6 4v1a3 3 0 01-3 3H6a3 3 0 01-3-3V7a3 3 0
013-3h4a3 3 0 013 3v1" />
                </svg>
                Sign Out
            </button>
        </div>
    </aside>

    <!-- Main Content Wrapper -->
    <div class="main-wrapper">

        <main id="main-content">

            <!-- ==========================================
                 VIEW: Dashboard Overview
                 ========================================== -->
            <div id="view-overview" class="view-section">
                <div class="dashboard-header-row mb-6">
                    <div class="dashboard-title">
                        <h2 class="text-2xl font-bold text-slate-800">Executive Overview</h2>
                        <p class="text-sm text-slate-500">Real-time production & revenue
insights</p>
                    </div>
                    <div class="dashboard-filter-bar flex items-center gap-4 p-1.5 rounded-
full ml-auto">
                        <div class="flex items-center gap-2 pl-3">
                            <span
                                class="text-[10px] font-bold text-slate-400 uppercase
tracking-widest text-slate-400">Filter
                                by Dept</span>
                            <select id="dashboard-dept-filter"
                                class="bg-white border-0 text-xs font-bold text-slate-700
rounded-full px-4 py-1.5 ring-1 ring-slate-200 focus:ring-2 focus:ring-teal-500 transition-all
cursor-pointer outline-none shadow-sm"
                                onchange="window.adminApp.refreshDashboard()">
```

```
                                <option value="all">All Departments</option>
                                <option value="Fab">Fabrication</option>
                                <option value="CNC">CNC</option>
                                <option value="VMC">VMC</option>
                                <option value="Turning">Turning</option>
                                <option value="Assembly">Assembly</option>
                            </select>
                        </div>
                        <div class="h-6 w-px bg-slate-200"></div>
                        <div class="flex items-center gap-2 pr-1">
                            <span
                                class="text-[10px] font-bold text-slate-400 uppercase
tracking-widest text-slate-400">Time
                                Period</span>
                            <select id="dashboard-month-filter"
                                class="bg-white border-0 text-xs font-bold text-slate-700
rounded-full px-4 py-1.5 ring-1 ring-slate-200 focus:ring-2 focus:ring-teal-500 transition-all
cursor-pointer outline-none shadow-sm"
                                onchange="window.adminApp.refreshDashboard()">
                                <option value="all">All Time</option>
                                <option value="2026-02">February 2026</option>
                                <option value="2026-01">January 2026</option>
                                <option value="2025-12">December 2025</option>
                            </select>
                        </div>
                    </div>
                </div>

                <div class="stats-grid">
                    <!-- KPI Card 1: Revenue -->
                    <div class="stat-card blue">
                        <div class="stat-icon blue">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 8c-1.657 0-3 .895-3 2s1.343 2 3 2 3 .895 3 2-1.343
2-3 2m0-8c1.11 0 2.08.402 2.599 1M12 8V7m0 1v8m0 0v1m0-1c-1.11 0-2.08-.402-2.599-1M21 12a9 9 0
11-18 0 9 9 0 0118 0z" />
                            </svg>
                        </div>
                        <div class="stat-content">
                            <h3>
                                Pending Order Value
                                <!-- Inline Privacy Toggle -->
                                <button class="privacy-toggle" onclick="toggleRevenue()"
title="Toggle Visibility">
                                    <svg id="eye-icon" width="16" height="16" fill="none"
stroke="currentColor"
                                        viewBox="0 0 24 24">
                                        <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
                                            d="M15 12a3 3 0 11-6 0 3 3 0 016 0z" />
```

```
                                        <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
                                            d="M2.458 12C3.732 7.943 7.523 5 12 5c4.478 0
8.268 2.943 9.542 7-1.274 4.057-5.064 7-9.542 7-4.477 0-8.268-2.943-9.542-7z" />
                                    </svg>
                                    <svg id="eye-off-icon" class="hidden" width="16"
height="16" fill="none"
                                        stroke="currentColor" viewBox="0 0 24 24">
                                        <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
                                            d="M13.875 18.825A10.05 10.05 0 0112 19c-4.478 0-
8.268-2.943-9.543-7a9.97 9.97 0 011.563-3.029m5.858.908a3 3 0 114.243 4.243M9.878 9.878l4.242
4.242M9.88 9.88l-3.29-3.29m7.532 7.532l3.29 3.29M3 3l3.59 3.59m0 0A9.953 9.953 0 0112 5c4.478
0 8.268 2.943 9.543 7a10.025 10.025 0 01-4.132 5.411m0 0L21 21" />
                                    </svg>
                                </button>
                            </h3>
                            <p id="stat-revenue" class="revenue-hidden">₹ 4,55,453</p>
                            <div class="stat-trend up">
                                <svg width="12" height="12" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                    <path d="M5 15l7-7 7 7"></path>
                                </svg>
                                <span>Active Portfolio</span>
                            </div>
                        </div>
                    </div>

                    <!-- Inline Script for Privacy Toggle -->
                    <script>
                        function toggleRevenue() {
                            const revEl = document.getElementById('stat-revenue');
                            const eye = document.getElementById('eye-icon');
                            const eyeOff = document.getElementById('eye-off-icon');

                            if (revEl.classList.contains('revenue-hidden')) {
                                revEl.classList.remove('revenue-hidden');
                                revEl.classList.add('revenue-visible');
                                eye.classList.add('hidden');
                                eyeOff.classList.remove('hidden');
                            } else {
                                revEl.classList.add('revenue-hidden');
                                revEl.classList.remove('revenue-visible');
                                eye.classList.remove('hidden');
                                eyeOff.classList.add('hidden');
                            }
                        }
                    </script>

                    <!-- KPI Card 2: Active Orders -->
                    <div class="stat-card emerald">
                        <div class="stat-icon emerald">
```

```
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M9 5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2V7a2 2
0 00-2-2h-2M9 5a2 2 0 002 2h2a2 2 0 002-2M9 5a2 2 0 012-2h2a2 2 0 012 2m-3 7h3m-3 4h3m-6-
4h.01M9 16h.01" />
                            </svg>
                        </div>
                        <div class="stat-content">
                            <h3>Active Orders</h3>
                            <p id="stat-active-orders">0</p>
                            <div class="stat-trend up">
                                <svg width="12" height="12" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                    <path d="M5 15l7-7 7 7"></path>
                                </svg>
                                <span id="stat-pending-count">0 Pending</span>
                            </div>
                        </div>
                    </div>

                    <!-- KPI Card 3: Unassigned -->
                    <div class="stat-card amber">
                        <div class="stat-icon amber">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667
1.732-3L13.732 4c-.77-1.333-2.694-1.333-3.464 0L3.34 16c-.77 1.333.192 3 1.732 3z" />
                            </svg>
                        </div>
                        <div class="stat-content">
                            <h3>Unassigned Orders</h3>
                            <p id="stat-unassigned">0</p>
                            <div class="stat-trend down">
                                <span>Requires Action</span>
                            </div>
                        </div>
                    </div>
                </div>

                <!-- Dashboard Layout Row (3 Columns) -->
                <div class="dashboard-row three-col-grid">

                    <!-- Column 1: Pending Orders List (Main) -->
                    <div class="dashboard-table-card col-main">
                        <div class="card-header">
                            <div class="card-title-group">
                                <h3>Pending Production Orders</h3>
                                <span class="count-badge" id="pending-orders-badge">0
Items</span>
                            </div>
```

```
                            <button class="btn-text"
onclick="window.adminApp.switchView('monitoring')">View
                                All</button>
                        </div>
                        <div class="table-responsive">
                            <table class="dashboard-table">
                                <thead>
                                    <tr>
                                        <th>IO No</th>
                                        <th>Customer</th>
                                        <th>Description</th>
                                        <th class="text-center">Drg No</th>
                                        <th>Qty</th>
                                        <th>Unit</th>
                                        <th>Delivery Date</th>
                                        <th>Status</th>
                                    </tr>
                                </thead>
                                <tbody id="dashboard-pending-body">
                                    <tr>
                                        <td colspan="5" class="text-center py-8 text-slate-
400">Loading orders...
                                        </td>
                                    </tr>
                                </tbody>
                            </table>
                        </div>
                    </div>

                    <!-- Column 2: Activity & Alerts (Middle) -->
                    <div class="flex flex-col gap-4 col-middle">
                        <!-- Production Pipeline Chart (Fixed Height) -->
                        <div class="card p-5">
                            <div class="mb-4">
                                <h3 style="font-size: 1.125rem; font-weight: 600;">Production
Pipeline</h3>
                                <p class="text-xs text-slate-500 mt-1">Status distribution of
all active orders</p>
                            </div>
                            <div id="production-pipeline-chart" class="w-full">
                                <!-- Production Pipeline Logic -->
                                <div class="flex flex-col gap-3">
                                    <div class="pipeline-item">
                                        <div class="flex justify-between text-xs mb-1">
                                            <span>Pending</span>
                                            <span id="pipeline-pending-pct">0%</span>
                                        </div>
                                        <div class="w-full h-2 bg-slate-100 rounded-full
overflow-hidden"
                                            style="background: #e2e8f0;">
                                            <div id="pipeline-pending-bar" class="h-full"
                                                style="width: 0%; background: #f59e0b;"></div>
```

```
                                    </div>
                                </div>
                                <div class="pipeline-item">
                                    <div class="flex justify-between text-xs mb-1">
                                        <span>Delivered</span>
                                        <span id="pipeline-delivered-pct">0%</span>
                                    </div>
                                    <div class="w-full h-2 bg-slate-100 rounded-full
overflow-hidden"
                                        style="background: #e2e8f0;">
                                        <div id="pipeline-delivered-bar" class="h-full"
                                            style="width: 0%; background: #10b981;"></div>
                                    </div>
                                </div>
                            </div>
                        </div>

                        <!-- Recently Added (Flex Grow) -->
                        <div class="card p-5 flex-1 flex flex-col min-h-0">
                            <div class="mb-4">
                                <h3 style="font-size: 1rem; font-weight: 600;">Recently
Added</h3>
                                <p class="text-xs text-slate-500 mt-1">Latest internal orders
created</p>
                            </div>
                            <ul class="activity-feed flex-1 overflow-y-auto" id="dashboard-
recent-feed">
                                <!-- Populated by JS -->
                                <li class="text-center py-4 text-xs text-slate-400">Loading
recent items...</li>
                            </ul>
                        </div>
                    </div>

                </div>
            </div>

            <!-- ============================================
                 VIEW: Internal Orders (Monitoring)
                 ============================================ -->
            <div id="view-monitoring" class="view-section hidden">
                <!-- Month Header Card -->
                <div class="month-header-card">
                    <div class="month-info">
                        <span class="month-label">Filter Period</span>
                        <div class="flex items-center gap-2">
                            <input type="month" id="order-month-from" class="month-picker"
title="From">
                            <span class="text-slate-400">to</span>
                            <input type="month" id="order-month-to" class="month-picker"
title="To">
```

```
                        </div>
                    </div>
                    <div class="header-divider"></div>
                    <div class="search-box">
                        <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z" />
                        </svg>
                        <input type="text" id="order-search-filter" class="search-input"
                            placeholder="Search by order no, customer, description...">
                    </div>
                    <div class="header-actions">
                        <button id="trash-toggle-btn" class="btn btn-ghost"
                            onclick="window.adminApp.toggleTrashMode()">
                            <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-
1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" />
                            </svg>
                            <span>Trash</span>
                        </button>
                        <button id="empty-trash-btn" class="btn btn-danger hidden"
                            onclick="window.adminApp.emptyTrash('internal')">
                            <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-
1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" />
                            </svg>
                            Empty Trash
                        </button>
                        <button class="btn btn-primary btn-lg"
onclick="window.adminApp.prepareAddOrder()">
                            <svg width="20" height="20" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 4v16m8-8H4" />
                            </svg>
                            Add Order
                        </button>
                        <button onclick="window.adminApp.exportToPDF()" class="btn btn-pdf
btn-lg flex items-center"
                            title="Export monthly report to PDF">
                            <svg class="w-5 h-5 mr-2" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
```

```
width="2"
                                    d="M7 21h10a2 2 0 002-2V9.414a1 1 0 00-.293-.707l-5.414-
5.414A1 1 0 0012.586 3H7a2 2 0 00-2 2v14a2 2 0 002 2z">
                                </path>
                            </svg>
                            Export PDF
                        </button>
                    </div>
                </div>

                <div class="table-container">
                    <div class="table-wrapper">
                        <table id="monitoring-table" class="excel-style-table">
                            <thead>
                                <!-- Row 1: Groups -->
                                <tr class="header-group-row">
                                    <th rowspan="2" width="50">S.No</th>
                                    <th colspan="2" class="text-center group-order">Internal
Order</th>
                                    <th colspan="2" class="text-center group-item">Item
Details</th>
                                    <th colspan="7" class="text-center group-pricing">Pricing
& Production</th>
                                    <th colspan="6" class="text-center group-
customer">Customer Data</th>
                                    <th colspan="5" class="text-center group-
delivery">Delivery date Actual</th>
                                    <th rowspan="2" width="80" class="text-right">Actions</th>
                                </tr>
                                <!-- Row 2: Columns -->
                                <tr class="header-column-row">
                                    <th class="sortable"
onclick="window.adminApp.sort('internalOrderNo')">IO No
                                    </th>
                                    <th class="sortable"
onclick="window.adminApp.sort('date')">Date</th>
                                    <th class="text-center">Drg No</th>
                                    <th style="min-width: 150px;">Description</th>
                                    <th class="text-right">Qty</th>
                                    <th>Unit</th>
                                    <th class="text-right">Sale Val</th>
                                    <th class="text-right">In-house</th>
                                    <th class="text-right">Outsource</th>
                                    <th class="text-center">Labor</th>
                                    <th class="text-right sortable"
onclick="window.adminApp.sort('total')">Total
                                    </th>

                                    <th class="sortable"
onclick="window.adminApp.sort('customer')">Customer</th>
                                    <th>PO No</th>
                                    <th>PO Date</th>
```

```
                                    <th title="Drawing Availability">Drg</th>
                                    <th title="Raw Material Availability">Raw</th>
                                    <th title="Finished Part Availability">Fin</th>

                                    <th>Del. Date</th>
                                    <th>DC No</th>
                                    <th class="text-right">Del. Qty</th>
                                    <th>Bill No</th>
                                    <th class="text-center sortable"
onclick="window.adminApp.sort('status')">Status
                                    </th>
                                </tr>
                            </thead>
                            <tbody id="monitoring-table-body">
                                <!-- Populated by JS -->
                            </tbody>
                        </table>
                    </div>

                    <!-- Pagination (REMOVED) -->
                    <div id="pagination-container" class="pagination-container hidden"
style="display: none;">
                        <!-- Kept hidden structure just in case JS references it, but content
cleared -->
                        <div class="pagination-info" id="pagination-info"
                            style="padding: 1rem; color: #64748b; font-size: 0.875rem;"></div>
                        <div class="pagination-controls" id="pagination-controls"></div>
                    </div>
                </div>
            </div>

            <!-- ============================================
                 VIEW: Team & Organization
                 ============================================ -->
            <div id="view-team_org" class="view-section hidden">
                <div class="page-header">
                    <div>
                        <h2>Team &amp;amp; Organization</h2>
                        <div class="tabs" style="margin-top: 1rem; margin-bottom: 0;">
                            <button id="tab-members" class="tab-btn active"

onclick="window.adminApp.switchTeamTab('members')">Members</button>
                            <button id="tab-hierarchy" class="tab-btn"

onclick="window.adminApp.switchTeamTab('hierarchy')">Organization Tree</button>
                        </div>
                    </div>
                    <button class="btn btn-primary"
onclick="window.adminApp.openAddMemberModal()">
                        <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-
```

```
width="2"
                                d="M18 9v3m0 0v3m0-3h3m-3 0h-3m-2-5a4 4 0 11-8 0 4 4 0 018
0zM3 20a6 6 0 0112 0v1H3v-1z" />
                        </svg>
                        Add Member
                    </button>
                </div>

                <!-- Sub-View: Member List -->
                <div id="subview-members" class="table-container">
                    <div class="table-wrapper">
                        <table>
                            <thead>
                                <tr>
                                    <th>Name</th>
                                    <th>Emp ID</th>
                                    <th>Role</th>
                                    <th>Section</th>
                                    <th>Phone</th>
                                    <th>Joining Date</th>
                                    <th>Status</th>
                                    <th class="text-right">Actions</th>
                                </tr>
                            </thead>
                            <tbody id="member-list-body">
                                <!-- Populated by JS -->
                            </tbody>
                        </table>
                    </div>
                </div>

                <!-- Sub-View: Hierarchy Tree -->
                <div id="subview-hierarchy" class="hidden">
                    <div class="card">
                        <div class="card-body">
                            <div id="hierarchy-container">
                                <!-- Org chart rendered by charts.js -->
                            </div>
                        </div>
                    </div>
                </div>
            </div>


            <!-- Hidden view for backward compatibility -->
            <div id="view-workflow" class="view-section hidden"></div>

            <!-- View: Pending Assignment -->
            <section id="view-pending_assignment" class="view-section hidden">
                <!-- Modern Header -->
                <div class="pa-header">
                    <div class="pa-header-left">
```

```
                        <div class="pa-title-group">
                            <h2 class="pa-title">Pending Orders</h2>
                            <span class="pa-subtitle">Assignment & Tracking</span>
                        </div>
                        <span id="pending-assignment-count" class="pa-count-badge">0
orders</span>
                    </div>
                    <div class="pa-header-right">
                        <button class="pa-btn-generate"
onclick="window.adminApp.generatePendingReport()">
                            <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M9 17v-2m3 2v-4m3 4v-6m2 10H7a2 2 0 01-2-2V5a2 2 0 012-
2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z" />
                            </svg>
                            Generate Report
                        </button>
                    </div>
                </div>

                <!-- Modern Filter Bar -->
                <div class="pa-filters">
                    <div class="pa-filter-group">
                        <label class="pa-filter-label">Department</label>
                        <select id="pending-filter-department" class="pa-filter-select">
                            <option value="">All Departments</option>
                            <option value="Fab">Fab</option>
                            <option value="CNC">CNC</option>
                            <option value="VMC">VMC</option>
                            <option value="Turning">Turning</option>
                            <option value="Assembly">Assembly</option>
                        </select>
                    </div>
                    <div class="pa-filter-group">
                        <label class="pa-filter-label">Assigned</label>
                        <select id="pending-filter-assigned" class="pa-filter-select">
                            <option value="">All</option>
                            <option value="assigned">Assigned</option>
                            <option value="unassigned">Unassigned</option>
                        </select>
                    </div>
                    <div class="pa-filter-group">
                        <label class="pa-filter-label">Priority</label>
                        <select id="pending-filter-priority" class="pa-filter-select">
                            <option value="">All</option>
                            <option value="urgent">Urgent Only</option>
                            <option value="normal">Normal Only</option>
                        </select>
                    </div>
                    <div class="pa-filter-group" style="margin-left: auto;">
```

```
                        <label class="pa-filter-label">Sort By</label>
                        <select id="pending-sort-by" class="pa-filter-select">
                            <option value="priority">Priority (Urgent First)</option>
                            <option value="dueDate">Due Date (Earliest)</option>
                            <option value="orderId">Order ID</option>
                        </select>
                    </div>
                </div>

                <!-- Modern Table Container -->
                <div class="pa-table-card">
                    <div class="pa-table-scroll">
                        <table class="pa-table" id="pending-assignment-table">
                            <thead>
                                <tr>
                                    <th style="width: 50px;"> ⚡ </th>
                                    <th>Order ID</th>
                                    <th>Customer</th>
                                    <th>Description</th>
                                    <th class="text-center">Drg No</th>
                                    <th style="width: 70px;">Qty</th>
                                    <th style="width: 60px;">Unit</th>
                                    <th style="width: 150px;">Due Date</th>
                                    <th style="min-width: 220px;">Assigned To</th>
                                    <th style="min-width: 200px;">Remarks</th>
                                </tr>
                            </thead>
                            <tbody id="pending-assignment-body">
                                <tr>
                                    <td colspan="10" class="text-center py-8 text-slate-
400">Loading pending
                                        orders...</td>
                                </tr>
                            </tbody>
                        </table>
                    </div>
                </div>
            </section>


            <!-- View: Daily Reports -->
            <section id="view-reports" class="view-section hidden">
                <!-- Header Row -->
                <div
                    class="flex items-center justify-between w-full bg-white p-4 rounded-xl
border border-slate-200 shadow-sm mb-4">
                    <div class="flex items-center gap-3">
                        <h2 style="font-size: 1.25rem; font-weight: 700; color: #1e293b;
margin: 0;">
                            📊 Daily Pending Orders Reports
                        </h2>
                        <span id="reports-auto-status" class="px-2 py-1 text-xs font-semibold
```

```
rounded-full"
                            style="background: #dcfce7; color: #166534;">
                            Auto-saves at 7 PM IST
                        </span>
                    </div>
                    <div class="flex items-center gap-3">
                        <button class="btn btn-primary"
onclick="window.adminApp.generateAndSaveReport()">
                            <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 4v16m8-8H4" />
                            </svg>
                            Generate Today's Report
                        </button>
                    </div>
                </div>

                <!-- Info Banner -->
                <div class="p-3 mb-4 rounded-lg" style="background: #f0f9ff; border: 1px solid
#bae6fd;">
                    <p class="text-xs" style="color: #0369a1; margin: 0;">
                        💡 Reports are automatically generated when you open the app after 7
PM IST (if not already
                        generated for that day).
                        You can also manually generate a report at any time using the button
above.
                    </p>
                </div>

                <!-- Reports List Table -->
                <div class="table-container" style="flex: 1; display: flex; flex-direction:
column; min-height: 0;">
                    <div class="table-scroll-wrapper" style="flex: 1; overflow: auto;">
                        <table class="monitoring-table" id="reports-table">
                            <thead>
                                <tr>
                                    <th style="width: 150px;">Date</th>
                                    <th style="width: 100px;">Total Orders</th>
                                    <th style="width: 100px;">Urgent</th>
                                    <th style="width: 100px;">Assigned</th>
                                    <th style="width: 100px;">Unassigned</th>
                                    <th>Generated At</th>
                                    <th style="width: 120px;">Actions</th>
                                </tr>
                            </thead>
                            <tbody id="reports-body">
                                <tr>
                                    <td colspan="7" class="text-center py-8 text-slate-
400">Loading reports...</td>
                                </tr>
```

```
                            </tbody>
                        </table>
                    </div>
                </div>
            </section>

            <!-- View: Inventory Management -->
            <section id="view-inventory_management" class="view-section hidden">
                <!-- Header Bar -->
                <div class="pm-header inventory-header">
                    <div class="pm-header-title">
                        <div style="display: flex; align-items: center; gap: 1rem;">
                            <h2>📦 Inventory Management</h2>
                            <span class="pm-header-badge inventory">Stock Control</span>
                        </div>
                    </div>
                    <div class="pm-header-actions">
                        <!-- MASTER ACTIONS -->
                        <div id="inventory-master-actions" class="flex gap-2">
                            <button id="inventory-trash-btn" class="pm-trash-toggle"
                                onclick="window.adminApp.toggleInventoryTrash()">
                                <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                    <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
                                        d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0
01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" />
                                </svg>
                                <span>Trash</span>
                            </button>
                            <button class="btn btn-primary"
onclick="window.adminApp.openAddInventoryModal()">
                                <svg width="16" height="16" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                    <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
                                        d="M12 4v16m8-8H4" />
                                </svg>
                                Add New Item
                            </button>
                        </div>

                        <!-- LEDGER ACTIONS -->
                        <div id="inventory-ledger-actions" class="flex gap-2 hidden">
                            <button class="btn btn-secondary"
onclick="window.adminApp.printInventoryLedger()">
                                <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                    <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
                                        d="M17 17h2a2 2 0 002-2v-4a2 2 0 00-2-2H5a2 2 0 00-2
2v4a2 2 0 002 2h2m2 4h6a2 2 0 002-2v-4a2 2 0 00-2-2H9a2 2 0 00-2 2v4a2 2 0 002 2zm8-12V5a2 2 0
00-2-2H9a2 2 0 00-2 2v4h10z" />
```

```
                                </svg>
                                Print Ledger
                            </button>
                        </div>
                    </div>
                </div>

                <!-- SHARED FILTERS & TABS -->
                <div class="pm-stats-ribbon inventory-stats">
                    <div class="pm-stat-card">
                        <div class="pm-stat-icon total">📦</div>
                        <div>
                            <div class="pm-stat-value" id="inv-stat-total">0</div>
                            <div class="pm-stat-label">Total Items</div>
                        </div>
                    </div>
                    <div class="pm-stat-card">
                        <div class="pm-stat-icon alert">⚠️</div>
                        <div>
                            <div class="pm-stat-value text-red-600" id="inv-stat-low">0</div>
                            <div class="pm-stat-label">Low Stock</div>
                        </div>
                    </div>
                    <div class="pm-stat-card">
                        <div class="pm-stat-icon val">💰</div>
                        <div>
                            <div class="pm-stat-value" id="inv-stat-value">₹0</div>
                            <div class="pm-stat-label">Total Value</div>
                        </div>
                    </div>
                </div>

                <div class="pm-filter-bar inventory-filters" style="margin-bottom: 0.5rem;">
                    <div class="filter-group">
                        <label class="filter-label">Search Inventory & History</label>
                        <input type="text" id="inv-search" class="filter-input"
placeholder="Name, ID or Bin..."
                            oninput="window.adminApp.filterInventory()">
                    </div>
                    <div class="filter-group">
                        <label class="filter-label">Category</label>
                        <select id="inv-category-filter" class="filter-input"
                            onchange="window.adminApp.filterInventory()">
                            <option value="all">All Categories</option>
                            <option value="Raw Material">Raw Material</option>
                            <option value="Consumable">Consumable</option>
                            <option value="Tool">Tool</option>
                            <option value="Fastener">Fastener</option>
                        </select>
                    </div>
                    <div class="filter-group">
                        <label class="filter-label">Status</label>
```

```
                        <select id="inv-status-filter" class="filter-input"
                            onchange="window.adminApp.filterInventory()">
                            <option value="all">All Status</option>
                            <option value="In Stock">In Stock</option>
                            <option value="Low Stock">Low Stock</option>
                            <option value="Out of Stock">Out of Stock</option>
                        </select>
                    </div>
                </div>

                <div class="tabs inventory-sub-tabs" style="margin-bottom: 1.5rem; padding: 0
1rem;">
                    <button id="tab-inventory-master" class="tab-btn active"
                        onclick="window.adminApp.switchInventoryTab('master')">Item
Master</button>
                    <button id="tab-inventory-ledger" class="tab-btn"
                        onclick="window.adminApp.switchInventoryTab('ledger')">Transaction
Ledger</button>
                </div>

                <!-- SUBVIEW: ITEM MASTER -->
                <div id="subview-inventory-master">

                    <!-- Inventory Master Table -->
                    <div class="pm-table-container inventory-table-container">
                        <table class="pm-table inventory-table">
                            <thead class="cr-emerald-header">
                                <tr>
                                    <th class="cr-emerald-bg">Item Details</th>
                                    <th class="cr-emerald-bg">Category</th>
                                    <th class="cr-emerald-bg">Location</th>
                                    <th class="cr-emerald-bg text-center">Current Stock</th>
                                    <th class="cr-emerald-bg">Status</th>
                                    <th class="cr-emerald-bg text-right">Actions</th>
                                </tr>
                            </thead>
                            <tbody id="inventory-list-body">
                                <tr>
                                    <td colspan="6" class="p-12 text-center text-slate-400">
                                        <div class="flex flex-col items-center gap-3">
                                            <div
                                                class="animate-pulse bg-slate-100 w-12 h-12
rounded-full flex items-center justify-center">
                                                📦</div>
                                            <span>Loading inventory master...</span>
                                        </div>
                                    </td>
                                </tr>
                            </tbody>
                        </table>
                    </div>
                </div>
```

```
                <!-- SUBVIEW: TRANSACTION LEDGER -->
                <div id="subview-inventory-ledger" class="hidden">
                    <div class="pm-table-container" id="inventory-ledger-print-area">
                        <!-- Print Header (Hidden on screen) -->
                        <div class="print-only"
                            style="display: none; margin-bottom: 2rem; border-bottom: 2px
solid #059669; padding-bottom: 1rem;">
                            <div style="display: flex; justify-content: space-between; align-
items: flex-end;">
                                <div>
                                    <h1
                                        style="font-size: 24px; color: #059669; font-weight:
800; margin-bottom: 0.25rem;">
                                        INVENTORY TRANSACTION LEDGER</h1>
                                    <p style="color: #64748b; font-size: 14px;">Innovative
Engineering Solutions -
                                        Asset Management</p>
                                </div>
                                <div style="text-align: right;">
                                    <p style="font-size: 12px; color: #94a3b8;">Report
Generated: <span
                                            id="ledger-print-date"></span></p>
                                </div>
                            </div>
                        </div>

                        <table class="pm-table">
                            <thead class="cr-emerald-header sticky top-0 z-10">
                                <tr>
                                    <th class="cr-emerald-bg">Timestamp</th>
                                    <th class="cr-emerald-bg">Item</th>
                                    <th class="cr-emerald-bg">Category</th>
                                    <th class="cr-emerald-bg">Type</th>
                                    <th class="cr-emerald-bg text-center">Qty</th>
                                    <th class="cr-emerald-bg">Cost / Unit</th>
                                    <th class="cr-emerald-bg">Total value</th>
                                    <th class="cr-emerald-bg">Reason</th>
                                    <th class="cr-emerald-bg">Order ID</th>
                                    <th class="cr-emerald-bg">By</th>
                                </tr>
                            </thead>
                            <tbody id="inventory-ledger-body">
                                <tr>
                                    <td colspan="9" class="p-12 text-center text-slate-
400">Loading ledger data...
                                    </td>
                                </tr>
                            </tbody>
                        </table>
                    </div>
                </div>
```

```
            </section>


            <!-- View: Delivery Report -->
            <section id="view-delivery_report" class="view-section hidden">
                <!-- Control Row -->
                <div
                    class="flex items-center justify-between w-full bg-white p-4 rounded-xl
border border-slate-200 shadow-sm mb-6">
                    <div class="flex items-center gap-3">
                        <div
                            class="flex items-center gap-2 bg-slate-50 border border-slate-200
rounded-lg px-3 py-2">
                            <svg class="text-slate-400 w-5 h-5" fill="none"
stroke="currentColor"
                                viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 002-2V7a2 2 0 00-2-
2H5a2 2 0 00-2 2v12a2 2 0 002 2z">
                                </path>
                            </svg>
                            <input type="week" id="delivery-week-picker"
                                class="text-sm bg-transparent border-none focus:outline-none
text-slate-700 font-medium">
                            <div class="w-px h-4 bg-slate-200 mx-2"></div>
                            <input type="month" id="delivery-month-picker"
                                class="text-sm bg-transparent border-none focus:outline-none
text-slate-700 font-medium"
                                title="Filter by Month">
                        </div>
                        <span id="delivery-week-range"
                            class="text-sm text-slate-500 font-medium bg-white px-3 py-2
rounded-lg border border-slate-200 shadow-sm hidden"></span>
                        <button class="btn btn-secondary" id="delivery-trash-btn"
                            onclick="window.adminApp.toggleDeliveryTrash()">
                            <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-
1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16">
                                </path>
                            </svg>
                            Trash
                        </button>
                        <button id="delivery-empty-trash-btn" class="btn btn-danger hidden"
                            onclick="window.adminApp.emptyTrash('delivery')">
                            <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
```

```
                                    d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-
1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" />
                            </svg>
                            Empty Trash
                        </button>
                    </div>

                    <div class="flex items-center gap-3">
                        <button class="btn btn-secondary"
onclick="window.adminApp.printDeliveryReport()">
                            <svg width="20" height="20" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M17 17h2a2 2 0 002-2v-4a2 2 0 00-2-2H5a2 2 0 00-2 2v4a2
2 0 002 2h2m2 4h6a2 2 0 002-2v-4a2 2 0 00-2-2H9a2 2 0 00-2 2v4a2 2 0 002 2zm8-12V5a2 2 0 00-2-
2H9a2 2 0 00-2 2v4h10z">
                                </path>
                            </svg>
                            Print Report
                        </button>
                        <button class="btn btn-purple"
onclick="window.adminApp.openAddDeliveryModal()">
                            <svg width="20" height="20" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 4v16m8-8H4" />
                            </svg>
                            Add Direct Entry
                        </button>
                    </div>
                </div>

                <!-- Report Summary Cards -->
                <div class="grid grid-cols-1 md:grid-cols-4 gap-6 mb-6">
                    <div class="stats-card">
                        <div class="stats-card-icon bg-purple-100 text-purple-600">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M9 17a2 2 0 11-4 0 2 2 0 014 0zM19 17a2 2 0 11-4 0 2 2
0 014 0z"></path>
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M13 16V6a1 1 0 00-1-1H4a1 1 0 00-1 1v10a1 1 0 001
1h1m8-1a1 1 0 01-1 1H9m4-1V8a1 1 0 011-1h2.586a1 1 0 01.707.293l3.414 3.414a1 1 0
01.293.707V16a1 1 0 01-1 1h-1m-6-1a1 1 0 001 1h1M5 17a2 2 0 104 0m-4 0a2 2 0 114 0m6 0a2 2 0
104 0m-4 0a2 2 0 114 0">
                                </path>
                            </svg>
                        </div>
```

```
                        <div>
                            <p class="text-sm text-slate-500 font-medium">Delivered Items</p>
                            <h3 class="text-2xl font-bold text-slate-800" id="report-total-
items">0</h3>
                        </div>
                    </div>
                    <div class="stats-card">
                        <div class="stats-card-icon bg-purple-100 text-purple-600">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 8c-1.657 0-3 .895-3 2s1.343 2 3 2 3 .895 3 2-1.343
2-3 2m0-8c1.11 0 2.08.402 2.599 1M12 8V7m0 1v8m0 0v1m0-1c-1.11 0-2.08-.402-2.599-1M21 12a9 9 0
11-18 0 9 9 0 0118 0z">
                                </path>
                            </svg>
                        </div>
                        <div>
                            <p class="text-sm text-slate-500 font-medium">Total Value</p>
                            <h3 class="text-2xl font-bold text-slate-800" id="report-total-
value">₹0</h3>
                        </div>
                    </div>
                    <div class="stats-card">
                        <div class="stats-card-icon bg-purple-100 text-purple-600">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 4.354a4 4 0 110 5.292M15 21H3v-1a6 6 0 0112 0v1zm0
0h6v-1a6 6 0 00-9-5.197M13 7a4 4 0 11-8 0 4 4 0 018 0z">
                                </path>
                            </svg>
                        </div>
                        <div>
                            <p class="text-sm text-slate-500 font-medium">Total Manpower</p>
                            <h3 class="text-2xl font-bold text-slate-800" id="report-total-
manpower">0</h3>
                        </div>
                    </div>
                </div>

                <!-- Delivery Report Table -->
                <div class="table-container">
                    <div class="table-wrapper">
                        <table id="delivery-report-table">
                            <thead>
                                <tr>
                                    <!-- Widths adjusted for report layout -->
                                    <th style="width: 50px;" class="text-center">S.No</th>
                                    <th style="width: 100px;">Date</th>
                                    <th style="width: 100px;">Order ID</th>
                                    <th style="width: 130px;">Customer</th>
```

```
                                    <th style="width: 160px;">Description</th>
                                    <th style="width: 80px; text-align: center !important;"
class="text-center">DRG
                                        NO</th>
                                    <th style="width: 100px;">Department</th>
                                    <th style="width: 80px;">DC.No</th>
                                    <th style="width: 80px;">Quantity</th>
                                    <th style="width: 50px;">Unit</th>
                                    <th style="width: 100px;">Delivery Value</th>
                                    <th style="width: 120px;">Daily Value</th>
                                    <th style="width: 120px;">Manpower</th>
                                    <th style="width: 80px;" class="text-center no-
print">Actions</th>
                                </tr>
                            </thead>
                            <tbody id="delivery-report-body">
                                <!-- Report rows will be injected here -->
                        </table>
                    </div>
                </div>
            </section>

            <!-- View: Project Management (Core Module) -->
            <section id="view-project_management" class="view-section hidden">
                <!-- Header Bar -->
                <div class="pm-header">
                    <div class="pm-header-title">
                        <h2>🏗️ Project Management</h2>
                        <span class="pm-header-badge">Regulated Workflow</span>
                    </div>
                    <div class="pm-header-actions">
                        <div class="pm-view-toggle">
                            <button class="view-btn active" id="btn-view-grid"
                                onclick="window.adminApp.setProjectView('grid')" title="Grid
View">
                                <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                    <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
                                        d="M4 6a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2H6a2 2
0 01-2-2V6zM14 6a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2h-2a2 2 0 01-2-2V6zM4 16a2 2 0 012-
2h2a2 2 0 012 2v2a2 2 0 01-2 2H6a2 2 0 01-2-2v-2zM14 16a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2
2h-2a2 2 0 01-2-2v-2z" />
                                </svg>
                            </button>
                            <button class="view-btn" id="btn-view-list"
                                onclick="window.adminApp.setProjectView('list')" title="List
View">
                                <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                    <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
```

```
                                    d="M4 6h16M4 12h16M4 18h16" />
                            </svg>
                        </button>
                    </div>

                    <button class="pm-trash-toggle" id="pm-trash-toggle-btn"
                        onclick="window.adminApp.toggleProjectTrash()">
                        <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-
1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" />
                        </svg>
                        <span id="pm-trash-toggle-label">Trash</span>
                        <span class="pm-trash-count" id="pm-trash-count"
style="display:none;">0</span>
                    </button>
                    <button class="btn btn-primary" id="pm-new-project-btn"
                        onclick="window.adminApp.openAddProjectModal()">
                        <svg width="16" height="16" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                d="M12 4v16m8-8H4" />
                        </svg>
                        New Project
                    </button>
                </div>
            </div>

            <!-- Stats Ribbon -->
            <div class="pm-stats-ribbon">
                <div class="pm-stat-card">
                    <div class="pm-stat-icon total">📁</div>
                    <div>
                        <div class="pm-stat-value" id="pm-stat-total">0</div>
                        <div class="pm-stat-label">Total Projects</div>
                    </div>
                </div>
                <div class="pm-stat-card">
                    <div class="pm-stat-icon active">⚡</div>
                    <div>
                        <div class="pm-stat-value" id="pm-stat-active">0</div>
                        <div class="pm-stat-label">Active</div>
                    </div>
                </div>
                <div class="pm-stat-card">
                    <div class="pm-stat-icon done">✅</div>
                    <div>
                        <div class="pm-stat-value" id="pm-stat-completed">0</div>
                        <div class="pm-stat-label">Completed</div>
                    </div>
```

```
                    </div>
                    <div class="pm-stat-card">
                        <div class="pm-stat-icon trash">🗑</div>
                        <div>
                            <div class="pm-stat-value" id="pm-stat-trash">0</div>
                            <div class="pm-stat-label">In Trash</div>
                        </div>
                    </div>
                </div>

                <!-- Trash Banner (shown when in trash view) -->
                <div class="pm-trash-banner" id="pm-trash-banner" style="display:none;">
                    <div class="pm-trash-banner-icon">🗑</div>
                    <div class="pm-trash-banner-text">
                        <strong>Viewing Trash</strong>
                        <p>These projects are soft-deleted. Restore them or permanently
delete.</p>
                    </div>
                </div>

                <!-- Filter Bar -->
                <div class="pm-filter-bar" id="pm-filter-bar">
                    <div class="filter-group">
                        <label class="filter-label">Search</label>
                        <input type="text" id="project-search" class="filter-input"
                            placeholder="Project ID or Name..."
oninput="window.adminApp.filterProjects()">
                    </div>
                    <div class="filter-group">
                        <label class="filter-label">Status</label>
                        <select id="project-status-filter" class="filter-input"
                            onchange="window.adminApp.filterProjects()">
                            <option value="all">All Statuses</option>
                            <option value="Draft">Draft</option>
                            <option value="Under Review">Under Review</option>
                            <option value="Approved">Approved</option>
                            <option value="In Progress">In Progress</option>
                            <option value="Completed">Completed</option>
                        </select>
                    </div>
                    <div class="filter-group">
                        <label class="filter-label">Customer</label>
                        <select id="project-customer-filter" class="filter-input"
                            onchange="window.adminApp.filterProjects()">
                            <option value="all">All Customers</option>
                        </select>
                    </div>
                    <div class="filter-group">
                        <label class="filter-label">Job Type</label>
                        <select id="project-type-filter" class="filter-input"
                            onchange="window.adminApp.filterProjects()">
                            <option value="all">All Types</option>
```

```
                            <option value="CNC">CNC</option>
                            <option value="VMC">VMC</option>
                            <option value="Fabrication">Fabrication</option>
                            <option value="Hybrid">Hybrid</option>
                        </select>
                    </div>
                </div>

                <!-- Project Grid -->
                <div id="project-grid" class="pm-grid">
                    <div class="pm-empty-state">
                        <div class="pm-empty-icon">
                            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="1.5"
                                    d="M19 11H5m14 0a2 2 0 012 2v6a2 2 0 01-2 2H5a2 2 0 01-2-
2v-6a2 2 0 012-2m14 0V9a2 2 0 00-2-2M5 11V9a2 2 0 012-2m0 0V5a2 2 0 012-2h6a2 2 0 012 2v2M7
7h10" />
                            </svg>
                        </div>
                        <div class="pm-empty-title">Loading projects...</div>
                        <div class="pm-empty-text">Your projects will appear here.</div>
                    </div>
                </div>

                <!-- Project List (Table) -->
                <div id="project-list-view" class="pm-list-view hidden">
                    <div class="pm-table-container">
                        <table class="pm-table">
                            <thead class="cr-emerald-header">
                                <tr>
                                    <th class="cr-emerald-bg">Project Details</th>
                                    <th class="cr-emerald-bg">Customer</th>
                                    <th class="cr-emerald-bg">Type</th>
                                    <th class="cr-emerald-bg">Status</th>
                                    <th class="cr-emerald-bg">Timeline</th>
                                    <th class="cr-emerald-bg text-right">Actions</th>
                                </tr>
                            </thead>
                            <tbody id="project-list-body">
                                <!-- Rows injected by JS -->
                            </tbody>
                        </table>
                    </div>
                </div>
            </section>


            <!-- View: Project Detail (Deep Dive Workspace) -->
            <section id="view-project_detail" class="view-section hidden">

                <!-- Compact Header -->
```

```
                <div class="dd-header">
                    <div class="dd-header-left">
                        <button class="dd-back-btn"
onclick="window.adminApp.switchView('project_management')">
                            <svg width="18" height="18" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M10 19l-7-7m0 0l7-7m-7 7h18" />
                            </svg>
                        </button>
                        <div>
                            <div style="display:flex;align-items:center;gap:0.5rem;">
                                <span id="detail-project-id" class="dd-project-id">IES-2026-
00000</span>
                                <div class="dd-status-container relative">
                                    <button id="detail-project-status"
                                        class="status-badge draft flex items-center gap-1
group"
                                        onclick="window.adminApp.toggleStatusMenu(event)">
                                        <span>Draft</span>
                                        <svg class="w-3 h-3 opacity-50 group-hover:opacity-100
transition-opacity"
                                            fill="none" stroke="currentColor" viewBox="0 0 24
24">
                                            <path stroke-linecap="round" stroke-
linejoin="round" stroke-width="2"
                                                d="M19 9l-7 7-7-7"></path>
                                        </svg>
                                    </button>
                                    <div id="dd-status-menu" class="dd-status-menu hidden">
                                        <div class="status-menu-item draft"

onclick="window.adminApp.updateProjectStatus('Draft')">
                                            <span class="status-dot"></span> Draft
                                        </div>
                                        <div class="status-menu-item under-review"

onclick="window.adminApp.updateProjectStatus('Under Review')">
                                            <span class="status-dot"></span> Under Review
                                        </div>
                                        <div class="status-menu-item approved"

onclick="window.adminApp.updateProjectStatus('Approved')">
                                            <span class="status-dot"></span> Approved
                                        </div>
                                        <div class="status-menu-item in-progress"
                                            onclick="window.adminApp.updateProjectStatus('In
Progress')">
                                            <span class="status-dot"></span> In Progress
                                        </div>
                                        <div class="status-menu-item completed"
```

```
onclick="window.adminApp.updateProjectStatus('Completed')">
                                        <span class="status-dot"></span> Completed
                                    </div>
                                </div>
                            </div>
                            <h2 id="detail-project-name" class="dd-project-name">Project
Name</h2>
                        </div>
                    </div>
                    <div class="dd-header-right">
                        <!-- Legacy buttons removed -->
                    </div>
                </div>



                <!-- Tab Navigation -->
                <div class="dd-tabs">
                    <button class="dd-tab active"
onclick="window.adminApp.switchDeepDiveTab('review')"
                        data-tab="review">
                        <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                d="M9 5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2V7a2 2 0
00-2-2h-2M9 5a2 2 0 002 2h2a2 2 0 002-2M9 5a2 2 0 012-2h2a2 2 0 012 2m-6 9l2 2 4-4" />
                        </svg>
                        Contract Review
                    </button>
                    <button class="dd-tab"
onclick="window.adminApp.switchDeepDiveTab('files')" data-tab="files">
                        <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                d="M7 21h10a2 2 0 002-2V9.414a1 1 0 00-.293-.707l-5.414-
5.414A1 1 0 0012.586 3H7a2 2 0 00-2 2v14a2 2 0 002 2z" />
                        </svg>
                        Files
                    </button>
                </div>

                <!-- ========== TAB PANELS ========== -->



                <!-- Panel: Contract Review -->
                <div id="dd-panel-review" class="dd-panel active h-full overflow-hidden">
                    <div class="cr-gateway-container h-full flex flex-col p-4 gap-4 overflow-
y-auto"
                        style="height: 100%; overscroll-behavior: contain;">
```

```
                        <!-- Monolithic Contract Review Master Table -->
                        <!-- Modern Card-Based Contract Review (Assembles to Monolithic in
Print) -->
                        <div class="cr-modern-container flex flex-col gap-6 pb-6">

                            <!-- Top actions/Title -->
                            <div class="cr-print-header" style="display: none;">
                                <div
                                    style="display: flex; justify-content: space-between;
align-items: center; border-bottom: 3px solid #0f172a; padding-bottom: 12px; margin-bottom:
20px;">
                                    <div>
                                        <div
                                            style="font-size: 24pt; font-weight: 800; color:
#0f172a; letter-spacing: 0.02em;">
                                            INNOVATIVE ENGINEERING SOLUTIONS</div>
                                        <div
                                            style="font-size: 11pt; color: #64748b; letter-
spacing: 0.05em; margin-top: 4px;">
                                            PRECISION • QUALITY • DELIVERY</div>
                                    </div>
                                    <div style="text-align: right;">
                                        <div style="font-size: 20pt; font-weight: 700; color:
#0f172a;">Contract
                                            Review</div>
                                        <div style="font-size: 11pt; color: #64748b;" id="cr-
print-date"></div>
                                    </div>
                                </div>
                            </div>
                            <div class="flex items-center justify-between mt-2">
                                <h3 class="text-xl font-bold text-slate-800 tracking-
tight">Contract Review</h3>
                                <div
                                    class="flex items-center gap-2 bg-slate-100 rounded-lg p-
1.5 border border-slate-200">
                                    <span
                                        class="text-xs font-bold text-slate-500 uppercase
tracking-widest px-2">Order
                                        No</span>
                                    <input type="text" id="cr-review-no"
                                        class="bg-white border border-slate-300 rounded px-3
py-1 text-sm font-bold text-emerald-700 w-44 outline-none focus:border-emerald-500"
                                        readonly>
                                </div>
                            </div>

                            <!-- Card 1: Customer & Internal Data -->
                            <div
                                class="cr-section-card bg-white rounded-xl border border-
slate-200 shadow-sm overflow-hidden">
```

```
                                <table class="cr-card-table w-full border-collapse text-sm">
                                    <colgroup>
                                        <col style="width: 5%;"> <!-- S.No -->
                                        <col style="width: 30%;"> <!-- Checklist -->
                                        <col style="width: 5%;"> <!-- Req Yes -->
                                        <col style="width: 5%;"> <!-- Req No -->
                                        <col style="width: 5%;"> <!-- Out Ok -->
                                        <col style="width: 5%;"> <!-- Out Nok -->
                                        <col style="width: 7%;"> <!-- Out NA -->
                                        <col style="width: 8%;"> <!-- Out More -->
                                        <col style="width: 30%;"> <!-- Remarks -->
                                    </colgroup>
                                    <tbody>
                                        <tr>
                                            <td colspan="4" class="cr-emerald-bg">
                                                <div class="flex items-center gap-2">
                                                    <svg width="14" height="14" viewBox="0 0
24 24" fill="none"
                                                        stroke="currentColor" stroke-
width="2.5"
                                                        stroke-linecap="round" stroke-
linejoin="round"
                                                        class="opacity-90">
                                                        <path d="M19 21v-2a4 4 0 0 0-4-4H9a4 4
0 0 0-4 4v2"></path>
                                                        <circle cx="12" cy="7" r="4"></circle>
                                                    </svg>
                                                    Customer Data
                                                </div>
                                            </td>
                                            <td colspan="5" class="cr-emerald-bg">
                                                <div class="flex items-center gap-2">
                                                    <svg width="14" height="14" viewBox="0 0
24 24" fill="none"
                                                        stroke="currentColor" stroke-
width="2.5"
                                                        stroke-linecap="round" stroke-
linejoin="round"
                                                        class="opacity-90">
                                                        <path
                                                            d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0
0 0 2 2h12a2 2 0 0 0 2-2V8z">
                                                        </path>
                                                        <polyline points="14 2 14 8 20 8">
</polyline>
                                                        <line x1="16" y1="13" x2="8" y2="13">
</line>
                                                        <line x1="16" y1="17" x2="8" y2="17">
</line>
                                                        <polyline points="10 9 9 9 8 9">
</polyline>
                                                    </svg>
```

```
                                                Internal Order
                                            </div>
                                        </td>
                                    </tr>
                                    <tr>
                                        <td colspan="2" class="cr-master-label">PO NO</td>
                                        <td colspan="2" class="p-0"><input type="text"
id="cr-po-no"
                                                class="cr-master-input"></td>
                                        <td colspan="2" class="cr-master-label">Date</td>
                                        <td colspan="3" class="p-0"><input type="date"
id="cr-internal-date"
                                                class="cr-master-input"></td>
                                    </tr>
                                    <tr>
                                        <td colspan="2" class="cr-master-label">Drg
No</td>
                                        <td colspan="2" class="p-0"><input type="text"
id="cr-drawing-no"
                                                class="cr-master-input"></td>
                                        <td colspan="2" class="cr-master-
label">Accountability/PL</td>
                                        <td colspan="3" class="p-0">
                                            <div class="member-search-container" id="cr-
search-accountability">
                                                <div class="search-tags-container">
                                                    <input type="text" class="search-
input-inline">
                                                </div>
                                                <div class="search-suggestions-dropdown
hidden"></div>
                                                <input type="hidden" id="cr-
accountability">
                                            </div>
                                        </td>
                                    </tr>
                                    <tr>
                                        <td colspan="2" class="cr-master-label">Date</td>
                                        <td colspan="2" class="p-0"><input type="date"
id="cr-date"
                                                class="cr-master-input"></td>
                                        <td colspan="2" class="cr-master-label">IO
Number</td>
                                        <td colspan="3" class="p-0"><input type="text"
id="cr-io-number"
                                                class="cr-master-input" readonly
                                                style="color: #059669; font-weight: 700;">
</td>
                                    </tr>
                                    <tr>
                                        <td colspan="2" class="cr-master-label">Delivery
date</td>
```

```
                                        <td colspan="2" class="p-0"><input type="date"
id="cr-delivery-date"
                                                class="cr-master-input"></td>
                                        <td colspan="2" class="cr-master-label">Team</td>
                                        <td colspan="3" class="p-0">
                                            <select id="cr-team" class="cr-master-select">
                                                <option value="">Select Team</option>
                                                <option value="CNC">CNC</option>
                                                <option value="Fab">Fab</option>
                                                <option value="Sourcing">Sourcing</option>
                                            </select>
                                        </td>
                                    </tr>
                                    <tr>
                                        <td colspan="2" class="cr-master-label">Contact
person</td>
                                        <td colspan="2" class="p-0"><input type="text"
id="cr-contact-person"
                                                class="cr-master-input"></td>
                                        <td colspan="2" class="cr-master-label">Team
leader</td>
                                        <td colspan="3" class="p-0">
                                            <div class="member-search-container" id="cr-
search-team-leader">
                                                <div class="search-tags-container">
                                                    <input type="text" class="search-
input-inline">
                                                </div>
                                                <div class="search-suggestions-dropdown
hidden"></div>
                                                <input type="hidden" id="cr-team-leader">
                                            </div>
                                        </td>
                                    </tr>
                                    <tr>
                                        <td colspan="2" class="cr-master-label">Ph No</td>
                                        <td colspan="2" class="p-0"><input type="text"
id="cr-phone"
                                                class="cr-master-input"></td>
                                        <td colspan="2" class="cr-master-
label">Members</td>
                                        <td colspan="3" class="p-0">
                                            <div class="member-search-container" id="cr-
search-members">
                                                <div class="search-tags-container">
                                                    <input type="text" class="search-
input-inline">
                                                </div>
                                                <div class="search-suggestions-dropdown
hidden"></div>
                                                <input type="hidden" id="cr-members">
                                            </div>
```

```
                                        </td>
                                    </tr>
                                </tbody>
                            </table>
                        </div>

                        <!-- Card 2: Checklist -->
                        <div
                            class="cr-section-card bg-white rounded-xl border border-
slate-200 shadow-sm overflow-hidden">
                            <table class="cr-card-table w-full border-collapse text-sm">
                                <colgroup>
                                    <col style="width: 5%;"> <!-- S.No -->
                                    <col style="width: 30%;"> <!-- Checklist -->
                                    <col style="width: 5%;"> <!-- Req Yes -->
                                    <col style="width: 5%;"> <!-- Req No -->
                                    <col style="width: 5%;"> <!-- Out Ok -->
                                    <col style="width: 5%;"> <!-- Out Nok -->
                                    <col style="width: 7%;"> <!-- Out NA -->
                                    <col style="width: 8%;"> <!-- Out More -->
                                    <col style="width: 30%;"> <!-- Remarks -->
                                </colgroup>
                                <thead class="bg-slate-50">
                                    <tr class="cr-emerald-header">
                                        <td rowspan="2" class="cr-emerald-bg text-center
border-b-0">
                                            <div class="flex flex-col items-center">
                                                <svg width="12" height="12" viewBox="0 0
24 24" fill="none"
                                                    stroke="currentColor" stroke-width="2"
                                                    class="mb-1 opacity-90">
                                                    <path
                                                        d="M13 2H6a2 2 0 0 0-2 2v16a2 2 0
0 0 2 2h12a2 2 0 0 0 2-2V9z">
                                                    </path>
                                                    <polyline points="13 2 13 9 20 9">
</polyline>
                                                </svg>
                                                S.No
                                            </div>
                                        </td>
                                        <td rowspan="2" class="cr-emerald-bg text-center
border-b-0">
                                            <div class="flex items-center gap-2 justify-
center">
                                                <svg width="14" height="14" viewBox="0 0
24 24" fill="none"
                                                    stroke="currentColor" stroke-width="2"
class="opacity-90">
                                                    <path
                                                        d="M16 4h2a2 2 0 0 1 2 2v14a2 2 0
0 1-2 2H6a2 2 0 0 1-2-2V6a2 2 0 0 1 2-2h2">
```

```
                                                    </path>
                                                    <rect x="8" y="2" width="8" height="4"
rx="1" ry="1"></rect>
                                                    <path d="M9 14l2 2 4-4"></path>
                                                </svg>
                                                Checklist Items
                                            </div>
                                        </td>
                                        <td colspan="2" class="cr-emerald-bg text-center
py-1">Requirement</td>
                                        <td colspan="4" class="cr-emerald-bg text-center
py-1">Review Outcome
                                        </td>
                                        <td rowspan="2" class="cr-emerald-bg text-center
border-b-0">
                                            <div class="flex items-center gap-2 justify-
center">
                                                <svg width="14" height="14" viewBox="0 0
24 24" fill="none"
                                                    stroke="currentColor" stroke-width="2"
class="opacity-90">
                                                    <path
                                                        d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2
2 0 0 1 2-2h14a2 2 0 0 1 2 2z">
                                                    </path>
                                                </svg>
                                                Remarks
                                            </div>
                                        </td>
                                    </tr>
                                    <tr>
                                        <td class="cr-emerald-bg text-center">YES</td>
                                        <td class="cr-emerald-bg text-center">NO</td>
                                        <td class="cr-emerald-bg text-center">OK</td>
                                        <td class="cr-emerald-bg text-center">NOK</td>
                                        <td class="cr-emerald-bg text-center">N.A</td>
                                        <td class="cr-emerald-bg text-center">Clarity</td>
                                    </tr>
                                </thead>
                                <tbody id="cr-excel-checklist">
                                    <!-- JS will inject rows here -->
                                </tbody>
                            </table>
                            <!-- Add button outside table in screen, hidden in print -->
                            <div class="p-3 bg-slate-50 border-t border-slate-200 cr-no-
print">
                                <button
                                    class="cr-master-add-btn w-full py-2 text-sm text-
emerald-700 bg-emerald-100 hover:bg-emerald-200 rounded-lg transition-colors font-bold shadow-
sm"
onclick="window.adminApp.addCustomContractReviewItem()">+ Add Special
```

```
                                        Requirement</button>
                                </div>
                            </div>

                            <!-- Card 3: Instructions & 6M -->
                            <div
                                class="cr-section-card bg-white rounded-xl border border-
slate-200 shadow-sm overflow-hidden">
                                <table class="cr-card-table w-full border-collapse text-sm">
                                    <colgroup>
                                        <col style="width: 5%;"> <!-- S.No -->
                                        <col style="width: 30%;"> <!-- Checklist -->
                                        <col style="width: 5%;"> <!-- Req Yes -->
                                        <col style="width: 5%;"> <!-- Req No -->
                                        <col style="width: 5%;"> <!-- Out Ok -->
                                        <col style="width: 5%;"> <!-- Out Nok -->
                                        <col style="width: 7%;"> <!-- Out NA -->
                                        <col style="width: 8%;"> <!-- Out More -->
                                        <col style="width: 30%;"> <!-- Remarks -->
                                    </colgroup>
                                    <tbody>
                                        <tr>
                                            <td colspan="5"
                                                class="cr-emerald-bg font-bold text-left
align-middle pl-3 py-2">
                                                <div class="flex items-center gap-2">
                                                    <svg width="14" height="14" viewBox="0 0
24 24" fill="none"
                                                        stroke="currentColor" stroke-
width="2.5"
                                                        stroke-linecap="round" stroke-
linejoin="round"
                                                        class="opacity-90">
                                                        <circle cx="12" cy="12" r="10">
</circle>
                                                        <line x1="12" y1="8" x2="12" y2="12">
</line>
                                                        <line x1="12" y1="16" x2="12.01"
y2="16"></line>
                                                    </svg>
                                                    Important Instructions
                                                </div>
                                            </td>
                                            <td colspan="2" class="cr-emerald-bg text-
center">Points</td>
                                            <td colspan="2" class="cr-emerald-bg text-
center">Comments</td>
                                        </tr>
                                        <tr>
                                            <td colspan="5" rowspan="6" class="p-0 align-top">
                                                <textarea id="cr-important-instructions"
                                                    class="cr-master-textarea w-full h-full p-
```

```
3 outline-none resize-none bg-yellow-50/30 focus:bg-yellow-50/70 transition-colors"
                                                    style="min-height: 150px;"></textarea>
                                            </td>
                                            <td colspan="2" class="cr-master-label text-left
pl-3">Matl</td>
                                            <td colspan="2" class="p-0"><input type="text"
data-field="cmt-matl"
                                                    class="cr-master-input"></td>
                                        </tr>
                                        <tr>
                                            <td colspan="2" class="cr-master-label text-left
pl-3">Machine</td>
                                            <td colspan="2" class="p-0"><input type="text"
data-field="cmt-machine"
                                                    class="cr-master-input"></td>
                                        </tr>
                                        <tr>
                                            <td colspan="2" class="cr-master-label text-left
pl-3">Man</td>
                                            <td colspan="2" class="p-0"><input type="text"
data-field="cmt-man"
                                                    class="cr-master-input"></td>
                                        </tr>
                                        <tr>
                                            <td colspan="2" class="cr-master-label text-left
pl-3">Method</td>
                                            <td colspan="2" class="p-0"><input type="text"
data-field="cmt-method"
                                                    class="cr-master-input"></td>
                                        </tr>
                                        <tr>
                                            <td colspan="2" class="cr-master-label text-left
pl-3">Measure</td>
                                            <td colspan="2" class="p-0"><input type="text"
data-field="cmt-measure"
                                                    class="cr-master-input"></td>
                                        </tr>
                                        <tr>
                                            <td colspan="2" class="cr-master-label text-left
pl-3">Tools</td>
                                            <td colspan="2" class="p-0"><input type="text"
data-field="cmt-tools"
                                                    class="cr-master-input"></td>
                                        </tr>
                                    </tbody>
                                </table>
                            </div>

                            <!-- Card 4: Decisions -->
                            <div
                                class="cr-section-card bg-white rounded-xl border border-
slate-200 shadow-sm overflow-hidden">
```

```
                                <table class="cr-card-table w-full border-collapse text-sm">
                                    <colgroup>
                                        <col style="width: 5%;"> <!-- S.No -->
                                        <col style="width: 30%;"> <!-- Checklist -->
                                        <col style="width: 5%;"> <!-- Req Yes -->
                                        <col style="width: 5%;"> <!-- Req No -->
                                        <col style="width: 5%;"> <!-- Out Ok -->
                                        <col style="width: 5%;"> <!-- Out Nok -->
                                        <col style="width: 7%;"> <!-- Out NA -->
                                        <col style="width: 8%;"> <!-- Out More -->
                                        <col style="width: 30%;"> <!-- Remarks -->
                                    </colgroup>
                                    <tbody>
                                        <tr>
                                            <td colspan="9" class="cr-emerald-bg">
                                                <div class="flex items-center gap-2">
                                                    <svg width="14" height="14" viewBox="0 0
24 24" fill="none"
                                                        stroke="currentColor" stroke-
width="2.5"
                                                        stroke-linecap="round" stroke-
linejoin="round"
                                                        class="opacity-90">
                                                        <path
                                                            d="M12 22c5.523 0 10-4.477 10-
10S17.523 2 12 2 2 6.477 2 12s4.477 10 10 10z">
                                                        </path>
                                                        <path d="m9 12 2 2 4-4"></path>
                                                    </svg>
                                                    Final Verification & Order Acceptance
                                                </div>
                                            </td>
                                        </tr>
                                        <tr class="cr-emerald-header">
                                            <td colspan="2" class="cr-emerald-bg text-
center">Decision</td>
                                            <td colspan="1" class="cr-emerald-bg text-center
p-0">Ok/Nok</td>
                                            <td colspan="2" class="cr-emerald-bg text-
center">Prepared By</td>
                                            <td colspan="2" class="cr-emerald-bg text-
center">Reviewed By</td>
                                            <td colspan="2" class="cr-emerald-bg text-
center">Approved By</td>
                                        </tr>
                                        <tr>
                                            <td colspan="2" class="cr-master-label text-right
pr-3">Capability</td>
                                            <td colspan="1" class="p-0 text-center">
                                                <select data-field="cr-decision-cap"
                                                    class="cr-master-select text-center font-
bold">
```

```
                                                <option value="">-</option>
                                                <option value="ok" class="text-emerald-
600">Ok</option>
                                                <option value="nok" class="text-red-
600">Nok</option>
                                            </select>
                                        </td>
                                        <td colspan="2" rowspan="2" class="p-0">
                                            <div class="member-search-container h-full"
id="cr-search-prepared">
                                                <div
                                                    class="search-tags-container h-full
border-0 rounded-none bg-transparent">
                                                    <input type="text"
                                                        class="search-input-inline h-full
text-center">
                                                </div>
                                                <div class="search-suggestions-dropdown
hidden text-left"></div>
                                                <input type="hidden" data-field="cr-
prepared-by"
                                                    id="cr-prepared-by">
                                            </div>
                                        </td>
                                        <td colspan="2" rowspan="2" class="p-0">
                                            <div class="member-search-container h-full"
id="cr-search-reviewed">
                                                <div
                                                    class="search-tags-container h-full
border-0 rounded-none bg-transparent">
                                                    <input type="text"
                                                        class="search-input-inline h-full
text-center">
                                                </div>
                                                <div class="search-suggestions-dropdown
hidden text-left"></div>
                                                <input type="hidden" data-field="cr-
reviewed-by"
                                                    id="cr-reviewed-by">
                                            </div>
                                        </td>
                                        <td colspan="2" rowspan="2" class="p-0">
                                            <div class="member-search-container h-full"
id="cr-search-approved">
                                                <div
                                                    class="search-tags-container h-full
border-0 rounded-none bg-transparent">
                                                    <input type="text"
                                                        class="search-input-inline h-full
text-center">
                                                </div>
                                                <div class="search-suggestions-dropdown
```

```
hidden text-left"></div>
                                                    <input type="hidden" data-field="cr-
approved-by"
                                                        id="cr-approved-by">
                                                </div>
                                            </td>
                                        </tr>
                                        <tr>
                                            <td colspan="2" class="cr-master-label text-right
pr-3">Order Acceptance
                                            </td>
                                            <td colspan="1" class="p-0 text-center">
                                                <select data-field="cr-decision-oa"
                                                    class="cr-master-select text-center font-
bold">
                                                    <option value="">-</option>
                                                    <option value="ok" class="text-emerald-
600">Ok</option>
                                                    <option value="nok" class="text-red-
600">Nok</option>
                                                </select>
                                            </td>
                                        </tr>
                                    </tbody>
                                </table>
                                <!-- Print-Only Notes Block -->
                                <div
                                    class="cr-decision-notes text-[10px] p-4 text-slate-500
bg-slate-50/50 text-left border-t border-slate-200">
                                    <div class="flex gap-2">
                                        <svg width="14" height="14" viewBox="0 0 24 24"
fill="none"
                                            stroke="currentColor" stroke-width="2"
                                            class="mt-0.5 flex-shrink-0 text-slate-400">
                                            <path d="M12 19l7-7 3 3-7 7-3-3z"></path>
                                            <path d="M18 13l-1.5-7.5L2 2l3.5 14.5L13 18l5-5z">
</path>
                                            <path d="m2 2 5 5"></path>
                                            <path d="m11 11 5 5"></path>
                                        </svg>
                                        <div class="leading-relaxed">
                                            <strong
                                                class="text-slate-700 uppercase tracking-wider
text-[9px]">Execution
                                                Guidelines:</strong><br>
                                            1. PL to prepare Job Card and plan for resources
from Day 1 itself for
                                            smooth completion.<br>
                                            2. TL to own full Responsibility for Job Quality
and Delivery.
                                        </div>
                                    </div>
```

```
                                </div>
                            </div>
                        </div>

                        <!-- Actions Footer (Hidden in Print) -->
                        <div
                            class="cr-footer-actions bg-white p-3 rounded shadow-sm border
border-slate-200 flex items-center justify-between mt-4 flex-shrink-0">
                            <div class="flex items-center gap-3">
                                <span id="cr-save-status" class="text-sm font-semibold text-
emerald-600"></span>
                                <div id="cr-overall-status" class="cr-status-badge hidden">
</div>
                            </div>
                            <div class="flex gap-2">
                                <button class="btn btn-sm btn-secondary"
                                    onclick="window.adminApp.printContractReview()">Print
Layout</button>
                                <button class="btn btn-sm btn-secondary"
                                    onclick="window.adminApp.saveContractReview('draft')">Save
Draft</button>
                                <button class="btn btn-sm btn-primary"

onclick="window.adminApp.saveContractReview('finalize')">Finalize Order</button>
                            </div>
                        </div>
                    </div>
                </div>
                <!-- Panel: Files -->
                <div id="dd-panel-files" class="dd-panel">
                    <input type="file" id="project-file-input" style="display:none;"
                        onchange="window.adminApp.handleFileSelection(event)">
                    <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:1rem;">
                        <h3 style="font-size:1rem;font-
weight:700;color:#0f172a;margin:0;">Files & Documents
                        </h3>
                        <button class="btn btn-secondary btn-sm"
onclick="window.adminApp.openUploadModal()">
                            <svg width="14" height="14" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 4v16m8-8H4" />
                            </svg>
                            Upload
                        </button>
                    </div>
                    <div style="background:white;border:1px solid #e2e8f0;border-
radius:12px;overflow:hidden;">
                        <table class="w-full text-sm">
                            <thead>
```

```
                                <tr class="text-left bg-slate-50 text-slate-400 font-bold
uppercase text-[10px]">
                                    <th class="p-3">File Name</th>
                                    <th class="p-3">Category</th>
                                    <th class="p-3 text-center">Version</th>
                                    <th class="p-3">Uploaded By</th>
                                    <th class="p-3 text-right">Actions</th>
                                </tr>
                            </thead>
                            <tbody id="project-files-body">
                                <tr>
                                    <td colspan="5" class="p-8 text-center text-slate-400
italic">No files
                                        uploaded
                                        yet.</td>
                                </tr>
                            </tbody>
                        </table>
                    </div>
                </div>



            </section>

            <!-- ============================================
                 VIEW: Daily Roster
                 ============================================ -->
            <section id="view-daily_roster" class="view-section hidden">
                <!-- Header Bar -->
                <div class="wf-header">
                    <div class="wf-header-left">
                        <h2 class="wf-title">
                            <svg width="22" height="22" fill="none" stroke="currentColor"
viewBox="0 0 24 24"
                                style="vertical-align: middle;">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 002-2V7a2 2 0 00-2-
2H5a2 2 0 00-2 2v12a2 2 0 002 2z" />
                            </svg>
                            Daily Roster
                        </h2>
                        <div class="wf-controls">
                            <div class="wf-control-group">
                                <label class="wf-label">Date</label>
                                <input type="date" id="wf-date-picker" class="form-input wf-
date-input">
                            </div>
                            <div class="wf-control-group">
                                <label class="wf-label">Department</label>
                                <select id="wf-dept-filter" class="form-input wf-dept-select">
```

```
                                    <option value="All">All Departments</option>
                                    <option value="Fab">Fab</option>
                                    <option value="CNC">CNC</option>
                                    <option value="VMC">VMC</option>
                                    <option value="Turning">Turning</option>
                                    <option value="Assembly">Assembly</option>
                                </select>
                            </div>
                        </div>
                    </div>
                    <div class="wf-header-right">
                        <button class="btn wf-btn-copy"
onclick="window.adminApp.wfCopyPreviousDay()"
                            title="Copy Previous Day">
                            <svg width="16" height="16" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M8 16H6a2 2 0 01-2-2V6a2 2 0 012-2h8a2 2 0 012 2v2m-6
12h8a2 2 0 002-2v-8a2 2 0 00-2-2h-8a2 2 0 00-2 2v8a2 2 0 002 2z" />
                            </svg>
                            Copy Prev Day
                        </button>
                        <button class="btn wf-btn-add"
onclick="window.adminApp.wfOpenAssignModal()">
                            <svg width="16" height="16" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M12 4v16m8-8H4" />
                            </svg>
                            Add Assignment
                        </button>
                        <button class="btn wf-btn-save" onclick="window.adminApp.wfSaveAll()">
                            <svg width="16" height="16" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M5 13l4 4L19 7" />
                            </svg>
                            Save
                        </button>
                        <button class="btn wf-btn-print" onclick="window.adminApp.wfPrint()">
                            <svg width="16" height="16" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2"
                                    d="M17 17h2a2 2 0 002-2v-4a2 2 0 00-2-2H5a2 2 0 00-2 2v4a2
2 0 002 2h2m2 4h6a2 2 0 002-2v-4a2 2 0 00-2-2H9a2 2 0 00-2 2v4a2 2 0 002 2zm8-12V5a2 2 0 00-2-
2H9a2 2 0 00-2 2v4h10z" />
                            </svg>
                            Print
```

```
                        </button>
                    </div>
                </div>

                <!-- Unassigned Orders Alert -->
                <div id="wf-unassigned-alert" class="wf-alert hidden">
                    <svg width="16" height="16" fill="none" stroke="currentColor" viewBox="0 0
24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                            d="M13 16h-1v-4h-1m1-4h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z" />
                    </svg>
                    <span id="wf-unassigned-count">0 pending orders</span> not yet assigned
today
                </div>

                <!-- Roster Table -->
                <div class="wf-table-wrap">
                    <table class="wf-table" id="wf-roster-table">
                        <thead>
                            <tr>
                                <th style="width:3%" class="text-center">#</th>
                                <th style="width:15%">Employee & Timing</th>
                                <th style="width:7%" class="text-center">IO No</th>
                                <th style="width:7%" class="text-center">Drawing</th>
                                <th style="width:12%">Description</th>
                                <th style="width:8%">Customer</th>
                                <th style="width:5%" class="text-center">Qty</th>
                                <th style="width:7%" class="text-right">Prod. Value</th>
                                <th style="width:8%" class="text-right">Total Overhead</th>
                                <th style="width:4%" class="text-center">MP</th>
                                <th style="width:8%">Assigned With</th>
                                <th style="width:8%" class="text-center">Work Duration</th>
                                <th style="width:5%" class="text-center">Status</th>
                                <th style="width:3%" class="text-center">Actions</th>
                            </tr>
                        </thead>
                        <tbody id="wf-roster-body">
                            <tr class="wf-empty-row">
                                <td colspan="14" style="text-align:center; padding:2.5rem;
color:#94a3b8;">No
                                    assignments for this date. Click <strong>"Add Assignment"
</strong> to get
                                    started.</td>
                            </tr>
                        </tbody>
                    </table>
                </div>

                <!-- Supervisor Notes -->
                <div class="wf-notes-section">
                    <label class="wf-notes-label">Supervisor Notes</label>
                    <textarea id="wf-supervisor-notes" class="wf-notes-input" rows="2"
```

```
                        placeholder="General notes for today's roster (e.g., Machine 3 under
maintenance)..."></textarea>
                </div>
            </section>

        </main>
    </div>
</div>

<!-- ============================================
     MODAL: Add/Edit Order (Premium Single-View Design)
     ============================================ -->
<div id="add-order-modal" class="modal hidden">
    <div class="modal-backdrop" onclick="window.adminApp.closeModal('add-order-modal')"></div>
    <div class="modal-content modal-wide">
        <div class="modal-header">
            <h3 class="modal-title">Internal Order</h3>
            <button class="modal-close" onclick="window.adminApp.closeModal('add-order-
modal')">
                <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                        d="M6 18L18 6M6 6l12 12" />
                </svg>
            </button>
        </div>

        <form id="add-order-form">
            <div class="modal-body compact modal-2col-layout">
                <!-- LEFT COLUMN -->
                <div class="modal-col-left">
                    <!-- Order Details -->
                    <div class="w-full">
                        <div class="inline-section-label">Order Details</div>
                        <div class="modal-list-view">
                            <div class="list-row list-row-2col">
                                <div class="form-group">
                                    <label class="form-label">Order No <span
                                            class="required-indicator">*</span></label>
                                    <input type="text" name="internalOrderNo" class="form-
input" required
                                        placeholder="IO-2026-001">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Order Date <span
                                            class="required-indicator">*</span></label>
                                    <input type="date" name="date" class="form-input"
required>
                                </div>
                            </div>
                            <div class="list-row">
                                <div class="form-group">
                                    <label class="form-label">Drawing No</label>
```

```
                                    <input type="text" name="drawingNo" class="form-input"
placeholder="Drg No">
                                </div>
                            </div>
                            <div class="list-row">
                                <div class="form-group">
                                    <label class="form-label">Description</label>
                                    <input type="text" name="description" class="form-input"
                                        placeholder="Enter order description...">
                                </div>
                            </div>
                        </div>
                    </div>

                    <!-- Pricing & Production -->
                    <div class="w-full">
                        <div class="inline-section-label">Pricing & Production</div>
                        <div class="modal-list-view">
                            <div class="list-row list-row-3col">
                                <div class="form-group">
                                    <label class="form-label">Order Qty</label>
                                    <input type="number" name="qty" class="form-input" min="0"
id="order-qty">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Unit</label>
                                    <input type="text" name="qtyUnit" class="form-input"
value="Nos"
                                        list="unit-suggestions" placeholder="e.g. Nos, Kgs">
                                    <datalist id="unit-suggestions">
                                        <option value="Nos">
                                        <option value="Kgs">
                                        <option value="mm">
                                        <option value="Mtrs">
                                        <option value="Sets">
                                        <option value="Lots">
                                    </datalist>
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Department</label>
                                    <select name="department" class="form-input">
                                        <option value="">Select Dept</option>
                                        <option value="Fab">Fab</option>
                                        <option value="CNC">CNC</option>
                                        <option value="VMC">VMC</option>
                                        <option value="Turning">Turning</option>
                                        <option value="Assembly">Assembly</option>
                                    </select>
                                </div>
                            </div>
                            <div class="list-row list-row-2col">
                                <div class="form-group">
```

```
                                    <label class="form-label">Sale Val / Unit</label>
                                    <input type="number" name="saleValueEa" class="form-input"
min="0" step="0.01"
                                        id="order-value">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Total (Sale)</label>
                                    <input type="text" name="total" class="form-input
computed" id="order-total"
                                        readonly>
                                </div>
                            </div>

                            <div class="production-cost-group">
                                <div class="production-cost-label">Target Production
Cost</div>
                                <div class="list-row list-row-2col">
                                    <div class="form-group">
                                        <label class="form-label" style="color: #475569;">In-
house (Unit)</label>
                                        <input type="number" name="prodValueEa" class="form-
input" min="0"
                                            step="0.01" placeholder="In-house per Unit"
id="order-prod-unit">
                                    </div>
                                    <div class="form-group">
                                        <label class="form-label" style="color:
#475569;">Total In-house</label>
                                        <input type="text" name="prodValueTotal" class="form-
input computed"
                                            id="order-prod-total" readonly placeholder="Total
In-house">
                                    </div>
                                </div>
                                <div class="list-row list-row-2col" style="margin-top:
0.5rem;">
                                    <div class="form-group">
                                        <label class="form-label" style="color:
#475569;">Outsource (Unit)</label>
                                        <input type="number" name="outsourceValue"
class="form-input" min="0"
                                            step="0.01" placeholder="Outsource per Unit"
id="order-outsource-unit">
                                    </div>
                                    <div class="form-group">
                                        <label class="form-label" style="color:
#475569;">Total Outsource</label>
                                        <input type="text" name="outsourceValueTotal"
class="form-input computed"
                                            id="order-outsource-total" readonly
placeholder="Total Outsource">
                                    </div>
```

```
                                </div>
                            </div>
                        </div>
                    </div>

                    <!-- RIGHT COLUMN -->
                    <div class="modal-col-right">
                        <!-- Customer & Checks -->
                        <div class="w-full">
                            <div class="inline-section-label">Customer Details</div>
                            <div class="modal-list-view">
                                <div class="form-group">
                                    <label class="form-label">Customer</label>
                                    <input type="text" name="customer" class="form-input"
placeholder="Customer Name">
                                </div>
                                <div class="list-row list-row-2col">
                                    <div class="form-group">
                                        <label class="form-label">PO No</label>
                                        <input type="text" name="poNo" class="form-input"
placeholder="PO #">
                                    </div>
                                    <div class="form-group">
                                        <label class="form-label">PO Date</label>
                                        <input type="date" name="poDate" class="form-input">
                                    </div>
                                </div>
                            </div>

                        </div>

                        <!-- Delivery & Billing -->
                        <div class="w-full">
                            <div class="inline-section-label">Delivery & Billing</div>
                            <div class="modal-list-view">
                                <div class="list-row list-row-2col">
                                    <div class="form-group">
                                        <label class="form-label">Del. Date</label>
                                        <input type="date" name="deliveryDateActual" class="form-
input">
                                    </div>
                                    <div class="form-group">
                                        <label class="form-label">Del. Qty</label>
                                        <input type="number" name="deliveryQty" class="form-input"
min="0">
                                    </div>
                                </div>
                                <div class="list-row list-row-2col">
                                    <div class="form-group">
                                        <label class="form-label">DC No</label>
                                        <input type="text" name="dcNo" class="form-input"
```

```
placeholder="DC #">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Bill No</label>
                                    <input type="text" name="billNo" class="form-input"
placeholder="Invoice #">
                                </div>
                            </div>
                            <div class="list-row">
                                <div class="form-group">
                                    <label class="form-label">Status</label>
                                    <input type="text" class="form-input" value="🟡 Pending"
readonly
                                        style="background:#f8fafc; cursor:default;
color:#94a3b8; font-weight:600;"
                                        id="order-status-display">
                                    <input type="hidden" name="status" value="Pending">
                                </div>
                            </div>
                        </div>
                    </div>

                    <!-- Availability Checks -->
                    <div class="avail-checks">
                        <div class="production-cost-label">Availability</div>
                        <div class="list-row list-row-2col">
                            <label class="check-item">
                                <input type="checkbox" name="drgAvail" value="y">
                                <span>Drg Available</span>
                            </label>
                            <label class="check-item">
                                <input type="checkbox" name="rawAvail" value="y">
                                <span>Raw Mat. Avail</span>
                            </label>
                        </div>
                        <div class="list-row list-row-2col">
                            <label class="check-item">
                                <input type="checkbox" name="isLaborJob" value="y">
                                <span>Labor Job</span>
                            </label>
                            <label class="check-item">
                                <input type="checkbox" name="finishAvail" value="y">
                                <span>Finished Part Avail</span>
                            </label>
                        </div>
                    </div>
                </div>

                <input type="hidden" id="orderId-input" name="orderId">
            </div>

            <div class="modal-footer" style="border-top: 1px solid #e2e8f0; margin-top: 0;
```

```
padding-top: 1.5rem;">
                <label class="check-item"
                    style="display: flex; align-items: center; gap: 0.5rem; cursor: pointer;
margin-right: auto;">
                    <input type="checkbox" id="io-create-project" checked
                        style="width: 16px; height: 16px; accent-color: #0d9488; cursor:
pointer;">
                    <span style="font-size: 0.85rem; font-weight: 600; color: #334155;">Also
create Project in
                        PM</span>
                </label>
                <button type="button" class="btn btn-secondary"
                    onclick="window.adminApp.closeModal('add-order-modal')">Cancel</button>
                <button type="submit" class="btn btn-primary">Save Order</button>
            </div>
        </form>
    </div>
</div>

<!-- ==========================================
     MODAL: Add Delivery Item (Simplified)
     ========================================== -->
<div id="add-delivery-modal" class="modal hidden">
    <div class="modal-backdrop" onclick="window.adminApp.closeModal('add-delivery-modal')">
</div>
    <div class="modal-content">
        <div class="modal-header purple-theme">
            <h3 class="modal-title">Add Delivered Item</h3>
            <button class="modal-close" onclick="window.adminApp.closeModal('add-delivery-
modal')">
                <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                        d="M6 18L18 6M6 6l12 12" />
                </svg>
            </button>
        </div>
        <form id="add-delivery-form" onsubmit="event.preventDefault();
window.adminApp.submitDeliveryForm()">
            <div class="modal-body compact modal-2col-layout">
                <input type="hidden" id="delivery-orderId-input" name="orderId">

                <!-- LEFT COLUMN -->
                <div class="modal-col-left">
                    <!-- Section: Order Lookup -->
                    <div class="w-full">
                        <div class="inline-section-label purple-theme">Order Lookup</div>
                        <div class="modal-list-view">
                            <div class="list-row">
                                <div class="form-group">
                                    <label class="form-label">IO No <span
                                            style="font-weight: 400; color: var(--slate-
400);">(type to
```

```
                                        auto-fill)</span></label>
                                    <input type="text" id="delivery-io-lookup"
name="internalOrderNo"
                                        class="form-input" placeholder="e.g. 202526-530"
                                        list="delivery-io-suggestions"

oninput="window.adminApp.lookupOrderForDelivery(this.value)">
                                    <datalist id="delivery-io-suggestions"></datalist>
                                </div>
                            </div>
                        </div>
                    </div>

                    <!-- Section: Delivery Details -->
                    <div class="w-full">
                        <div class="inline-section-label purple-theme">Delivery Details</div>
                        <div class="modal-list-view">
                            <div class="list-row list-row-2col">
                                <div class="form-group">
                                    <label class="form-label">Date *</label>
                                    <input type="date" name="date" class="form-input"
required>
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Customer *</label>
                                    <input type="text" name="customer" class="form-input"
required
                                        placeholder="Customer Name">
                                </div>
                            </div>
                            <div class="list-row">
                                <div class="form-group">
                                    <label class="form-label">Drawing No</label>
                                    <input type="text" name="drawingNo" class="form-input"
placeholder="Drg No">
                                </div>
                            </div>
                            <div class="list-row">
                                <div class="form-group">
                                    <label class="form-label">Description</label>
                                    <input type="text" name="description" class="form-input"
                                        placeholder="Description">
                                </div>
                            </div>
                        </div>
                    </div>
                </div>

                <!-- RIGHT COLUMN -->
                <div class="modal-col-right">
                    <!-- Section: Values & Logistics -->
                    <div class="w-full">
```

```
                        <div class="inline-section-label purple-theme">Values &
Logistics</div>
                        <div class="modal-list-view">
                            <div class="list-row list-row-3col">
                                <div class="form-group">
                                    <label class="form-label">Quantity</label>
                                    <input type="number" name="qty" class="form-input" min="0"
id="del-qty">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Unit</label>
                                    <input type="text" name="qtyUnit" class="form-input"
value="Nos"
                                        list="unit-suggestions-del" placeholder="e.g. Nos,
Kgs">
                                    <datalist id="unit-suggestions-del">
                                        <option value="Nos">
                                        <option value="Kgs">
                                        <option value="mm">
                                        <option value="Mtrs">
                                        <option value="Sets">
                                        <option value="Lots">
                                    </datalist>
                                </div>
                                <div class="form-group">
                                    <label class="form-label">DC No</label>
                                    <input type="text" name="dcNo" class="form-input"
placeholder="DC Number">
                                </div>
                            </div>
                            <div class="list-row list-row-2col">
                                <div class="form-group">
                                    <label class="form-label">Delivery Value</label>
                                    <input type="number" name="total" class="form-input"
min="0" step="0.01"
                                        id="del-total" placeholder="Total Value">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Labour Cost</label>
                                    <input type="number" name="labourCost" class="form-input"
min="0"
                                        placeholder="Cost">
                                </div>
                            </div>
                        </div>
                    </div>

                    <!-- Section: Department -->
                    <div class="w-full">
                        <div class="inline-section-label purple-theme">Department &
Manpower</div>
                        <div class="modal-list-view">
```

```
                            <div class="list-row list-row-2col">
                                <div class="form-group">
                                    <label class="form-label">Department</label>
                                    <select name="department" class="form-input">
                                        <option value="">Select Dept</option>
                                        <option value="Fab">Fab</option>
                                        <option value="CNC">CNC</option>
                                        <option value="VMC">VMC</option>
                                        <option value="Turning">Turning</option>
                                        <option value="Assembly">Assembly</option>
                                    </select>
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Manpower</label>
                                    <input type="number" name="manpower" class="form-input"
min="0"
                                        placeholder="Daily Value (₹)">
                                </div>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
            <div class="modal-footer">
                <button type="button" class="btn btn-secondary"
                    onclick="window.adminApp.closeModal('add-delivery-modal')">Cancel</button>
                <button type="submit" class="btn btn-purple">Add Delivery</button>
            </div>
        </form>
    </div>
</div>

<!-- ============================================
     MODAL: Add/Edit Member
     ============================================ -->
<div id="add-member-modal" class="modal hidden">
    <div class="modal-backdrop" onclick="window.adminApp.closeModal('add-member-modal')">
</div>
    <div class="modal-content">
        <div class="modal-header">
            <h3 class="modal-title">Add Team Member</h3>
            <button class="modal-close" onclick="window.adminApp.closeModal('add-member-
modal')">
                <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                        d="M6 18L18 6M6 6l12 12" />
                </svg>
            </button>
        </div>
        <form id="add-member-form">
            <div class="modal-body">
                <input type="hidden" id="memberId-input" name="memberId">
```

```
                <div class="modal-list-view">
                    <div class="list-row list-row-2col">
                        <div class="form-group">
                            <label class="form-label">Full Name *</label>
                            <input type="text" name="name" class="form-input" required
                                placeholder="Enter full name">
                        </div>
                        <div class="form-group">
                            <label class="form-label">Employee ID</label>
                            <input type="text" name="employeeId" class="form-input"
placeholder="e.g. EMP001">
                        </div>
                    </div>
                    <div class="list-row list-row-2col">
                        <div class="form-group">
                            <label class="form-label">Email Address</label>
                            <input type="email" name="email" class="form-input"
placeholder="email@company.com">
                        </div>
                        <div class="form-group">
                            <label class="form-label">Phone Number</label>
                            <input type="tel" name="phone" class="form-input" placeholder="+91
98765 43210">
                        </div>
                    </div>
                    <div class="list-row">
                        <div class="form-group">
                            <label class="form-label">Roles/Designations *</label>
                            <!-- Multi-Select Container -->
                            <div class="multi-select-container" id="role-select-container">
                                <div class="select-display form-input"
                                    onclick="window.adminApp.toggleRoleDropdown()">
                                    <span id="role-display-text">Select Roles...</span>
                                    <svg class="w-4 h-4 ml-auto opacity-50" fill="none"
stroke="currentColor"
                                        viewBox="0 0 24 24">
                                        <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
                                            d="M19 9l-7 7-7-7"></path>
                                    </svg>
                                </div>
                                <div id="role-dropdown-options" class="select-options hidden">
                                    <!-- Options populated by JS -->
                                </div>
                                <input type="hidden" name="roles" id="roles-input">
                            </div>
                        </div>
                    </div>
                    <div class="list-row">
                        <div class="form-group">
                            <label class="form-label">Departments *</label>
```

```
                            <!-- Multi-Select Container for Departments -->
                            <div class="multi-select-container" id="dept-select-container">
                                <div class="select-display form-input"
                                    onclick="window.adminApp.toggleDeptDropdown()">
                                    <span id="dept-display-text">Select Departments...</span>
                                    <svg class="w-4 h-4 ml-auto opacity-50" fill="none"
stroke="currentColor"
                                        viewBox="0 0 24 24">
                                        <path stroke-linecap="round" stroke-linejoin="round"
stroke-width="2"
                                            d="M19 9l-7 7-7-7"></path>
                                    </svg>
                                </div>
                                <div id="dept-dropdown-options" class="select-options hidden">
                                    <!-- Options populated by JS -->
                                </div>
                                <input type="hidden" name="departments" id="departments-
input">
                            </div>
                        </div>
                    </div>
                    <div class="list-row list-row-2col">
                        <div class="form-group">
                            <label class="form-label">Reporting Manager</label>
                            <select name="reportingManager" id="manager-select" class="form-
input">
                                <option value="">Select Manager</option>
                                <!-- Populated by JS -->
                            </select>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Status</label>
                            <select name="status" class="form-input">
                                <option value="Active">Active</option>
                                <option value="On Leave">On Leave</option>
                                <option value="Inactive">Inactive</option>
                            </select>
                        </div>
                    </div>
                    <div class="list-row list-row-2col">
                        <div class="form-group">
                            <label class="form-label">Joining Date</label>
                            <input type="date" name="joiningDate" class="form-input">
                        </div>
                        <div class="form-group">
                            <label class="form-label">Daily Overhead (₹)</label>
                            <input type="number" name="overheads" class="form-input"
placeholder="e.g. 500" min="0"
                                step="0.01">
                        </div>
                    </div>
                </div>
```

```
            </div>
            <div class="modal-footer">
                <button type="button" class="btn btn-secondary"
                    onclick="window.adminApp.closeModal('add-member-modal')">Cancel</button>
                <button type="submit" class="btn btn-primary">Save Member</button>
            </div>
        </form>
    </div>
</div>

<!-- ==========================================
     MODAL: Add Project (Regulatory Step-Based)
     ========================================== -->
<div id="add-project-modal" class="modal hidden">
    <div class="modal-backdrop" onclick="window.adminApp.closeModal('add-project-modal')">
</div>
    <div class="modal-content modal-wide">
        <div class="modal-header">
            <h3 class="modal-title">Initialize New Project</h3>
            <button class="modal-close" onclick="window.adminApp.closeModal('add-project-
modal')">
                <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                        d="M6 18L18 6M6 6l12 12" />
                </svg>
            </button>
        </div>

        <form id="add-project-form">
            <div class="modal-body compact modal-2col-layout">
                <!-- LEFT COLUMN: Setup -->
                <div class="modal-col-left">
                    <div class="inline-section-label">Core Metadata</div>
                    <div class="modal-list-view">
                        <div class="form-group">
                            <label class="form-label">Project Name *</label>
                            <input type="text" name="name" class="form-input" required
                                placeholder="e.g. Turbine Shaft Fabrication">
                        </div>
                        <div class="form-group">
                            <label class="form-label">Customer Name *</label>
                            <input type="text" name="customerName" class="form-input" required
                                placeholder="Client Company name">
                        </div>
                        <div class="list-row list-row-2col">
                            <div class="form-group">
                                <label class="form-label">Job Type</label>
                                <select name="jobType" class="form-input">
                                    <option value="CNC">CNC</option>
                                    <option value="VMC">VMC</option>
                                    <option value="Fabrication">Fabrication</option>
                                    <option value="Hybrid">Hybrid</option>
```

```
                                </select>
                            </div>
                            <div class="form-group">
                                <label class="form-label">Drawing Source</label>
                                <select name="drawingSource" class="form-input">
                                    <option value="Customer Supplied">Customer
Supplied</option>
                                    <option value="In-house Design">In-house Design</option>
                                </select>
                            </div>
                        </div>
                    </div>
                </div>

                <!-- RIGHT COLUMN: Targets -->
                <div class="modal-col-right">
                    <div class="inline-section-label">Production Targets</div>
                    <div class="modal-list-view">
                        <div class="form-group">
                            <label class="form-label">Expected Completion Date</label>
                            <input type="date" name="expectedCompletion" class="form-input">
                        </div>
                        <div class="form-group">
                            <label class="form-label">Initial Internal Notes</label>
                            <textarea name="internalNotes" class="form-input" rows="3"
                                placeholder="Production constraints, material
availability..."></textarea>
                        </div>
                    </div>
                </div>
            </div>

            <div class="modal-footer">
                <button type="button" class="btn btn-secondary"
                    onclick="window.adminApp.closeModal('add-project-modal')">Cancel</button>
                <button type="submit" class="btn btn-primary">Initialize Project</button>
            </div>
        </form>
    </div>
</div>

<!-- ============================================
     MODAL: Member Workload & Task Report
     ============================================ -->
<div id="member-workload-modal" class="modal hidden">
    <div class="modal-backdrop" onclick="window.adminApp.closeModal('member-workload-modal')">
</div>
    <div class="modal-content">
        <div class="modal-header">
            <h3 class="modal-title">Member Workload Report</h3>
            <div class="flex items-center gap-2 no-print">
                <button class="btn btn-secondary btn-sm"
```

```
onclick="window.adminApp.printMemberWorkload()">
                    <svg width="16" height="16" fill="none" stroke="currentColor" viewBox="0 0
24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                            d="M17 17h2a2 2 0 002-2v-4a2 2 0 00-2-2H5a2 2 0 00-2 2v4a2 2 0 002
2h2m2 4h6a2 2 0 002-2v-4a2 2 0 00-2-2H9a2 2 0 00-2 2v4a2 2 0 002 2zm8-12V5a2 2 0 00-2-2H9a2 2
0 00-2 2v4h10z" />
                    </svg>
                    Print
                </button>
                <button class="modal-close" onclick="window.adminApp.closeModal('member-
workload-modal')">
                    <svg fill="none" stroke="currentColor" viewBox="0 0 24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                            d="M6 18L18 6M6 6l12 12" />
                    </svg>
                </button>
            </div>
        </div>

        <div class="modal-body">
            <!-- Print Header (Only visible in print) -->
            <div class="workload-print-only">
                <div style="display: flex; justify-content: space-between; align-items:
center;">
                    <h1 style="font-size: 24pt; font-weight: 800; color: #0f172a; margin:
0;">WORKLOAD REPORT</h1>
                    <div style="text-align: right;">
                        <div style="font-weight: 700; color: #1e293b;">Innovative Engineering
Solutions</div>
                        <div style="font-size: 10pt; color: #64748b;" id="workload-print-
date"></div>
                    </div>
                </div>
            </div>

            <!-- Main Info Header -->
            <div class="workload-header-main">
                <div>
                    <div class="flex items-center gap-4 mb-2">
                        <div id="workload-member-avatar"
                            style="width: 56px; height: 56px; border-radius: 50%; background:
#059669; display: flex; align-items: center; justify-content: center; color: white; font-size:
1.5rem; font-weight: 700;">
                            ?
                        </div>
                        <div>
                            <h2 id="workload-member-name"
                                style="font-size: 1.5rem; font-weight: 700; color: #1e293b;
margin: 0;">-</h2>
                            <p id="workload-member-role" style="font-size: 0.875rem; color:
#64748b; margin: 0;">-
```

```
                        </p>
                    </div>
                </div>
            </div>
            <div style="text-align: right;">
                <span class="text-xs font-semibold uppercase tracking-wider text-slate-
400">Department</span>
                <div id="workload-member-dept" style="font-size: 1.25rem; font-weight:
700; color: #334155;">-
                </div>
            </div>
        </div>

        <!-- Task Summary Mini Cards -->
        <div class="grid grid-cols-4 gap-4 mb-6 no-print">
            <div class="card p-4 flex flex-col items-center justify-center text-center">
                <span class="text-xs font-bold text-slate-400 uppercase">Assigned
Total</span>
                <span id="workload-total-tasks" class="text-2xl font-extrabold text-slate-
800">0</span>
            </div>
            <div class="card p-4 flex flex-col items-center justify-center text-center">
                <span class="text-xs font-bold text-amber-500 uppercase">Pending</span>
                <span id="workload-pending-tasks" class="text-2xl font-extrabold text-
amber-600">0</span>
            </div>
            <div class="card p-4 flex flex-col items-center justify-center text-center">
                <span class="text-xs font-bold text-teal-500 uppercase">Completed</span>
                <span id="workload-completed-tasks" class="text-2xl font-extrabold text-
teal-600">0</span>
            </div>
            <div class="card p-4 flex flex-col items-center justify-center text-center">
                <span class="text-xs font-bold text-emerald-500 uppercase">Pending Order
Value</span>
                <span id="workload-pending-value" class="text-2xl font-extrabold text-
emerald-600">₹0</span>
            </div>
        </div>

        <!-- Tasks Table -->
        <div class="table-container">
            <table id="workload-tasks-table">
                <thead>
                    <tr>
                        <th style="width: 100px;">IO No</th>
                        <th>Description</th>
                        <th style="width: 120px;">Drg No</th>
                        <th style="width: 150px;">Customer</th>
                        <th style="width: 80px;" class="text-center">Qty</th>
                        <th style="width: 120px;">Due Date</th>
                        <th style="width: 150px;">Assigned With</th>
                        <th style="width: 100px;" class="text-center">Status</th>
```

```
                        </tr>
                    </thead>
                    <tbody id="workload-tasks-body">
                        <!-- Tasks will be injected here -->
                    </tbody>
                </table>
            </div>
        </div>
        <div class="modal-footer no-print">
            <button class="btn btn-secondary"
                onclick="window.adminApp.closeModal('member-workload-modal')">Close</button>
        </div>
    </div>
</div>




<!-- ============================================
     MODAL: Add Inventory Item
     ============================================ -->
<div id="add-inventory-modal" class="modal hidden">
    <div class="modal-backdrop" onclick="window.adminApp.closeModal('add-inventory-modal')">
</div>
    <div class="modal-content" style="max-width: 500px; padding: 0; overflow: hidden; border-
radius: 16px;">
        <div class="modal-header cr-emerald-header" style="padding: 1.25rem 1.5rem;">
            <h3 class="modal-title"
                style="color: white; font-weight: 800; letter-spacing: 0.025em; display: flex;
align-items: center; gap: 0.75rem;">
                <span>📦</span> ADD NEW INVENTORY ITEM
            </h3>
            <button class="modal-close" onclick="window.adminApp.closeModal('add-inventory-
modal')"
                style="color: white; opacity: 0.8; transition: opacity 0.2s;">
                <svg fill="none" stroke="currentColor" viewBox="0 0 24 24" style="width: 24px;
height: 24px;">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                        d="M6 18L18 6M6 6l12 12" />
                </svg>
            </button>
        </div>
        <form id="add-inventory-form" onsubmit="window.adminApp.handleAddInventoryItem(event)"
            style="background: white;">
            <div class="modal-body" style="padding: 1.5rem; max-height: 70vh; overflow-y:
auto;">
                <div class="space-y-4">
                    <div class="form-group">
                        <label class="form-label"
                            style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Item
```

```
                            Name</label>
                        <input type="text" name="name" class="form-input" required
                            style="height: 48px; border-radius: 10px; border: 1px solid
#e2e8f0; padding: 0 1rem; font-weight: 600;"
                            placeholder="e.g. M20 Stainless Bolt">
                    </div>

                    <div class="form-group">
                        <label class="form-label"
                            style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Unit
                            Price / Cost</label>
                        <input type="number" name="price" class="form-input" step="0.01"
                            style="height: 48px; border-radius: 10px; border: 1px solid
#e2e8f0; padding: 0 1rem; text-align: center; font-weight: 700; color: #059669;"
                            placeholder="0.00">
                    </div>

                    <div class="grid grid-cols-2 gap-4">
                        <div class="form-group">
                            <label class="form-label"
                                style="font-size: 0.7rem; font-weight: 800; color: #475569;
text-transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block;
text-align: center;">Category</label>
                            <select name="category" class="form-input"
                                style="height: 48px; border-radius: 10px; font-weight: 600;"
required

onchange="window.adminApp.onInventoryCategoryChange(this.value)">
                                <option value="Raw Material">Raw Material</option>
                                <option value="Component">Component</option>
                                <option value="Tool">Tool / Equipment</option>
                                <option value="Consumable">Consumable</option>
                                <option value="Fastener">Fastener</option>
                                <option value="Other">Other</option>
                            </select>
                        </div>
                        <div class="form-group">
                            <label class="form-label"
                                style="font-size: 0.7rem; font-weight: 800; color: #475569;
text-transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block;
text-align: center;">Unit</label>
                            <input type="text" name="unit" class="form-input" required
                                style="height: 48px; border-radius: 10px; border: 1px solid
#e2e8f0; padding: 0 1rem; text-align: center;"
                                placeholder="Nos, Kg, Pkts">
                        </div>
                    </div>

                    <!-- Tool Photo Section (Conditional) -->
                    <div id="inv-photo-section" class="form-group hidden">
```

```
                        <label class="form-label"
                            style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Tool
                            Photo (Max 1MB)</label>
                        <div class="inventory-preview-container"
                            onclick="document.getElementById('inv-photo-input').click()"
                            style="border-radius: 12px; height: 140px; border: 2px dashed
#cbd5e1; background: #f8fafc; cursor: pointer; display: flex; align-items: center; justify-
content: center; overflow: hidden;">
                            <input type="file" id="inv-photo-input" class="hidden"
accept="image/*"
                                onchange="window.adminApp.handleInventoryPhotoSelect(event)">
                            <div id="inv-photo-preview-placeholder" class="text-center">
                                <span style="font-size: 2rem;">📸</span>
                                <p class="text-[10px] text-slate-400 mt-2 font-bold uppercase
tracking-wider">Click
                                    to upload photo</p>
                            </div>
                            <img id="inv-photo-preview" class="inventory-preview-img hidden"
                                style="width: 100%; height: 100%; object-fit: cover;">
                        </div>
                    </div>

                    <div class="grid grid-cols-2 gap-4">
                        <div class="form-group">
                            <label class="form-label"
                                style="font-size: 0.7rem; font-weight: 800; color: #475569;
text-transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block;
text-align: center;">Initial
                                Stock</label>
                            <input type="number" name="currentStock" class="form-input"
                                style="height: 48px; border-radius: 10px; text-align: center;
font-weight: 700;"
                                value="0" min="0">
                        </div>
                        <div class="form-group">
                            <label class="form-label"
                                style="font-size: 0.7rem; font-weight: 800; color: #475569;
text-transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block;
text-align: center;">Min
                                level (Alert)</label>
                            <input type="number" name="minimumLevel" class="form-input"
                                style="height: 48px; border-radius: 10px; text-align: center;
font-weight: 700; color: #ef4444;"
                                value="5" min="0">
                        </div>
                    </div>

                    <div class="form-group">
                        <label class="form-label"
                            style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
```

```
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Initially
                            Link to Order ID</label>
                        <select name="orderId" class="form-input"
                            style="height: 48px; border-radius: 10px; font-weight: 600;">
                            <option value="">-- No Order ID --</option>
                            <!-- Will be populated by JS -->
                        </select>
                    </div>

                    <div class="form-group">
                        <label class="form-label"
                            style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Storage
                            Location (Bin/Rack)</label>
                        <input type="text" name="location" class="form-input"
                            style="height: 48px; border-radius: 10px; border: 1px solid
#e2e8f0; padding: 0 1rem;"
                            placeholder="e.g. Rack A-04">
                    </div>
                </div>
            </div>
            <div class="modal-footer"
                style="padding: 1.25rem 1.5rem; background: #f8fafc; border-top: 1px solid
#e2e8f0; display: flex; justify-content: flex-end; gap: 1rem;">
                <button type="button" class="btn btn-secondary"
                    onclick="window.adminApp.closeModal('add-inventory-modal')"
                    style="height: 44px; padding: 0 1.5rem; border-radius:
10px;">Cancel</button>
                <button type="submit" class="btn btn-primary"
                    style="height: 44px; padding: 0 2rem; border-radius: 10px; font-weight:
700;">Add Item</button>
            </div>
        </form>
    </div>
</div>

<!-- ============================================
     MODAL: Adjust Stock (In/Out)
     ============================================ -->
<div id="adjust-stock-modal" class="modal hidden">
    <div class="modal-backdrop" onclick="window.adminApp.closeModal('adjust-stock-modal')">
</div>
    <div class="modal-content" style="max-width: 450px; padding: 0; overflow: hidden; border-
radius: 16px;">
        <div class="modal-header cr-emerald-header" style="padding: 1.25rem 1.5rem;">
            <h3 class="modal-title"
                style="color: white; font-weight: 800; letter-spacing: 0.025em; display: flex;
align-items: center; gap: 0.75rem;">
                <span>🔄</span> ADJUST STOCK LEVEL
            </h3>
```

```
            <button class="modal-close" onclick="window.adminApp.closeModal('adjust-stock-
modal')"
                style="color: white; opacity: 0.8;">
                <svg fill="none" stroke="currentColor" viewBox="0 0 24 24" style="width: 24px;
height: 24px;">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                        d="M6 18L18 6M6 6l12 12" />
                </svg>
            </button>
        </div>
        <form id="adjust-stock-form" onsubmit="window.adminApp.handleAdjustStock(event)"
style="background: white;">
            <input type="hidden" name="itemId" id="adjust-item-id">
            <input type="hidden" name="itemName" id="adjust-item-name">
            <div class="modal-body" style="padding: 1.5rem;">
                <div id="adjust-item-display"
                    style="margin-bottom: 1.5rem; padding: 1.25rem; background: #ffffff;
border-radius: 12px; border: 1px solid #e2e8f0; box-shadow: 0 2px 4px rgba(0,0,0,0.02);">
                    <div
                        style="font-size: 0.65rem; font-weight: 800; color: #94a3b8; text-
transform: uppercase; letter-spacing: 0.075em; margin-bottom: 0.5rem;">
                        Item Selected</div>
                    <div class="font-bold text-slate-800" id="adjust-item-name-text"
                        style="font-size: 1.125rem; line-height: 1.2;">-</div>
                    <div style="font-size: 0.8125rem; color: #64748b; margin-top:
0.25rem;">Current Stock: <span
                            id="adjust-current-stock-text" class="font-bold text-teal-
600">0</span></div>
                </div>

                <div class="grid grid-cols-2 gap-4 mb-5">
                    <div class="form-group">
                        <label class="form-label"
                            style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Action
                            Type</label>
                        <select name="type" class="form-input"
                            style="height: 44px; border-radius: 10px; font-weight: 600;"
required
                            onchange="window.adminApp.onStockActionChange(this.value)">
                            <option value="IN">ADD (+ Stock In)</option>
                            <option value="OUT">USE (- Stock Out)</option>
                        </select>
                    </div>
                    <div class="form-group">
                        <label class="form-label"
                            style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Quantity</label>
                        <input type="number" name="quantity" class="form-input"
                            style="height: 44px; border-radius: 10px; text-align: center;
```

```
font-weight: 700; font-size: 1rem;"
                            min="1" required>
                    </div>
                </div>

                <div class="grid grid-cols-2 gap-4 mb-5">
                    <div class="form-group">
                        <label class="form-label"
                            style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Cost
                            per Unit</label>
                        <input type="number" name="price" step="0.01" class="form-input"
                            style="height: 44px; border-radius: 10px; text-align: center;
font-weight: 700; color: #059669;"
                            placeholder="0.00">
                    </div>
                    <div class="form-group">
                        <label class="form-label"
                            style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Performed
                            By</label>
                        <input type="text" name="performedBy" class="form-input"
                            style="height: 44px; border-radius: 10px; padding: 0 1rem;"
placeholder="Name/Inits">
                    </div>
                </div>

                <div class="form-group mb-5">
                    <label class="form-label"
                        style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Reason
                        / Reference</label>
                    <input type="text" name="reason" class="form-input"
                        style="height: 44px; border-radius: 10px; padding: 0 1rem;"
                        placeholder="e.g. Purchase Invoice #1234, Maintenance">
                </div>

                <!-- Order ID Linking (Conditional) -->
                <div id="adjust-project-section" class="form-group hidden">
                    <label class="form-label"
                        style="font-size: 0.7rem; font-weight: 800; color: #475569; text-
transform: uppercase; letter-spacing: 0.05em; margin-bottom: 0.5rem; display: block; text-
align: center;">Link
                        to Order ID (Optional)</label>
                    <select name="orderId" id="adjust-project-select" class="form-input"
                        style="height: 44px; border-radius: 10px;">
                        <option value="">-- No Order ID --</option>
                        <!-- Order IDs will be injected here -->
                    </select>
```

```
                    <p
                        style="font-size: 0.65rem; color: #94a3b8; margin-top: 0.5rem; font-
style: italic; text-align: center;">
                        Tracks material usage against a specific Order.</p>
                </div>
            </div>
            <div class="modal-footer"
                style="padding: 1.25rem 1.5rem; background: #f8fafc; border-top: 1px solid
#e2e8f0; display: flex; justify-content: flex-end; gap: 1rem;">
                <button type="button" class="btn btn-secondary"
                    onclick="window.adminApp.closeModal('adjust-stock-modal')"
                    style="height: 44px; padding: 0 1.25rem; border-radius:
10px;">Cancel</button>
                <button type="submit" class="btn btn-primary" id="adjust-stock-submit"
                    style="height: 44px; padding: 0 1.5rem; border-radius: 10px; font-weight:
700;">Update
                    Stock</button>
            </div>
        </form>
    </div>
</div>


<!-- Confirmation Modal -->
<div id="confirm-modal" class="modal">
    <div class="modal-overlay" onclick="window.adminApp.closeModal('confirm-modal')"></div>
    <div class="modal-content" style="max-width: 400px; text-align: center; padding: 2rem;">
        <div style="margin-bottom: 1rem; color: #ef4444;">
            <svg width="48" height="48" fill="none" stroke="currentColor" viewBox="0 0 24 24"
                style="margin: 0 auto;">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                    d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-3L13.732
4c-.77-1.333-2.694-1.333-3.464 0L3.34 16c-.77 1.333.192 3 1.732 3z">
                </path>
            </svg>
        </div>
        <h3 id="confirm-title" style="margin-bottom: 0.5rem; font-size: 1.25rem;">Are you
sure?</h3>
        <p id="confirm-message" style="color: #64748b; margin-bottom: 1.5rem;">This action
cannot be undone.</p>
        <div style="display: flex; gap: 1rem; justify-content: center;">
            <button class="btn btn-secondary" onclick="window.adminApp.closeModal('confirm-
modal')">Cancel</button>
            <button id="confirm-yes-btn" class="btn btn-primary"
                style="background: #ef4444 !important; border-color: #ef4444;">Yes,
Proceed</button>
        </div>
    </div>
</div>

<!-- ============================================
     MODAL: Inventory Image Viewer
```

```
     ========================================== -->
<div id="inventory-image-viewer" class="modal hidden" style="z-index: 10000;">
    <div class="modal-backdrop" onclick="window.adminApp.closeModal('inventory-image-viewer')"
        style="background: rgba(0,0,0,0.9);"></div>
    <div class="modal-content"
        style="max-width: 90vw; max-height: 90vh; background: transparent; box-shadow: none;
border: none; padding: 0; display: flex; flex-direction: column; align-items: center;
overflow: visible !important;">
        <div id="inventory-viewer-title"
            style="color: white; font-size: 1.25rem; font-weight: 700; margin-bottom: 1rem;
text-shadow: 0 2px 4px rgba(0,0,0,0.5); text-align: center;">
        </div>
        <button class="modal-close" onclick="window.adminApp.closeModal('inventory-image-
viewer')"
            style="position: absolute; top: 10px; right: 10px; color: white; background:
rgba(255,255,255,0.25); border-radius: 50%; width: 44px; height: 44px; display: flex; align-
items: center; justify-content: center; backdrop-filter: blur(12px); border: 1px solid
rgba(255,255,255,0.4); z-index: 100; transition: all 0.2s;">
            <svg fill="none" stroke="currentColor" viewBox="0 0 24 24" style="width: 28px;
height: 28px;">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2.5" d="M6
18L18 6M6 6l12 12" />
            </svg>
        </button>
        <img id="inventory-viewer-img" src="" alt="Inventory Detail"
            style="max-width: 100%; max-height: 85vh; object-fit: contain; border-radius: 8px;
box-shadow: 0 20px 50px rgba(0,0,0,0.5);">
    </div>
</div>

<!-- Application Script -->

<script type="module" src="assets/admin/js/app.js?v=12"></script>
```
\n```\n